TRƯỜNG ĐẠI HỌC PHẠM VĂN ĐỒNG

KHOA CÔNG NGHỆ THÔNG TIN

BÀI GIẢNG:

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU 2
SQL SERVER

(Dùng cho bậc Cao đẳng)

Giảng viên: Th.S *Nguyễn Trí Nhân*

Quảng Ngãi, Spring-2014

MỤC LỤC

LỜI NÓI ĐẦU 5
Chương 1. TỔNG QUAN VỀ SQL SERVER 2000 7
1.1. GIỚI THIỆU SQL SERVER 7
1.1.1. Giới thiệu chung về hệ quản trị cơ sở dữ liệu 7
1.1.2. Giới thiệu về phiên bản SQL Server 7
1.2. Các thành phần trong SQL Server 8
1.3. CÀI ĐẶT SQL SERVER 9
1.4. CÁC CSDL HỆ THỐNG TRONG SQL SERVER 17
1.5. CÁC CÔNG CỤ TRONG SQL SERVER 19
1.5.1. Tiện ích Book Online 20
1.5.2. Tiện ích Client NetWork Utility 20
1.5.3. Tiện ích Enterprise Manager 21
1.5.4. Tiện ích Import and Export Data 22
1.5.5. Tiện ích Profiler 22
1.5.6. Tiện ích Query Analyzer 23
1.5.7. Tiện ích Server Network Utility 25
1.5.8. Tiện ích Service Manager 25
Chương 2. GIỚI THIỆU NGÔN NGỮ TRANSACT – SQL 27
2.1. GIỚI THIỆU 27
2.2. KIỂU DỮ LIỆU 27
2.3. TRUY XUẤT DỮ LIỆU (DATA QUERY LANGUAGE) 28
2.3.1. Truy vấn dữ liệu đơn giản với SELECT 28
2.3.2. Tìm hiểu các trường tính toán 31
2.3.3. Lọc dữ liệu với mệnh đề WHERE 33
2.3.4. Sắp xếp dữ liệu với ORDER BY 38
2.3.5. Nhóm dữ liệu với mệnh đề GROUP BY 40
2.3.6. Query con 44
2.3.7. Các phép nối 46
2.3.8. Kết hợp các query 52
2.4. CẬP NHẬT DỮ LIỆU (DATA MANIPULATION LANGUAGE) 53

2.4.1. Chèn dữ liệu 53
2.4.2. Cập nhật, xóa dữ liệu 55
2.5. BÀI TẬP THỰC HÀNH 56
Chương 3. TẠO CƠ SỞ DỮ LIỆU TRÊN SQL SERVER 57
3.1. CÁC LOẠI FILE LƯU TRỮ CƠ SỞ DỮ LIỆU 57
3.1.1. Các tập tin vật lý lưu trữ cơ sở dữ liệu 57
3.1.2. Tập tin dữ liệu chính (Primary Data File) 57
3.1.3. Tập tin dữ liệu thứ yếu (Secondary Data Files) 58
3.1.4. Tập tin lưu vết (Log Files) 58
3.2. TẠO CƠ SỞ DỮ LIỆU 58
3.2.1. Sử dụng tiện ích Enterprise Manager: 58
3.2.2. Sử dụng câu lệnh Transaction - SQL: 61
3.3. XÓA, SỬA, ĐỔI TÊN, XEM THÔNG TIN CƠ SỞ DỮ LIỆU 62
3.3.1. Xóa cơ sở dữ liệu 62
3.3.2. Đổi tên Database (Rename) 64
Chương 4. TẠO VÀ QUẢN LÝ BẢNG (TABLE) 65
4.1. TẠO BẢNG (TABLE) 65
4.1.1. Khái niệm về bảng 65
4.1.2. Tạo cấu trúc bảng dữ liệu 65
4.2. THAY ĐỔI CẤU TRÚC BẢNG (TABLE) 69
4.2.1. Thêm một cột mới trong bảng: 69
4.2.2. Hủy bỏ cột hiện có bên trong bảng: 70
4.2.3. Sửa đổi kiểu dữ liệu của cột: 70
4.2.4. Tắt bỏ quy tắc kiểm tra toàn vẹn dữ liệu: 71
4.2.5. Bật lại quy tắc kiểm tra toàn vẹn dữ liệu: 71
4.2.6. Đổi tên cột, tên bảng dữ liệu: 72
4.3. XÓA BẢNG (TABLE) 73
4.4. QUẢN LÝ BẢNG 73
4.4.1. Tính toàn vẹn dữ liệu trong cơ sở dữ liệu 73
4.5. TẠO MỘT DATABASE DIAGRAM 88
4.5.1. Khái niệm về mô hình quan hệ dữ liệu: 88

4.5.2. Tạo mới mô hình quan hệ dữ liệu: 88
4.6. BÀI TẬP THỰC HÀNH 92
Chương 5. TẠO VIEW (BẢNG ẢO) 97
5.1. KHÁI NIỆM VIEW 97
5.2. SỬ DỤNG T – SQL 97
5.3. SỬ DỤNG ENTERPRISE MANAGER 102
5.4. BÀI TẬP THỰC HÀNH 105
Chương 6. STORE PROCEDURE 106
6.1. KHÁI NIỆM STORE PROCEDURE 106
6.1.1. Thủ tục nội là gì ? 106
6.1.2. Các **Procdure Store** hệ thống: 106
6.1.3. Các lợi ích khi sử dụng **Procdure Store**: 107
6.2. TẠO STORE PROCEDURE BẰNG LỆNH CREATE 107
6.3. THI HÀNH STORE PROCEDURE 110
6.4. CÁC PHÁT BIỂU ĐIỀU KHIỂN 110
6.4.1. Biểu thức Case 110
6.4.2. Cấu trúc rẽ nhánh IF... ELSE 114
6.4.3. Cấu trúc lặp WHILE 117
6.5. SỬA, XÓA, ĐỔI TÊN THỦ TỤC (STORE PROCEDURE) 121
6.5.1. Hủy bỏ **Procdure Store**: 121
6.5.2. Thay đổi nội dung của **Procdure Store**: 121
6.6. XỬ LÝ LỖI TRONG STORE PROCEDURE (SP) 123
6.6.1. Mã trả về 123
6.6.2. Câu lệnh RAISERROR 124
6.7. BÀI TẬP THỰC HÀNH 125
Chương 7. CHUYỂN TÁC, BẪY LỖI VÀ KIỂU CON TRỎ TRONG SQL SERVER 126
7.1. CHUYỂN TÁC (TRANSACTION) 126
7.1.1. Khái niệm 126
7.1.2. Cách tạo và ứng dụng 126
7.2. BẪY LỖI (TRIGGER) 133

7.2.1. Khái niệm 133
7.2.2. Cách tạo và ứng dụng 133
7.3. KIỂU CON TRỎ (SQL CURSOR) 134
7.3.1. Khái niệm 134
7.3.2. Cách tạo và ứng dụng 134
7.4. BÀI TẬP THỰC HÀNH 136
Chương 8. QUẢN LÝ BẢO MẬT VÀ NGƯỜI DÙNG TRONG SQL SERVER ... 137
8.1. KHÁI NIỆM 137
8.2. TẠO VÀ QUẢN LÝ NGƯỜI DÙNG ĐĂNG NHẬP 137
8.2.1. Sử dụng Enterprise Manager 137
8.2.2. Sử dụng Query Analyzer: 139
8.3. CẤP PHÁT QUYỀN CHO NGƯỜI DÙNG 139
8.3.1. Sử dụng Enterprise Manager 139
8.3.2. Sử dụng Query Analyzer 140
8.4. THU HỒI QUYỀN ĐÃ CẤP PHÁT CHO NGƯỜI DÙNG 142
8.4.1. Xóa quyền truy xuất CSDL hiện hành: 142
8.4.2. Xóa bỏ quyền thực thi đã cấp hoặc từ chối trên CSDL: 142
8.4.3. Xóa bỏ quyền tạo đối tượng đã cấp hoặc từ chối trên CSDL: 143
TÀI LIỆU THAM KHẢO 144

LỜI NÓI ĐẦU

Ngôn ngữ hỏi có cấu trúc (SQL), có tiền thân là SEQUEL, là một ngôn ngữ được IBM phát triển và sử dụng trong hệ cơ sở dữ liệu thử nghiệm có tên là System/R vào năm 1974, chính thức được ANSI/ISO công nhận là một chuẩn ngôn ngữ sử dụng trong cơ sở dữ liệu quan hệ vào năm 1986. Cho đến hiện nay, SQL đã được sử dụng phổ biến trong các hệ quản trị cơ sở dữ liệu thương mại và có vai trò quan trọng trong những hệ thống này.

Được sự động viên của các đồng nghiệp trong Khoa Công nghệ Thông tin, Trường Đại học Phạm Văn Đồng, chúng tôi mạnh dạn viết và giới thiệu Bài giảng Hệ quản trị cơ sở dữ liệu 2 SQL Server cho sinh viên bậc Cao đẳng ngành Công nghệ thông tin đến bạn đọc. Trong bài giảng này, chúng tôi không có tham vọng đề cập đến mọi khía cạnh của SQL mà chỉ mong muốn rằng đây sẽ là tài liệu tham khảo tương đối đầy đủ về các câu lệnh thường được sử dụng trong SQL. Bài giảng được chia thành tám chương với nội dung như sau:

- Chương 1 TỔNG QUAN VỀ SQL SERVER 2000
- Chương 2 GIỚI THIỆU NGÔN NGỮ TRANSACT – SQL
- Chương 3 TẠO CƠ SỞ DỮ LIỆU TRÊN SQL SERVER
- Chương 4 TẠO VÀ QUẢN LÝ BẢNG (TABLE)
- Chương 5 TẠO VIEW (BẢNG ẢO)
- Chương 6 STORE PROCEDURE
- Chương 7 CHUYỂN TÁC, BẪY LỖI VÀ KIỂU CON TRỎ TRONG SQL SERVER
- Chương 8 QUẢN LÝ BẢO MẬT VÀ NGƯỜI DÙNG TRONG SQL SERVER

So với chuẩn SQL do ANSI/ISO đề xuất, bản thân các hệ quản trị cơ sở dữ liệu quan hệ thương mại lại có thể có một số thay đổi nào đó; Điều này đôi khi dẫn đến sự khác biệt, mặc dù không đáng kể, giữa SQL chuẩn và SQL được sử dụng trong các hệ quản trị cơ sở dữ liệu cụ thể. Trong bài giảng này, chúng tôi chọn hệ quản trị cơ sở dữ liệu SQL Server 2000 của hãng Microsoft để sử dụng cho các ví dụ minh hoạ cũng như lời giải của các bài tập.

Chúng tôi hi vọng rằng bài giảng này sẽ thực sự có ích đối với bạn đọc. Chúng tôi rất mong nhận được sự cổ vũ và những ý kiến đóng góp thẳng thắn của các bạn. Cuối cùng, xin gởi lời cảm ơn đến các thầy cô, đồng nghiệp và các bạn sinh viên đã động viên và giúp đỡ chúng tôi hoàn thành bài giảng này.

Quảng Ngãi, 2014

Nguyễn Trí Nhân

Chương 1. TỔNG QUAN VỀ SQL SERVER 2000

1.1. GIỚI THIỆU SQL SERVER

1.1.1. Giới thiệu chung về hệ quản trị cơ sở dữ liệu

Hệ quản trị cơ sở dữ liệu (tiếng Anh: Database Management System - DBMS), là phần mềm hay hệ thống được thiết kế để quản trị một cơ sở dữ liệu. Cụ thể, các chương trình thuộc loại này hỗ trợ khả năng lưu trữ, sửa chữa, xóa và tìm kiếm thông tin trong một cơ sở dữ liệu (CSDL). Có rất nhiều loại hệ quản trị CSDL khác nhau: từ phần mềm nhỏ chạy trên máy tính cá nhân cho đến những hệ quản trị phức tạp chạy trên một hoặc nhiều siêu máy tính.

Tuy nhiên, đa số hệ quản trị CSDL trên thị trường đều có một đặc điểm chung là sử dụng ngôn ngữ truy vấn theo cấu trúc mà tiếng Anh gọi là Structured Query Language (SQL). Các hệ quản trị CSDL phổ biến được nhiều người biết đến là MySQL, Oracle, PostgreSQL, SQL Server, DB2, Infomix, v.v. Phần lớn các hệ quản trị CSDL kể trên hoạt động tốt trên nhiều hệ điều hành khác nhau như Linux, Unix và MacOS ngoại trừ SQL Server của Microsoft chỉ chạy trên hệ điều hành Windows.

- **Ưu điểm của HQTCSDL**:
  + Quản lý được dữ liệu dư thừa.
  + Đảm báo tính nhất quán cho dữ liệu.
  + Tạo khả năng chia sẻ dữ liệu nhiều hơn.
  + Cải tiến tính toàn vẹn cho dữ liệu.
- **Nhược điểm**:
  + HQTCSDL tốt thì khá phức tạp.
  + HQTCSDL tốt thường rất lớn chiếm nhiều dung lượng bộ nhớ.
  + Giá cả khác nhau tùy theo môi trường và chức năng.
  + HQTCSDL được viết tổng quát cho nhiều người dùng thì thường chậm.

1.1.2. Giới thiệu về phiên bản SQL Server

Microsoft SQL Server là một hệ quản trị cơ sở dữ liệu quan hệ (relational database management system – RDBMS) do Microsoft phát triển.

SQL Server là một hệ quản trị cơ sở dữ liệu quan hệ mạng máy tính hoạt động theo mô hình khách chủ cho phép đồng thời cùng lúc có nhiều người dùng truy xuất đến dữ liệu, quản lý việc truy nhập hợp lệ và các quyền hạn của từng người dùng trên mạng.

Ngôn ngữ truy vấn quan trọng của Microsoft SQL server là Transact-SQL

Lần lượt các phiên bản của Microsoft SQL Server đã ra đời sau sự kiện này, từ 4.2 sau đó được nâng cấp thành 4.21, 6.0, 6.5, 7.0 và hiện giờ là Microsoft SQL Server 2000, 2005 và mới nhất là Microsoft SQL Server 2008, Microsoft SQL Server 2012, Microsoft SQL Server 2013 .

- SQL Server 2005: SQL Server 2005, được phát hành vào tháng 11 năm 2005, là phiên bản tiếp theo của SQL Server 2000. Sử dụng trên nền Net Framework 2.0

- SQL Server 2008: Đây là phiên bản mới nhất của SQl Server. Ngày 27/02/2008, sử dụng trên nền Net Framework 3.0.

- SQL Server 2012: Ngày 6/3/2012, tập đoàn Microsoft tuyên bố phiên bản mới nhất Microsoft SQL Server 2012, nền tảng dữ liệu được sử dụng nhiều nhất trên thế giới, sẵn. SQL Server 2012 giúp giải quyết các thách thức liên quan đến sự gia tăng khối lượng dữ liệu bằng cách chuyển các dữ liệu này thành các tầm nhìn mang tính quyết định nhanh chóng, giúp khách hàng quản lý bất kì dữ liệu nào, với bất kì kích thước nào, tại cả cơ sở và trên điện toán đám mây. SQL Server 2012 đưa đến một nền tảng với các công cụ quen thuộc để quản lý dữ liệu dù ở định dạng nào, kích thước lớn tới đâu.

1.2. Các thành phần trong SQL Server

SQL Server được cấu thành bởi nhiều thành phần khác nhau, các thành phần có mối quan hệ trong một hệ thống, phối hợp với nhau để tạo thành một giải pháp hoàn chỉnh, nâng cao hiệu quả quản trị, phân tích, lưu trữ dữ liệu:

- ***SQL Server Database:*** Là cỗ máy cơ sở dữ liệu bao gồm Database Engine, lõi dịch vụ cho việc lưu trữ, xử lý và bảo mật dữ liệu, sao lưu và đồng bộ (Replication), tìm kiếm toàn văn (Full-Text Search) và các công cụ cho việc quản trị dữ liệu quan hệ và XML.

- ***Analysis services****:* Bao gồm các công cụ cho việc tạo và quản lý tiến trình phân tích trực tuyến (online analytical processing – OLAP) và các ứng dụng khai thác dữ liệu.

- ***Reporting Services****:* Bao gồm các thành phần server và client cho việc tạo, quản lý và triển khai các báo cáo. Reporting Services cũng là nền tảng cho việc phát triển và xây dựng các ứng dụng báo cáo.

- ***Notification Services****:* Là nền tảng cho sự phát triển và triển khai các ứng dụng tạo và gửi thông báo. Notification Services có thể gửi thông báo theo đích thời đến hàng ngàn người đăng ký sử dụng nhiều loại thiết bị.

- ***Integration Services****:* Là một tập hợp các công cụ đồ họa và các đối tượng lập trình cho việc di chuyển, sao chép và chuyển đổi dữ liệu.

- ***Connectivity Components****:* Là các thành phần cho việc truyền thông giữa client và server, và các thư viện mạng như DB-Library, ODBC và OLEDB.

- ***SQL Server Management Studio (SSMS)****:* Là môi trường được tích hợp cho việc truy xuất, cấu hành, quản trị và phát triển tất cả các thành phần của SQL Server. SSMS kết hợp tất cả các tính năng của Enterprise Manager, Query Analyzer và Analysis Manager, được bao hàm trong các phiên bản trước của SQL Server, thành một môi trường đơn mà cung cấp truy xuất SQL Server để phát triển và quản trị tất cả các mức kỹ năng trên.

- ***Server SQL Configuration Manager****:* Cung cấp các quản trị cấu hình cơ sở cho các dịch vụ SQL Server (SQL Server services), các giao thức server (server protocols), các giao thức client (client protocols) và các bí danh client (client aliases).

- ***SQL Server Profiler****:* Cung cấp giao diện người dùng đồ họa cho việc giám sát thể hiện của Database Engine hoặc thể hiện của Analysis Services.

- ***Database Engine Tuning Advisor****:* giúp tạo các tập tối ưu chỉ số (indexes), indexed views và các phân vùng (partitions).

1.3. CÀI ĐẶT SQL SERVER

Khác với một số phần mềm khác như Microsoft Office, Visual Studio, Delphi,... việc cài đặt Microsoft SQL Server không đơn giản. Do đó, sẽ hướng dẫn bạn các bước để tự cài đặt cơ sở dữ liệu Microsoft SQL Server Personal.

Đây là phiên bản làm việc trong môi trường **Windows Me**, **Windows 98**, **Windows NT Workstation 4.0** hoặc **Windows 2000 Professional**. Bạn có thể cài đặt và sử dụng **Microsoft SQL Server** trên máy tính đơn (**Stand-Alone**) mà không cần phải cài đặt **Windows NT Server** hoặc không cần trang bị một hệ thống mạng máy tính.

**** Cấu hình máy tối thiểu để cài đặt SQL Server 2000:***

- CPU: Pentium 166 Mhz hoặc Pentium Pro.

- RAM: 64MB cho Windows 2000 Professional (128MB thì tốt hơn).

- Dung lượng đĩa trống: 95MB – 270MB với phiên bản đầy đủ (Full); 50MB với phiên bản tối thiểu (Minimun); 130MB với phiên bản thường dùng (Typical).

- Hệ điều hành: Windows Me, Windows 98, Windows NT Workstation 4.0, Windows 2000 Professional.

- Màn hình: VGA với độ phân giải 800 x 600 hoặc cao hơn.

- Internet Browser: Microsoft Internet Explorer 5.0 trở lên.

*** Các bước thực hiện việc cài đặt:**

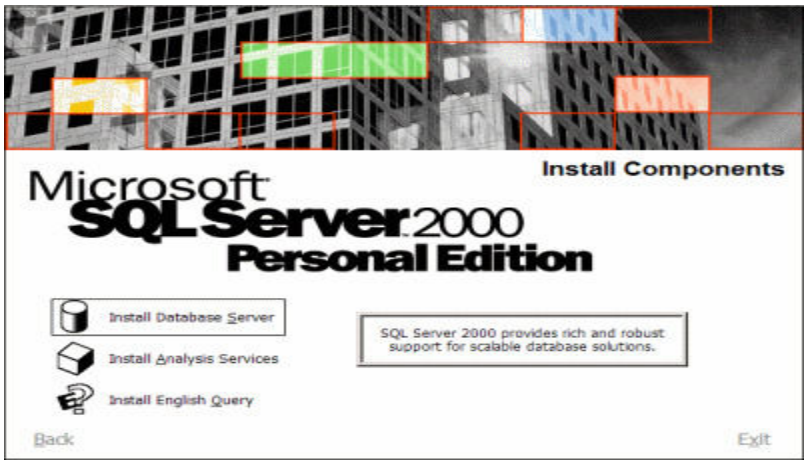
Bước 1: Đưa đĩa **Microsoft SQL Server Personal** vào ổ đĩa **CDROM**. Nếu ổ đĩa không tự động chạy, bạn double click chuột vào tập tin **autorun.exe** để khởi động chương trình cài đặt.

Bước 2: Trong màn hình khởi động cài đặt **Microsoft SQL Server Personal**, bạn chọn mục **SQL Server 2000 Components** để bắt đầu việc cài đặt.



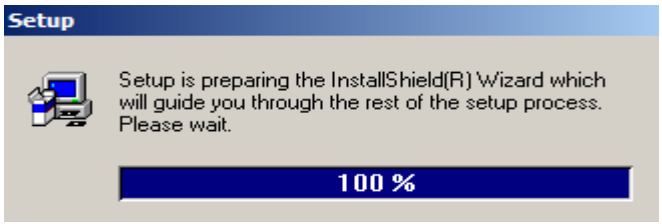
Hình 1.1. Màn hình khởi động cài đặt SQL

Bước 3: Bạn chọn chức năng **Install Database Server** để cài đặt cơ sở dữ liệu **Microsoft SQL Server Personal**.

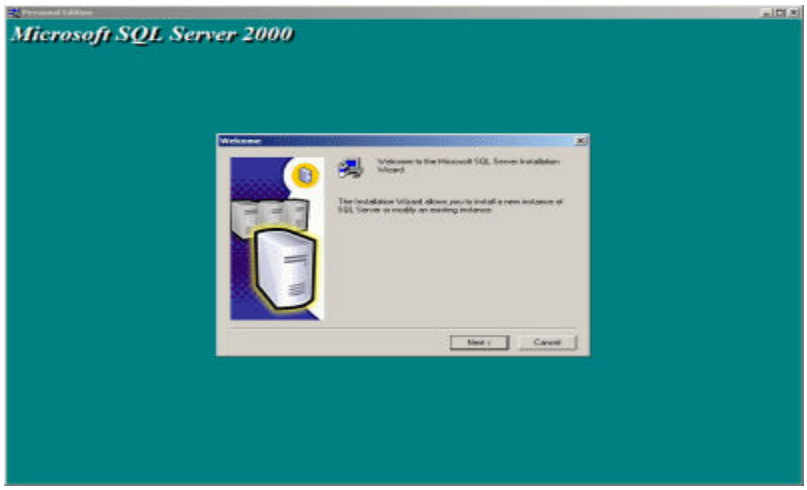


Hình 1.2. Màn hình chọn thành phần cài đặt SQL

Đối với hai thành phần còn lại, được dùng để cài đặt các thành phần liên quan đến việc xử lý phân tích dữ liệu trực tuyến (**OLAP – Online Analytical Processing**) và xử lý truy vấn dữ liệu bằng ngôn ngữ tiếng Anh. Hai thành phần này không nhất thiết phải được cài đặt chung với cơ sở dữ liệu **Microsoft SQL Server Personal**.

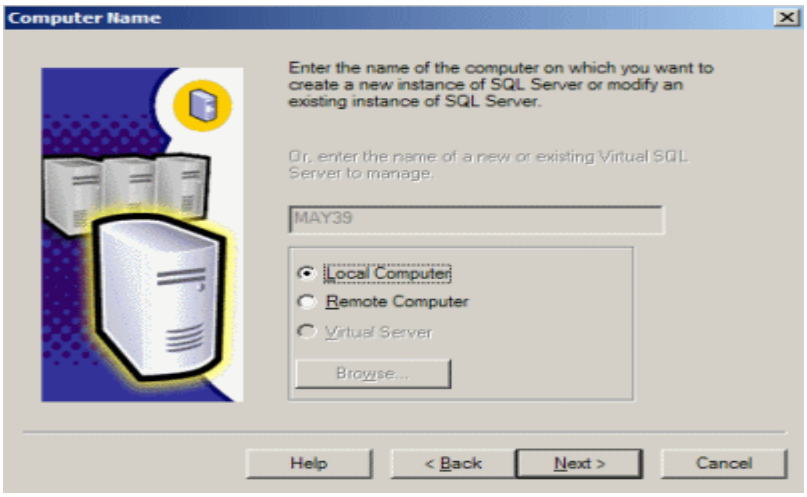


Hình 1.3. Trình cài đặt tiến hành sao chép một số file cần thiết



Hình 1.4. Chọn Next để tiếp tục cài đặt.

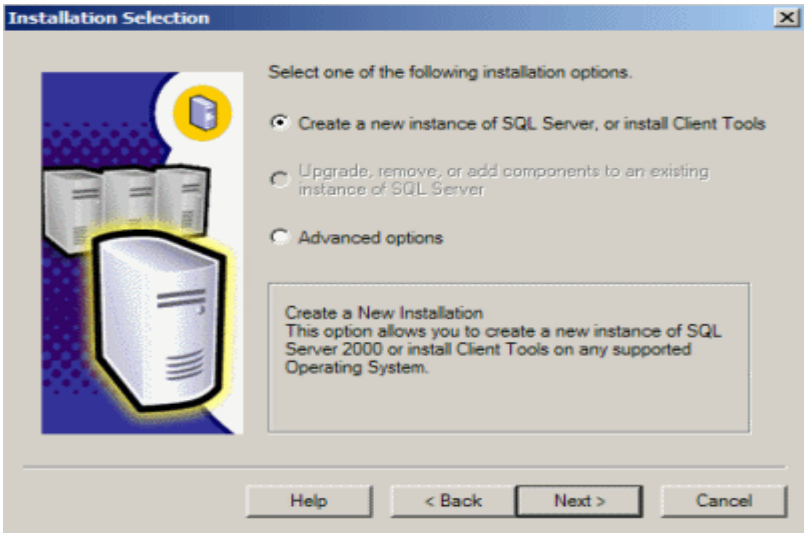
Bước 4: Trong màn hình **Welcome**, nhấn nút **Next** để tiếp tục việc cài đặt. Chọn **Local Computer** để cài đặt cơ sở dữ liệu ngay trên máy tính hiện hành, nhấn **Next** để tiếp tục.



Hình 1.5. Màn hình chọn cách thức cài đặt SQL

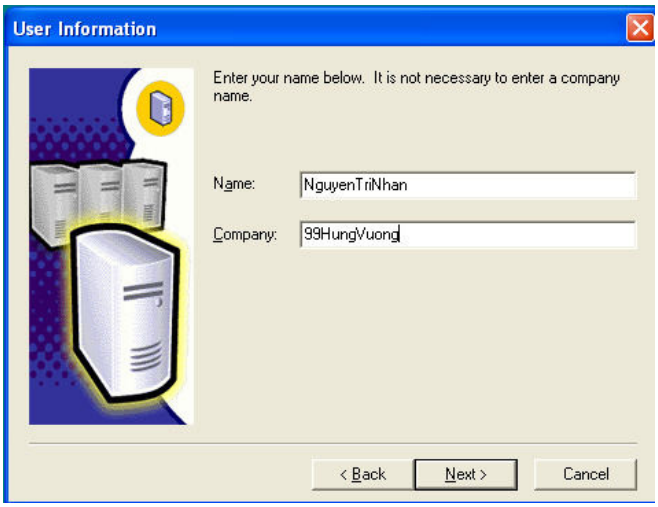
Trong trường hợp bạn muốn thực hiện việc cài đặt cơ sở dữ liệu cho một máy chủ từ xa bên trong một hệ thống mạng cục bộ, bạn chọn chức năng **Remote Computer**. Khi đó, bạn sẽ cung cấp thêm tài khoản người dùng, mật khẩu và **domain** của người dùng thuộc nhóm người quản trị để chương trình cài đặt có thể truy xuất vào máy chủ trong quá trình cài đặt.

Bước 5: Trong màn hình lựa chọn cài đặt (**Installation Selection**), bạn chọn chức năng **Create a new instance of SQL Server** để đơn giản trong quá trình cài đặt, nhấn nút **Next** để tếp tục.



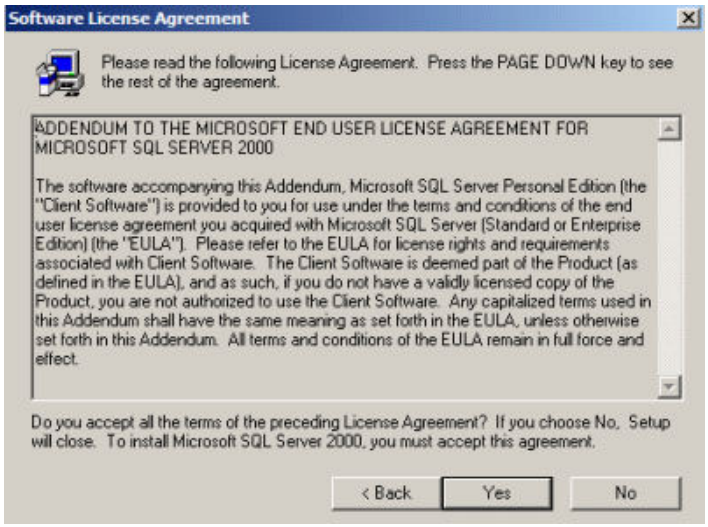
Hình 1.6. Màn hình lựa chọn cài đặt SQL

Bước 6: Trong màn hình thông tin người dùng (**User Information**), bạn nhập vào tên của mình và tên cơ quan. Có thể bỏ trống tên cơ quan. Nhấn nút **Next** để tiếp tục.

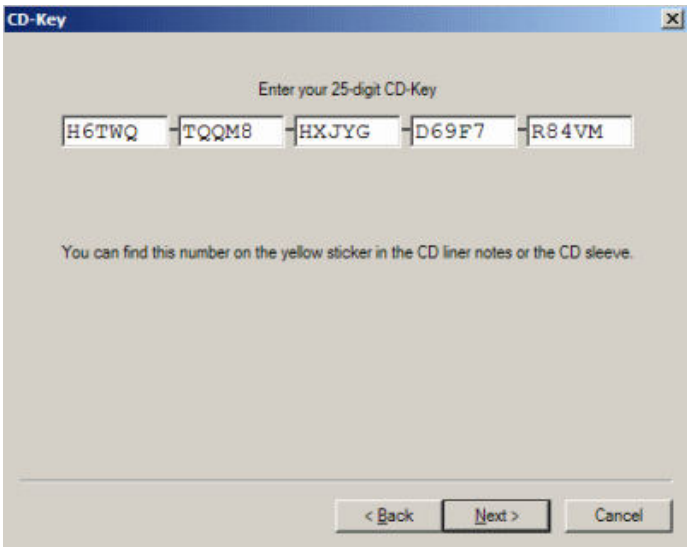


Hình 1.7. Màn hình nhập thông tin người dùng

Bước 7: Trong màn hình thỏa ước bản quyền phần mềm (**Software License Agreement**), **Microsoft** đưa ra một số thỏa ước cho người dùng khi sử dụng phần mềm. Nhấn **Yes** để đồng ý các thỏa ước này và tiếp tục quá trình cài đặt.



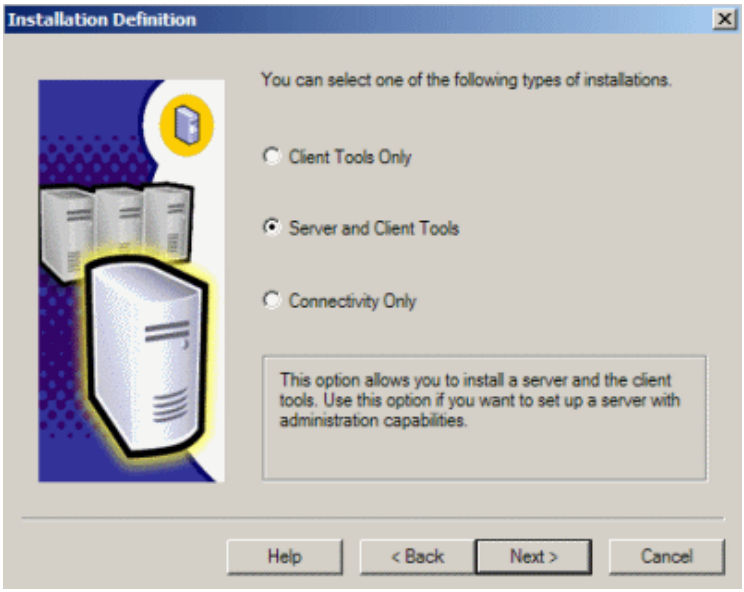
Hình 1.8. Màn hình thỏa ước bản quyền phần mềm



Hình 1.9. Màn hình nhập số CDKey phần mềm

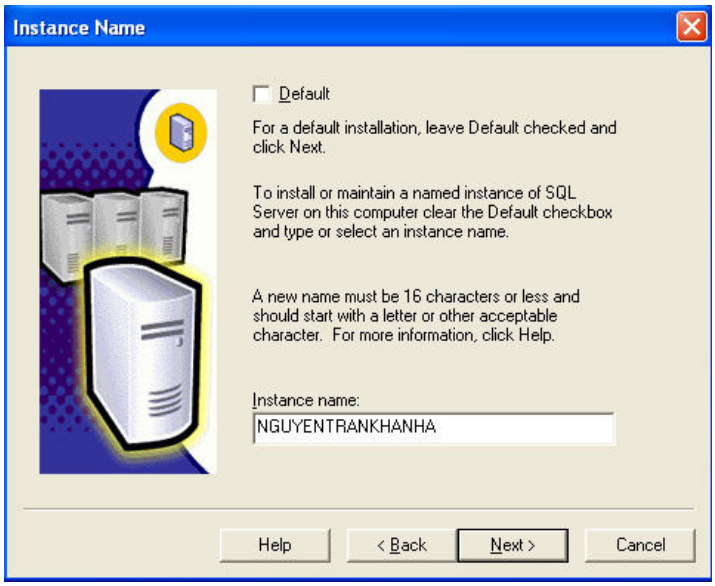
Chọn **Next** để tiếp tục cài đặt.

Bước 8: Trong màn hình định nghĩa loại cài đặt (**Installation Definition**), chọn **Server and Client Tools** để chỉ định việc cài đặt cơ sở dữ liệu và các tiện ích khác. Nhấn nút **Next** để tiếp tục.



Hình 1.10. Màn hình định nghĩa loại cài đặt SQL

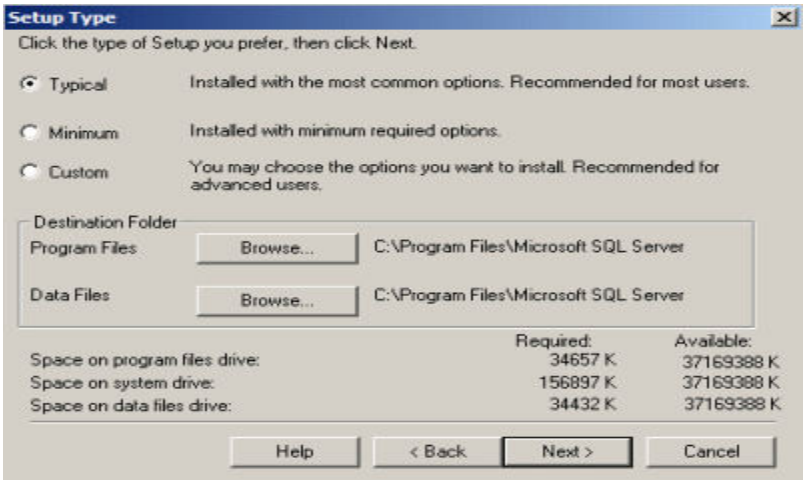
Bước 9: Trong màn hình định nghĩa tên thể hiện (**Instance Name**), chọn ô kiểm tra **Default** nếu muốn tên thể hiện của **Microsoft SQL Server Personal** trùng với tên của máy tính bạn đang cài đặt. Nhấn nút **Next** để tiếp tục.



Hình 1.11. Màn hình nhập tên thể hiện

Trong trường hợp bạn muốn đặt tên khác thì bỏ chọn ô kiểm tra **Default**, nhập vào tên thể hiện tại mục **Instance Name**.

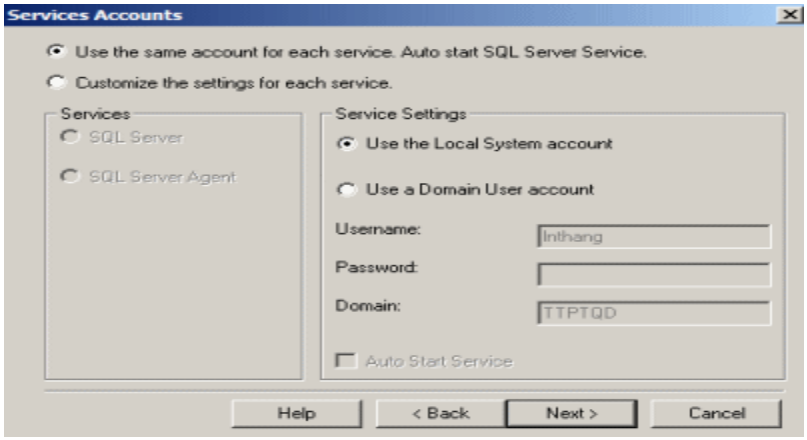
Bước 10: Trong màn hình chọn loại cài đặt (**Setup Type**) bạn nên chọn loại thường dùng là **Typical** nếu dung lượng đĩa cứng còn trống lớn hơn 130MB, ngược lại nếu dung lượng đĩa cứng còn trống lớn hơn 50MB thì nên chọn loại tối thiểu (**Minimum**). Nhấn nút **Next** để tiếp tục cài đặt.



Hình 1.12. Màn hình chọn loại cài đặt SQL

Mặc định thư mục **C:\Programs\Microsoft SQL Server** sẽ chứa các tập tin sau khi cài đặt, bao gồm các tập tin chương trình và tập tin dữ liệu. Bạn nên chọn một thư mục khác để lưu trữ các tập tin cơ sở dữ liệu với mục đích an toàn hơn. Bạn có thể nhấn nút **Browser** để thay đổi các thư mục khác nếu muốn.

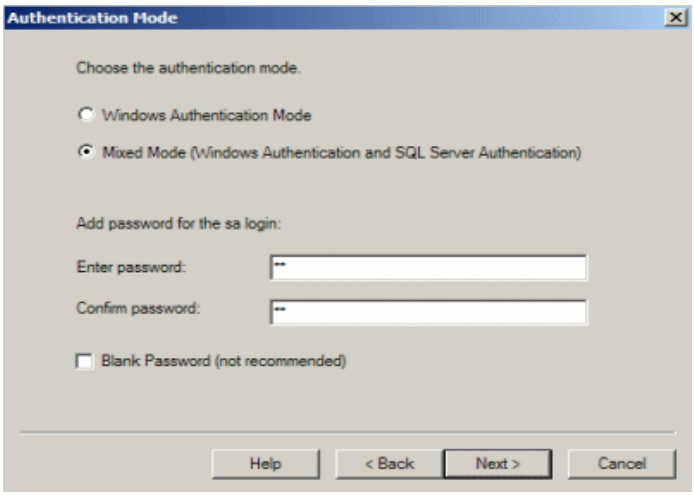
Bước 11: Trong màn hình chọn tài khoản người dùng khởi động dịch vụ (**Services Accounts**), cho phép bạn chỉ định tên tài khoản người dùng để có thể khởi động hai dịch vụ chính yếu của **Micorsoft SQL Server** khi hoạt động là: **SQL Server** và **SQL Server Agent** trong trường hợp **Microsoft SQL Server** chạy trong môi trường **Windows NT**. Nhấn nút **Next** để tiếp tục.



Hình 1.13. Màn hình chọn tài khoản người dùng

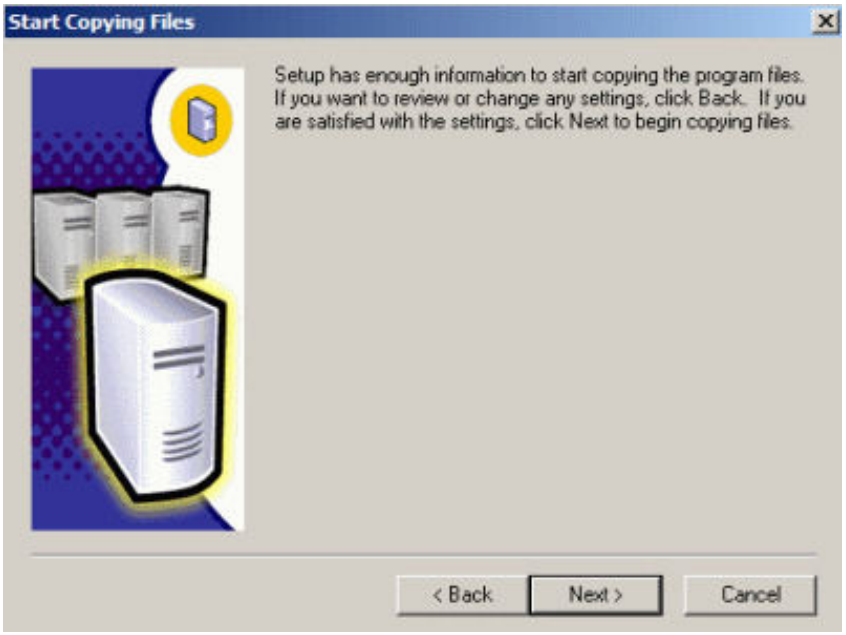
Nếu hệ điều hành máy tính bạn cài đặt là Windows 98 thì màn hình này sẽ không xuất hiện. Bạn nên chọn sử dụng tài khoản người dùng hệ thống cục bộ (**Use the Local System Account**) để khởi động cùng lúc cả 2 dịch vụ (**Use the same Account for each service**) đối với phiên bản **Microsoft SQL Server Personal**.

Bước 12: Trong màn hình chỉ định cơ chế xác thực đăng nhập (**Authentication Mode**), bạn chọn **Mixed Mode** để chỉ định việc đăng nhập (**Login**) vào **Microsoft SQL Server Personal** bằng 2 cách: tài khoản người dùng của **Windows** hoặc tài khoản người dùng trong **Microsoft SQL Server**. Bạn nhập tài khoản **SA** cho tài khoản người dùng **SA** trong **Microsoft SQL Server**. Nhấn nút **Next** đề tiếp tục.



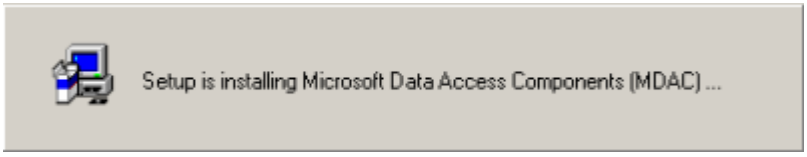
Hình 1.14. Màn hình cơ chế xác thực đăng nhập

Mặc định luôn luôn có một tài khoản người dùng có tên là **sa** (**system administrator**), người quản trị cơ sở dữ liệu trong **Microsoft SQL Server** được tạo sau khi kết thúc quá trình cài đặt cơ sở dữ liệu **Microsoft SQL Server**.

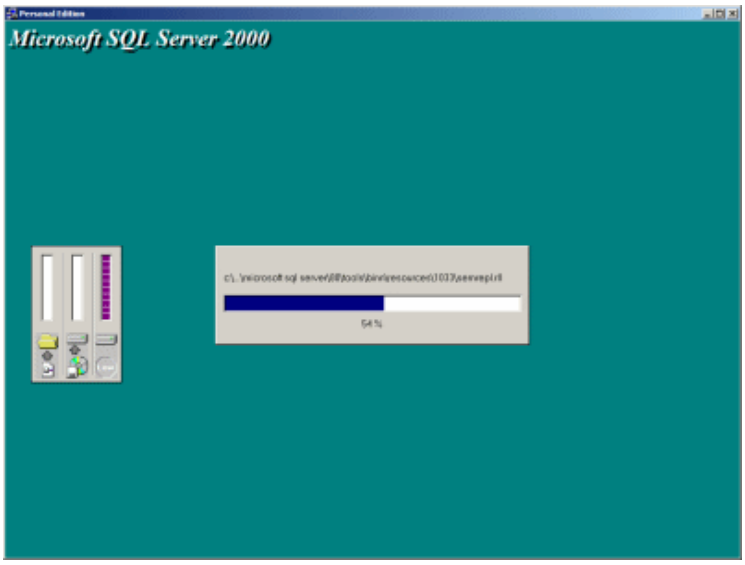


Hình 1.15. Màn hình chuẩn bị sao chép các tập tin

Bước 13: Trong màn hình bắt đầu sao chép tập tin (**Start Copying Files**), hệ thống thông báo cho biết các thông tin cài đặt đã được cung cấp đầy đủ. Nhấn nút **Next** đề bắt đầu quá trình sao chép tập tin từ **CDRom** vào đĩa cứng máy tính.

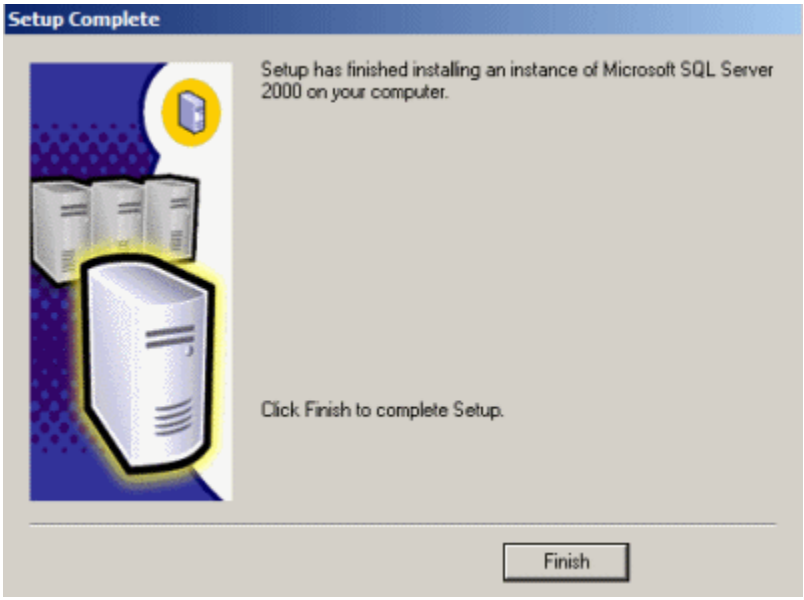


Hình 1.16. Chuẩn bị tiến trình sao chép các tập tin



Hình 1.17. Tiến hành sao chép các tập tin

Bước 14: Bạn phải chờ trong một thời gian để quá trình cài đặt thực hiện. Cuối cùng khi xuất hiện màn hình hoàn thành cài đặt (**Setup Complete**), bạn nhấn nút **Finish** đề kết thúc quá trình cài đặt cơ sở dữ liệu **Microsoft SQL Server Personal**.



Hình 1.18. Màn hình hoàn thành cài đặt SQL

1.4. CÁC CSDL HỆ THỐNG TRONG SQL SERVER

Các CSDL chứa các dữ liệu của người dùng được gọi là CSDL của người dùng (user database). Ngoài ra, các hoạt động của SQL Server tùy thuộc vào bốn CSDL hệ thống (system Database): master, model, Tempdb, và msdb. Các CSDL này tồn tại trong mỗi thể hiện của SQL Server.

CSDL hệ thống và CSDL người dùng:

CSDL **Master**: được dùng để điều khiển các thao tác và thông tin mức hệ thống của SQL Server. Lưu trữ thông tin hệ thống trong 16 bảng hệ thống, gọi là system catalog. System catalog lưu trữ các thông tin tài khoản người dùng, bao gồm các mục như là bảo mật, ID, mật khẩu, các CSDL lưu trên server, các biến môi trường, các thông điệp lỗi hệ thống, và các thủ tục hệ thống.

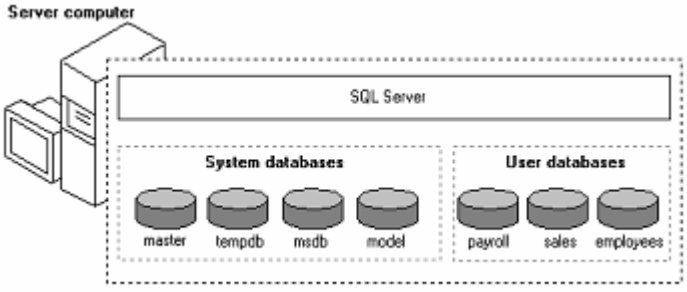
CSDL **Model**: được dùng như là một CSDL mẫu cho tất cả các CSDL mới được tạo ra trong hệ thống. CSDL Model có thể được tùy biến để tạo một cấu trúc CSDL mặc định mới cho CSDL mới. Ví dụ, nếu một bảng cần phải tồn tại trong CSDL mới sau này thì các bảng đó nên được tạo trong CSDL model. Các CSDL mới vừa được tạo sẽ có kích cỡ ít nhất bằng kích cỡ CSDL model.

CSDL **Tempdb**: được dùng như là vùng lưu chứa tạm thời đối với các bảng và các thủ tục tạm. SQL Server 2008 hỗ trợ hai loại bảng tạm: bảng tạm toàn cục (global temporary table) và bảng tạm cục bộ (local temporary table). Tên của bảng tạm toàn cục được bắt đầu ##, có thể được truy xuất đối với tất cả các client, trong khi tên bảng tạm cục bộ bắt đầu #, chỉ có thể truy xuất đối với những client mà tạo chúng. Làm việc trong Tempdb rất nhanh vì các hoạt động không được ghi nhận lại. Tuy nhiên, khi client kết thúc kết nối đến Server thì toàn bộ các bảng và thủ tục trong TempDB sẽ bị xóa.

CSDL **Msdb**: SQL Server Agent dùng CSDL msdb cho các tác vụ khác nhau, như lập biểu, cảnh báo và ghi nhận các thao tác. Dữ liệu được lưu trong các bảng hệ thống trong CSDL msdb. Các bảng hệ thống được dùng bởi SQL Server Agent là sysalerts, syscategories, sysdownloadlist, sysjobhistory, sysjobs, sysjobschedules, sysjobservers, sysjobsteps, sysnotifications, sysoperators, systargetservergroupmembers, systargetservergroups, systargetservers và systaskids. Ngoài ra, CSDL msdb có 7 bảng dùng cho thao tác dự phòng và phục hồi dữ liệu: backupfile, backupmediafamily, backupmediaset, backupset, restorefile, restorefilegroup, và restorehistory. SQL Server có thể tự động tăng hoặc giảm kích cỡ của CSDL khi các dòng được thêm vào hoặc xóa đi.

CSDL **Pubs**, **Northwind**: là hai CSDL ví dụ được dùng trong các tài liệu của SQL Server

Các CSDL khác là các CSDL do người dùng tạo ra.



Hình 1.19. Cở sở dữ liệu hệ thống và người dùng tạo**.**

Khái niệm về cơ sở dữ liệu quan hệ: Databases, Tables, Columns, Views, Datatypes, và Database Schemas

Dữ liệu trong một RDBMS như SQL Server 2000 được sắp xếp trong một số đối tượng, dễ thấy nhất là bảng (table). Dữ liệu được lưu trong các Table theo dòng và cột.

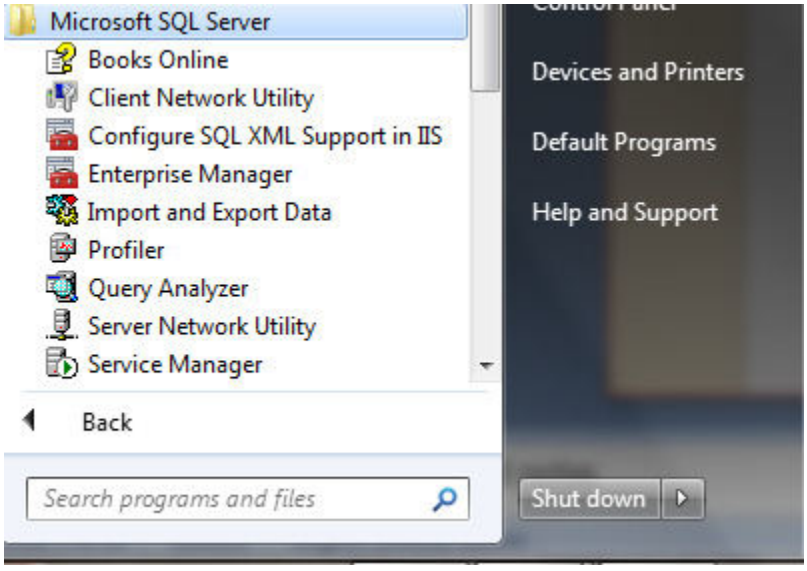
Dữ liệu liên quan đến các mục thực tế như các nhân viên, các sản phẩm, việc gửi hàng, người tham gia… được lưu trong các table riêng biệt. Ví dụ, hệ thống quản lý nhân viên của một công ty có thể có một bảng gọi là nhân viên, bảng này dùng để lưu chi tiết tất cả của các nhân viên trong công ty. Chi tiết của nhân viên bao gồm Họ nhân viên, Tên nhân viên, Địa chỉ, Số chứng minh nhân dân… Thỉnh thoảng cần ẩn các thông tin riêng tư từ người dùng bằng cách dùng một truy vấn. Các truy vấn là các câu lệnh SQL lưu trong CSDL và có thể được tham chiếu đến các câu lệnh SQL theo cách như các table. SQL Server cũng sử dụng các kiểu dữ liệu, mỗi cột có thể có các kiểu khác nhau.

Tất cả các kiểu khác nhau này của các đối tượng, giống như bảng, truy vấn và kiểu dữ liệu được lưu trong một CSDL. Trong môi trường SQL Server, một lược đồ CSDL (database schema) là tập hợp các đối tượng CSDL có liên quan đến việc sử dụng một tên duy nhất và nó thuộc về một người dùng đơn lẻ.

1.5. CÁC CÔNG CỤ TRONG SQL SERVER

Sau khi cài đặt xong Microsoft SQL Server công việc kế tiếp mà chúng ta sẽ làm là tìm hiểu về các tiện ích bên trong Microsoft SQL Server nhằm để sử dụng một cách đúng đắn và có hiệu quả cao. Các tiện ích này đã được cài đặt chung với Microsoft SQL Server. Để kích hoạt các tiện ích này, chúng ta sẽ chọn những biểu tượng nằm trong đường dẫn như sau:

Start → Programs → Microsoft SQL Server



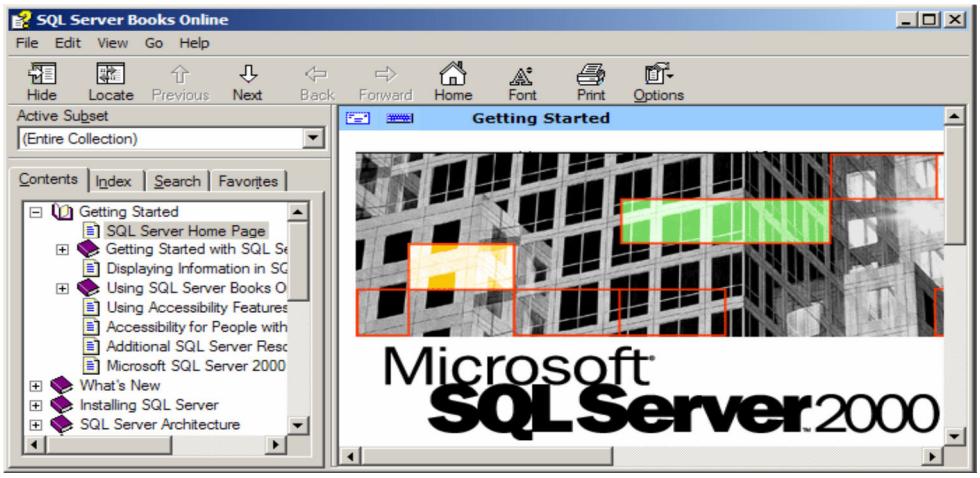
Hình 1.20. Các tiện ích bên trong Microsoft SQL Server.

1.5.1. Tiện ích Book Online

Ứng dụng này cho phép chúng ta có thể **tra cứu trực tuyến** tất cả các thông tin liên quan đến Microsoft SQL Server một cách đầy đủ với các tính năng tìm kiếm dễ dàng và một giao diện dễ sử dụng.

Nội dung được trình bày theo từng phần dễ dàng xem trong trang Contents. Ngoài ra còn các trang Index và Search cho phép các bạn tra cứu nhanh theo chỉ mục đã được sắp xếp trước đó hoặc gõ vào các từ khóa cần tìm.

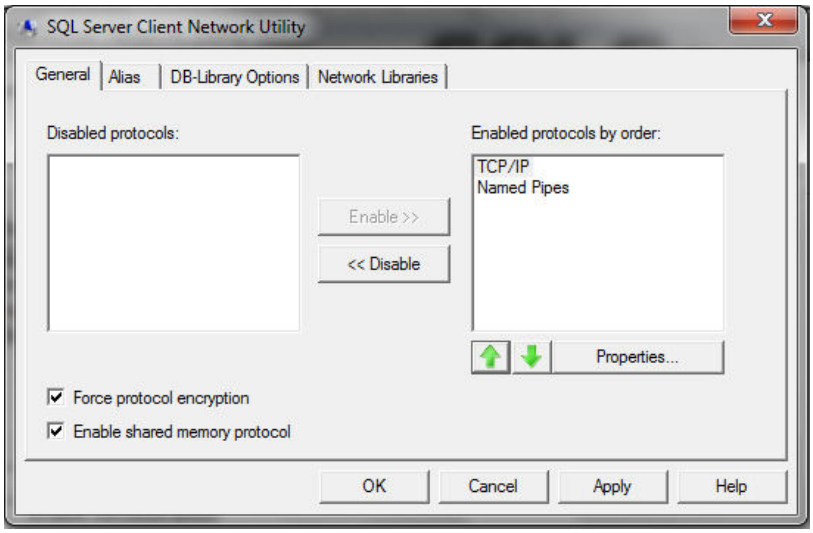
Khi cần tham khảo cú pháp các lệnh, hàm, biến hệ thống... một cách đầy đủ nhất thì chúng ta vào đây xem là hoàn toàn chính xác. Tuy nhiên nội dung trình bày hoàn toàn là bằng tiếng Anh.



Hình 1.21. Tiện ích Book Online.

1.5.2. Tiện ích Client NetWork Utility

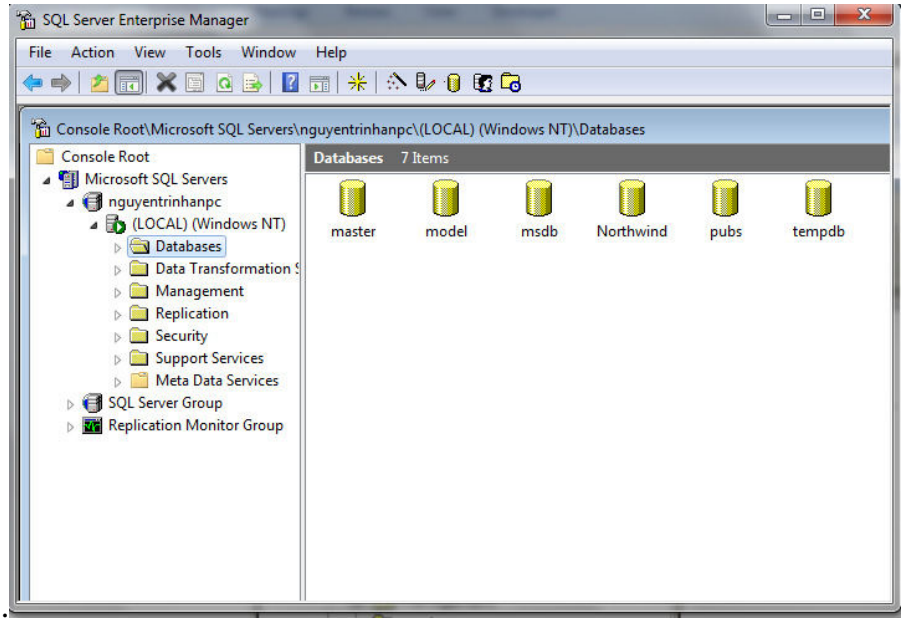
Tiện ích này cho phép chúng ta thay đổi, tạo mới và lưu lại các **nghi thức nối kết mạng** (network protocol) mặc định của máy trạm khi thực hiện nối kết vào Microsoft SQL Server tại các máy chủ.



Hình 1.22. Tiện ích Client NetWork Utility.

1.5.3. Tiện ích Enterprise Manager

Tiện ích này cho phép chúng ta khởi động hoặc tạm ngưng các dịch vụ của Microsoft SQL Server. Trong phần cài đặt ở bước 9 chúng tôi đã giới thiệu các dịch vụ của Microsoft SQL Server phải được khởi động trước thì các bạn mới có thể làm việc được với Microsoft SQL Server



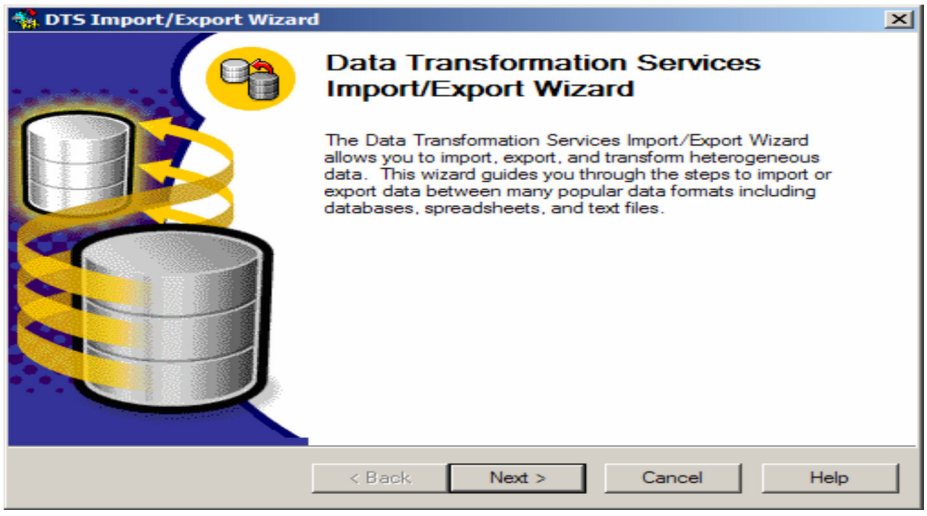
Hình 1.23. Tiện ích Enterprise Manager.

Ngoài ra tiện ích này còn giúp chúng ta **quản trị** một hoặc nhiều Microsoft SQL Server khác nhau, với giao diện đồ họa thân thiện (user friendly) tiện ích này sẽ giúp cho các bạn có thể tạo lập cơ sở dữ liệu và các thành phần bên trong Microsoft SQL Server một cách dễ dàng hơn.

Tuy nhiên muốn quản trị Microsoft SQL Server thì chúng ta phải **đăng ký** (register) máy chủ vào tiện ích này, việc đăng ký như thế nào chúng tôi sẽ trình bày trong những phần tiếp theo.

1.5.4. Tiện ích Import and Export Data

Tiện ích này cho phép chúng ta thực hiện các tính năng trong việc nhập (import), xuất (export) và **chuyển đổi dữ liệu** qua lại giữa Microsoft SQL Server và những loại cơ sở dữ liệu khác thường dùng như: Microsoft Access, Visual FoxPro, Microsoft Excel, tập tin văn bản ASCII...

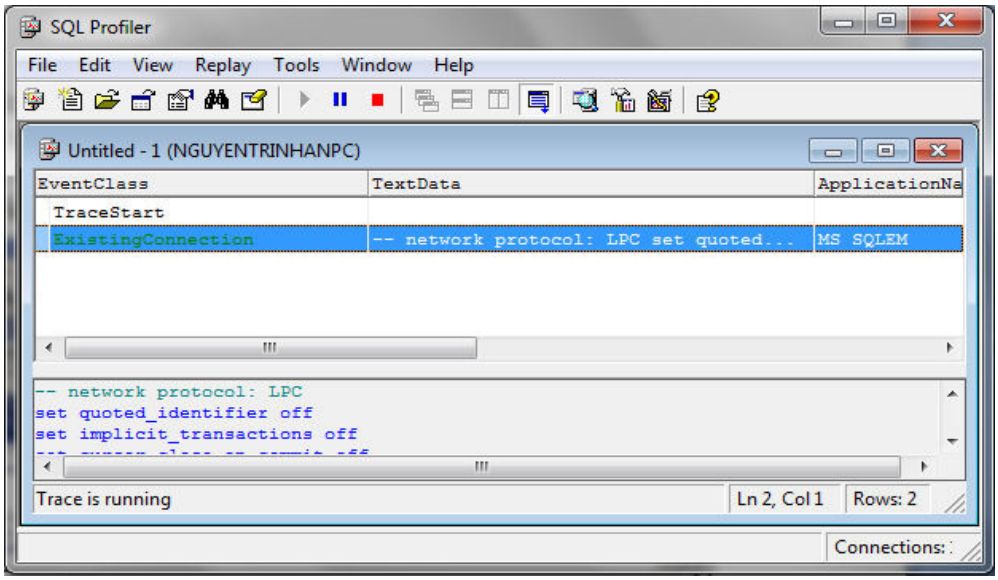


Hình 1.24. Tiện ích Import and Export Data.

Trong phần bài giảng này chúng tôi chỉ trình bày cách sử dụng Microsoft SQL Server nên không giới thiệu nhiều về tiện ích này, tuy nhiên nếu các bạn muốn trở thành người quản trị Microsoft SQL Server thì chắc rằng công cụ này là rất cần thiết cho người quản trị.

1.5.5. Tiện ích Profiler

Tiện ích này cho phép chúng ta phát hiện ra những **biến cố** đã xảy ra của Microsoft SQL Server khi đang thực hiện một xử lý nào đó trên máy chủ. Các biến cố này có thể được ghi lại trong một tập tin lưu vết (trace file) để sau này sử dụng lại cho việc phân tích nhằm phát hiện ra những vấn đề khi thực hiện các câu lệnh truy vấn trong xử lý đó.

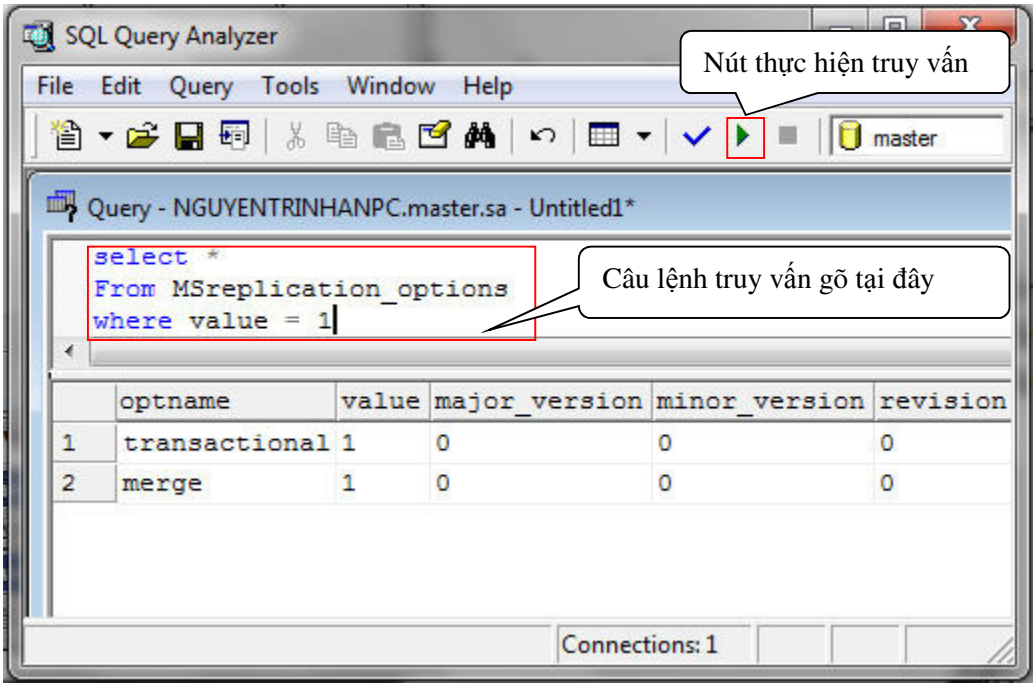


Hình 1.25. Tiện ích Profiler.

Hoạt động của tiện ích này có phần gần giống công cụ tìm lỗi trong các ngôn ngữ lập trình, có nghĩa là các bạn sẽ cho thực hiện tuần tự các câu lệnh trong một xử lý lô (batch) để có thể phát hiện ra lỗi của một câu lệnh nào đó (nếu có).

1.5.6. Tiện ích Query Analyzer

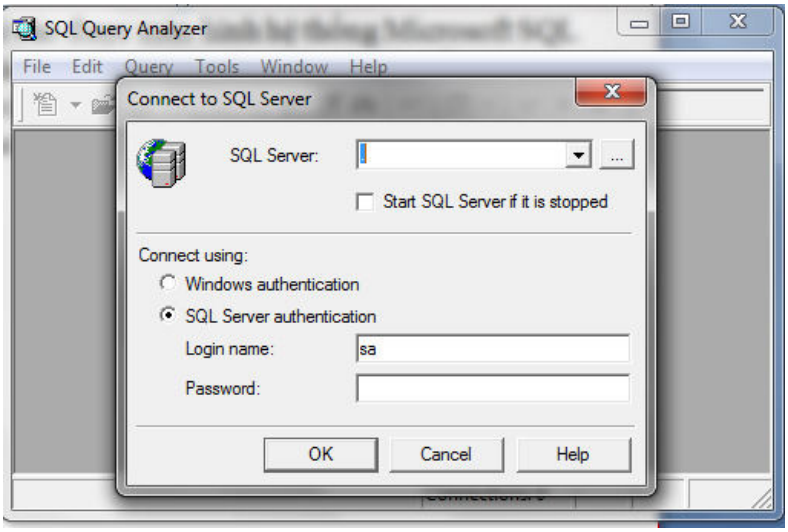
Tiện ích này cho phép chúng ta **soạn thảo** các tập tin kịch bản (script file) – là tập tin văn bản ASCII chứa các câu lệnh SQL giao tác trên cơ sở dữ liệu Microsoft SQL Server hoặc có thể **thực hiện các truy vấn** trực tiếp trên cơ sở dữ liệu Microsoft SQL Server và nhận được kết quả trực tiếp ngay sau khi thực hiện truy vấn đó.



Hình 1.26. Tiện ích Query Analyzer**.**

Tuy nhiên trước khi vào được màn hình 1-19 hệ thống Microsoft SQL Server sẽ yêu cầu các bạn đăng nhập (login) vào hệ thống với tên tài khoản người dùng và mật khẩu hợp lệ bởi vì Microsoft SQL Server là một hệ quản trị cơ sở dữ liệu mạng máy tính.

Trong đó tên tài khoản người dùng và mật khẩu này sẽ do người quản trị Microsoft SQL Server tạo ra trước đó, trong trường hợp làm việc trên Microsoft SQL Server Desktop thì chúng ta có thể nhập vào với tên là SA (System Administrator – người quản trị hệ thống cơ sở dữ liệu Microsoft SQL Server) và mật khẩu để trống.

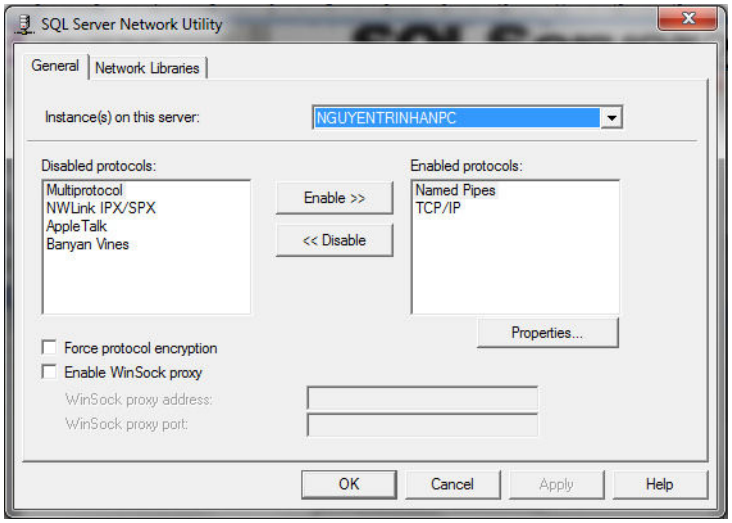


Hình 1.27. Đăng nhập đến Server.

Trong các chương còn lại chúng ta chủ yếu làm việc với tiện ích này để tạo ra các đối tượng lưu trữ dữ liệu bên trong Microsoft SQL Server.

1.5.7. Tiện ích Server Network Utility

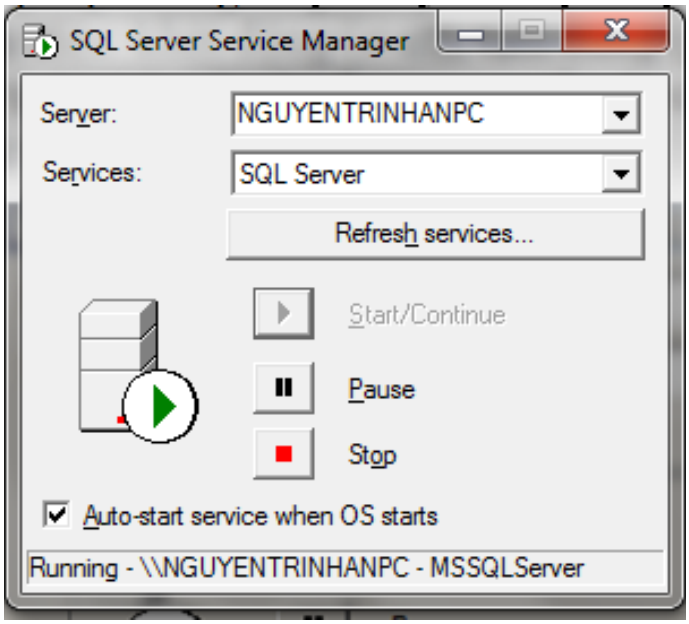
Tiện ích này cho phép chúng ta quản lý các **thư viện nghi thức nối kết mạng** của máy chủ dùng để lắng nghe các yêu cầu từ các máy trạm – có nghĩa các nghi thức nối kết mạng giữa máy chủ và máy trạm phải ăn khớp nhau để chúng có thể giao tiếp qua lại.



Hình 1.28. Tiện ích Server Network Utility

1.5.8. Tiện ích Service Manager

Tiện ích này cho phép chúng ta **quản lý các dịch vụ** liên quan đến Microsoft SQL Server. Có thể thực hiện việc: khởi động (start), tạm dừng (pause) và ngưng lại (stop) các dịch vụ đó. Các dịch vụ (services) này được xem như là các ứng dụng chạy ngầm định bên dưới hệ thống trong môi trường Windows.

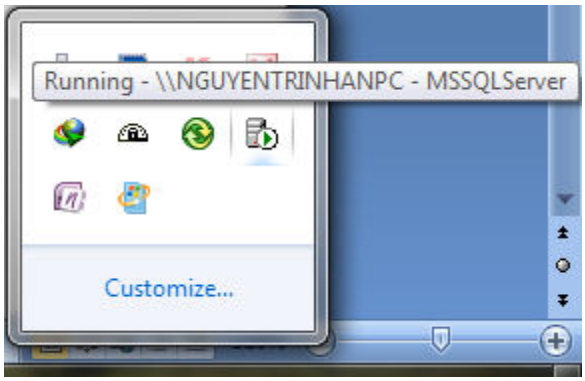


Hình 1.29. Tiện ích Service Manager.

Đối với dịch vụ MSSQLSever bắt buộc phải được khởi động để người dùng mới có thể làm việc với cơ sở dữ liệu trong Microsoft SQL Server. Do đó sau khi cài đặt hoàn thành Microsoft SQL Server các bạn phải tự khởi động dịch vụ này để chúng ta có thể làm việc với Microsoft SQL Server.

Tuy nhiên kể từ các lần bật máy tính sau đó các dịch vụ này sẽ được tự khởi động nếu chúng ta đã chọn vào ô kiểm tra **Auto-start** như hình ở phía trên.

Thông thường tiện ích Service Manager sẽ xuất hiện bên dưới thanh tác vụ (task tray) của Windows. Để kích hoạt tiện ích này, chúng ta có nhấn đúp vào biểu tượng Service Manager.



Hình 1.30. Tiện ích Service Manager xuất hiện trên thanh công cụ.

Chương 2. GIỚI THIỆU NGÔN NGỮ TRANSACT – SQL

2.1. GIỚI THIỆU

- Structured Query Language (SQL) là ngôn ngữ do IBM phát triển từ năm 1970, dùng để giao tiếp với cơ sở dữ liệu. Các hệ thống cơ sở dữ liệu có cách viết và thi hành câu lệnh SQL riêng như: ANSI SQL (SQL chuẩn), T-SQL (SQL Server, Sybase), PL/SQL (Oracle), Access SQL.

- Transact-SQL là ngôn ngữ SQL mở rộng dựa trên SQL chuẩn của ISO (International Organization for Standardization) và ANSI (American National Standards Institute) được sử dụng trong SQL Server khác với P-SQL (Procedural-SQL) dùng trong Oracle.

Các lệnh T-SQL được chia làm 3 nhóm:

+ Data Definition Language (DDL): Đây là những lệnh dùng để quản lý các thuộc tính của một Database như định nghĩa các hàng hoặc cột của một table, hay vị trí data file của một database... như Create, Alter, Drop.

+ Data Control Language (DCL): Đây là những lệnh quản lý các quyền truy cập lên từng object (table, view, stored procedure...) như Grant, Revoke, Deny.

+ Data Manipulation Language (DML): Đây là những lệnh phổ biến dùng để xử lý data như Select, Update, Insert, Delete.

Trong phần này chúng ta chủ yếu nghiên cứu nhóm lệnh **Data Manipulation Language (DML).**

2.2. KIỂU DỮ LIỆU

Trong SQL Server, mỗi cột, biến địa phương, thể hiện và tham số có một kiểu dữ liệu liên quan. Một kiểu dữ liệu là một thuộc tính chỉ định kiểu dữ liệu mà các đối tượng có thể giữ: số nguyên, ký tự, tiền tệ, ngày tháng và thời gian, các chuỗi nhị phân, và như vậy SQL Server cung cấp một bộ các hệ thống kiểu dữ liệu để xác định tất cả các loại dữ liệu có thể được sử dụng với SQL Server. Ngoài ra bạn cũng có thể tự định nghĩa kiểu dữ liệu cho riêng mình (sinh viên tự nghiên cứu).

Kiểu số

Kiểu dữ liệu	Giới hạn mô tả	Byte lưu trữ
Bit	Nhận giá trị 0 (False), 1 (True) hoặc NULL	
Tinyint	Từ 0 đến 255	1 byte
Smallint	Từ -2^{15} (-32,768) đến $2^{15}-1$ (32,767)	2 bytes
Int	Từ -2^{31} (-2,147,483,648) đến $2^{31}-1$ (2,147,483,647)	4 bytes
Bigint	Từ -2^{63} (-9,223,372,036,854,775,808) đến $2^{63}-1$ (9,223,372,036,854,775,807)	8 bytes

Decimal, Numeric	Từ $-10^{38}+1$ đến $10^{38}-1$	Phụ thuộc và mô tả, từ 5 đến 17 bytes
Real	Từ -3.40E + 38 đến -1.18E - 38, 0 và từ 1.18E - 38 đến 3.40E + 38	4 bytes
Float	Từ -1.79E+308 đến -2.23E-308, 0 và từ 2.23E-308 to 1.79E+308	Tối đa 8 bytes
Smallmoney	Từ -214.748,3648 đến 214.748,3647	4 bytes
Money	Từ -922.337.203.685.477,5808 đến 922.337.203.685.477,5807	8 bytes

Kiểu ngày tháng

Kiểu dữ liệu	Giới hạn mô tả	Byte lưu trữ
Date	Từ 01/01/0001 đến 31/12/9999	3 bytes
Time	Từ 00:00:00.0000000 đến 23:59:59.9999999	5 bytes
SmallDatetime	Ngày từ 01/01/1900 đến 06/06/2079 Giờ từ 00:00:00 đến 23:59:59	4 bytes
Datetime	Ngày từ 1/1/1753 đến 31/12/9999 Giờ từ 00:00:00 đến 23:59:59.997	8 bytes

Kiểu kí tự

Kiểu dữ liệu	Giới hạn mô tả	Byte lưu trữ
Char	Cú pháp: char(n), n từ 0 đến 8.000	n bytes
Varchar	Cú pháp: varchar(n), n từ 0 đến 8.000	Tối đa n bytes
Text	Lưu trữ tối đa 2.147.483.647 kí tự	Tối đa 2,147,483,647 bytes
nChar	Cú pháp: nchar(n), n từ 0 đến 4.000	2*n bytes
nVarchar	Cú pháp: nvarchar(n), n từ 0 đến 4.000	2*n bytes
nText	Lưu trữ tối đa 1,073,741,823 ký tự	Tối đa 2,147,483,647 bytes

2.3. TRUY XUẤT DỮ LIỆU (DATA QUERY LANGUAGE)

2.3.1. Truy vấn dữ liệu đơn giản với SELECT

Trong thực tế chúng ta thường xuyên phải thực hiện các công việc như lập danh sách sinh viên trong lớp hay lập danh sách các nhà cung cấp hàng hóa cho công ty… Để thực

hiện các công việc đó trong hệ quản trị CSDL với ngôn ngữ T-SQL, chúng ta sử dụng câu truy vấn SELECT.

- ***Cú pháp***:

```
SELECT [DISTINCT] [TOP So_Ban_Ghi]
     <Danh_Sach_Truong>
  FROM <Ten_Bang>
```

Giải thích:

- Distinct: Chỉ định không trùng lặp thông tin giữa các bản ghi của kết quả truy vấn tương ứng với danh sách trường hiển thị.
- So_Ban_Ghi: Số bản ghi trả về trong kết quả truy vấn
  - ***Lưu ý***: Nếu danh sách kết quả không được sắp xếp, kết quả sẽ hiển thị theo thứ tự dữ liệu được thêm vào bảng.
- Danh_Sach_Truong: Trường hiển thị trong danh sách kết quả truy vấn, danh sách trường có thể là tên các trường hoặc các biểu thức được ngăn cách với nhau bằng dấu phẩy dưới ",". Số lượng tối đa các trường, biểu thức trong danh sách này là 4096.
  - ***Lưu ý***: Sử dụng ký tự "*" khi cần lấy tất cả các trường trong bảng gốc vào danh sách kết quả truy vấn.
- Ten_Bang: Tên bảng cần truy vấn dữ liệu

Ví dụ:

Cho các bảng cơ sở dữ liệu sau:

- Bảng **danh sách nhà cung cấp, khách hàng (DMDT)** bao gồm các trường: Id, Ma_Dt, Ten_Dt, Dia_Chi, So_Dt, Fax, Email, Ghi_Chu.

Dữ liệu mẫu

Id	Ma_Dt	Ten_Dt	Dia_Chi	So_Dt	Fax	Email	Ghi chú
1	N001	Cty TNHH ABC	Hà Nội	04 3640 0119			
3	N002	Cty Cổ phần CB thực phẩm Miền Bắc	Hà Nội	046 3423 3438			
4	N003	Cty liên doanh Việt Pháp	Hải Phòng	033 6534 391			
5	N004	Cty TNHH một thành viên cấp nước Yên Bái	Yên Bái	02183 543 443			
6	N005	Cty Cổ phần Lạc Hồng	Bắc Ninh	0240 362 552			
7	N006	Cty XNK Á Châu	Hải Dương	0320 3111 411			

- Bảng **danh sách vật tư, hàng hóa (DMVT)** bao gồm các trường: Id, Ma_Vt, Ten_Vt, Dvt, Quy_Cach, Ghi_Chu.

Id	Ma_Vt	Ten_Vt	Dvt	Quy_Cach
1	TP001	Bánh trứng Custas	Hộp	Hộp 20 cái
2	TP002	Bánh kem xốp	Gói	Gói 200 gram
3	VT001	Bột mì loại 1	Kg	
4	VT002	Hương liệu	Kg	
5	VT003	Bột nở	Kg	
6	TP003	Bánh kem bơ	Gói	Gói 150 gram
7	TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái
8	VT004	Socola nguyên liệu	Kg	
9	VT005	Sữa Ông Thọ	Thùng	

- Bảng **chứng từ (CT)** bao gồm các trường: Id, Ma_Ct, Nhom_Ct, So_Ct, Ngay_Ct, Ma_Dt, Ong_Ba, Dia_Chi, Dien_Giai.

Stt_Ct	Ma_Ct	Nhom_Ct	So_Ct	Ngay_Ct	Ma_Dt	Ong_Ba	Dia_Chi	Dien_Giai
1	PX	1	PX001	31/05/11	N001	Lê Văn Khương	Hà Nội	Xuất hàng hóa bán đại lý
2	PX	1	PX002	01/06/11	N001	Đào Thị Hạnh	Hà Nội	Xuất hàng hóa bán đại lý
3	PN	2	PN001	01/06/11	N002	Tạ Thu Loan	Công ty	Nhập vật tư phục vụ sản xuất
4	PX	1	PX003	01/06/11	N004	Phan Thế Anh	Hà Nội	Xuất bán hàng hóa
5	PN	2	PN002	02/06/11	N005	Tạ Thị Minh	Hà Tây	Nhập nguyên liệu
6	PX	1	PX004	05/06/11	N004	Phan Thế Anh	Hà Nội	Xuất bán hàng hóa

- Bảng **chi tiết chứng từ (CTCT)** bao gồm các trường: Id, Id_Ct, Ma_Vt, Ma_Kho, So_Luong, Don_Gia.

Stt_Dong	Stt_Ct	Ma_Vt	Ma_Kho	So_Luong	Don_Gia
1	1	TP001	KTP	120.000	120,00
2	1	TP002	KTP	23.500	45,00
1	2	TP001	KTP	11.500	35,00
1	3	VT001	KVT	30.000	120,00
2	3	VT002	KVT	120.000	15,00
1	4	TP001	KTP	110.000	75,00
2	4	TP004	KTP	34.000	44,00

1	5	VT004	KVT	75.000	30,00
2	5	VT001	KVT	31.000	32,00
3	5	VT003	KVT	120.000	56,00
1	6	TP001	KVT	13.000	78,00

Yêu cầu truy vấn:

- Lập danh sách nhà cung cấp, khách hàng bao gồm các thông tin: Tên, Địa chỉ, Số điện thoại, Email.

Câu lệnh SQL:

```
SELECT Ten_Dt, Dia_Chi, So_Dt, Email
  FROM DmDt
```

- Lập danh sách vật tư hàng hóa bao gồm toàn bộ thông tin có trong bảng DMVT

```
SELECT * FROM DmVt
```

- Lập danh sách 5 vật tư xuất hiện đầu tiên trong bảng DMVT

```
SELECT TOP 5 Id, Ma_Vt, Ten_Vt, Quy_Cach, Ghi_Chu
  FROM DmVt
```

- Lập bảng kê chứng từ nhập xuất bao gồm các thông tin Ngày chứng từ, số chứng từ, Diễn giải, mã đối tượng, người nhập xuất (học viên tự viết).

2.3.2. Tìm hiểu các trường tính toán

Các trường tính toán không tồn tại trong cơ sở dữ liệu đã được chuẩn hóa, nó chỉ được xác định trong quá trình xử lý, tính toán. Ví dụ trong bảng DMDT (như giới thiệu phần trước) bạn muốn lấy danh sách nhà cung cấp với tên nhà cung cấp kèm theo số điện thoại để tiện liên lạc như minh họa dưới đây:

Mã số	Tên nhà cung cấp	Địa chỉ
N001	Cty TNHH ABC (04 3640 0119)	Hà Nội
N002	Cty Cổ phần CB thực phẩm Miền Bắc (046 3423 3438)	Hà Nội
…	…	…

Trong ví dụ này cột "Tên nhà cung cấp" được tạo ra từ trường Ten_Dt và trường So_Dt trong bảng DMDT.

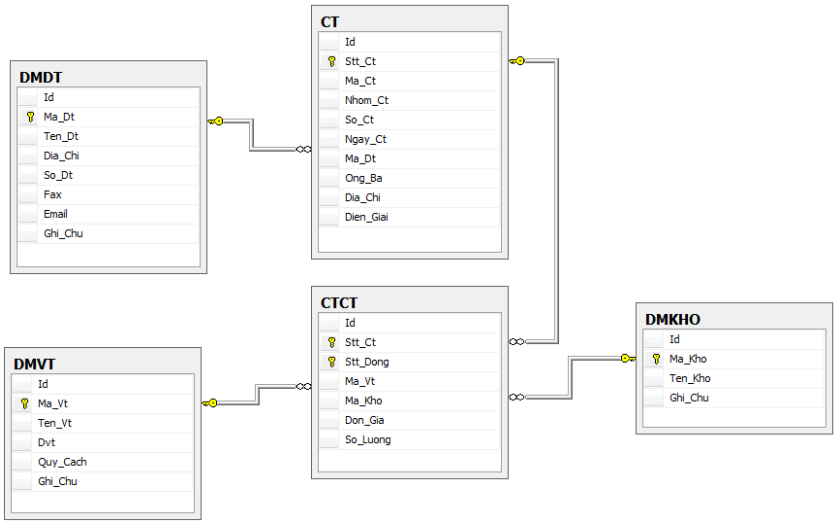
Câu lệnh SQL tương ứng:

```
SELECT Ma_Dt, Ten_Dt + ' (' + So_Dt + ')', Dia_Chi
  FROM DmDt
```

Hoặc trong bảng CTCT (như giới thiệu phần trước) bạn muốn lấy danh sách chứng từ bao gồm các trường Ma_Vt, Ma_Kho, So_Luong, Don_Gia, Thanh_Tien.

Mã vật tư	Mã kho	Số lượng	Đơn giá	Thành tiền
TP001	KTP	120.000	120,00	14.400.000
TP002	KTP	23.500	45,00	1.075.500
…	…			…

- Mô hình quan hệ giữa các bảng trong hệ thống



Trong ví dụ này cột "Thành tiền" được tạo ra từ phép nhân giữa hai trường Don_Gia và So_Luong.

Câu lệnh SQL tương ứng:

```
SELECT Ma_Vt, Ma_Kho, So_Luong, Don_Gia, Don_Gia * So_Luong
  FROM DmDt
```

Như vậy trong hai ví dụ trên trường "Tên nhà cung cấp" mới và trường "Thành tiền" là các trường tính toán.

❖ ***Lưu ý***:

- Tên bí danh: Câu lệnh SQL ở hai ví dụ trên khi thực thi đều cho kết quả đúng, tuy nhiên hai trường tính toán chưa có tên (thường trả về tên mặc định theo hệ quản trị cơ sở dữ liệu ví dụ như "(no column name)"), để gán tên cho hai trường này chúng ta sử dụng cú pháp <Bieu_Thuc> AS Ten_Bi_Danh.

Có thể viết lại hai câu truy vấn trên như sau:

```
SELECT Ma_Dt, Ten_Dt + ' (' + So_Dt + ')' AS Ten_Dt_Moi,
     Dia_Chi
  FROM DmDt
```

Và

```
SELECT Ma_Vt, Ma_Kho, So_Luong,
```

```
    Don_Gia, Don_Gia * So_Luong AS Thanh_Tien
  FROM DmDt
```

Trong hai câu lệnh mới tên trường "Thanh_Tien" và "Ten_Dt_Moi" gọi là tên bí danh.

- Các toán tử trong biểu thức trường tính toán: Có thể sử dụng các toán tử +, -, *, / trong biểu thức của trường tính toán.
- Chúng ta cũng có thể sử dụng các hàm xử lý chuỗi, ngày tháng, số trong biểu thức của trường tính toán.

Danh sách một số hàm thường dùng

Hàm	Giải thích
GetDate()	Lấy thời gian hiện thời tại Client
GetUtcDate()	Lấy thời gian hiện thời tại Server
Day()	Trả về giá trị ngày của đối số
Month()	Trả về giá trị tháng của đối số
Year()	Trả về giá trị năm của đối số
ABS()	Trả về giá trị tuyệt đối của đối số
LEFT()	Lấy các ký tự bên trái của đối số
RIGHT()	Lấy các ký tự bên phải đối số
SUBSTRING()	Lấy ký tự bất kỳ của đối số
LEN()	Trả về độ dài của đối số
LTRIM(), RTRIM(), ALLTRIM()	Cắt khoảng trắng bên trái, phải và cả trái và phải của đối số
UPPER(), LOWER()	Chuyển đối số thành chữ viết hoa, viết thường
…	…

2.3.3. Lọc dữ liệu với mệnh đề WHERE

Trong thực tế bảng thường chứa một lượng lớn dữ liệu và ít khi chúng ta lấy toàn bộ các bản ghi trong bảng dữ liệu đó. Chúng ta thường truy xuất một tập con dữ liệu của bảng ứng với các hoạt động cụ thể. Việc truy vấn như vậy đòi hỏi phải chỉ định một vùng điều kiện tìm kiếm hay còn gọi là điều kiện lọc. Ví dụ lấy danh sách các nhà cung cấp tại Hà Nội hoặc lấy bảng kê các mặt hàng nhập xuất trong năm 2013,…

❖ ***Cú pháp***:

```
SELECT [DISTINCT] [TOP So_Ban_Ghi]
    <Danh_Sach_Truong>
```

```
  FROM <Ten_Bang>
  WHERE <Dieu_Kien_Loc>
```

❖ ***Giải thích***:

- Dieu_Kien_Loc: Biểu thức điều kiện lọc dữ liệu

Một số toán tử cơ bản trong biểu thức điều kiện lọc dữ liệu:

Toán tử	Giải thích
=	Bằng
<>	Khác
!=	Khác
<	Nhỏ hơn
<=	Nhỏ hơn hoặc bằng
!<	Không nhỏ hơn
>	Lớn hơn
>=	Lớn hơn hoặc bằng
!>	Không lớn hơn
BETWEEN	Nằm giữa hai giá trị cụ thể
ISNULL	Là một giá trị NULL

Ví dụ: (Sử dụng các bảng dữ liệu ở phần trước)

- Lập danh sách khách hàng, nhà cung cấp tại Hà Nội

```
SELECT Ma_Dt, Ten_Dt, So_Dt, Fax, Email
  FROM DmDt
  WHERE Dia_Chi = 'Hà Nội'
```

Kết quả

Ma_Dt	Ten_Dt	So_Dt	Fax	Email
N001	Cty TNHH ABC	04 3640 0119		
N002	Cty Cổ phần CB thực phẩm miền bắc	046 3423 3438		

- Lập bảng kê chứng từ nhập xuất trong tháng 06

```
SELECT Ma_Ct, Ngay_Ct, So_Ct, Ma_Dt, Dien_Giai
  FROM Ct
  WHERE MONTH(Ngay_Ct) = 6
```

Kết quả

Ma_Ct	Ngay_Ct	So_Ct	Ma_Dt	Dien_Giai
PX	01/06/2013	PX002	N001	Xuất hàng hóa bán đại lý
PN	01/06/2013	PN001	N002	Nhập vật tư phục vụ sản xuất
PX	01/06/2013	PX003	N004	Xuất bán hàng hóa
PN	02/06/2013	PN002	N005	Nhập nguyên liệu
PX	05/06/2013	PX004	N004	Xuất bán hàng hóa

Tìm hiểu một số toán tử cao cấp:

- Toán tử AND và OR: Sử dụng các toán tử này để kết hợp nhiều điều kiện lọc.

+ Lập danh sách chứng từ xuất hàng cho công ty Cty TNHH một thành viên cấp nước Yên Bái vào ngày 01/06/2013.

```
SELECT Ngay_Ct, So_Ct, Dien_Giai, Ong_Ba
  FROM Ct
  WHERE Ma_Dt = 'N004' AND Ngay_Ct = '01/06/2013'
```

Kết quả:

Ngay_Ct	So_Ct	Dien_Giai	Ong_Ba
01/06/2013	PX003	Xuất bán hàng hóa	Phan Thế Anh

Lưu ý: Cty TNHH một thành viên cấp nước Yên Bái có mã là "N004"

+ Lập bảng kê xuất hàng chi tiết mặt hàng "Bánh trứng Custas (TP001)" hoặc "Bánh kem bơ (TP003)".

```
SELECT Ma_Vt, So_Luong, Don_Gia,
     So_Luong * Don_Gia AS Thanh_Tien
  FROM CtCt
  WHERE Ma_Vt = 'TP001' OR Ma_Vt = 'TP003'
```

Kết quả:

Ma_Vt	So_Luong	Don_Gia	Thanh_Tien
TP001	120	120,000	14,400,000
TP002	45	23,500	1,057,500
TP001	35	11,500	402,500
TP001	75	110,000	8,250,000
TP001	78	13,000	1,014,000

+ Lập bảng kê chứng từ xuất hàng cho Cty TNHH ABC (N001) hoặc các chứng từ không phải cho công ty này nhưng được thực hiện vào ngày 05/06/2013.

```
SELECT Ngay_Ct, So_Ct, Dien_Giai, Ma_Dt
```

```
  FROM CT
  WHERE Ma_Dt = 'N001'
    OR (Ma_Dt <> 'N001' AND Ngay_Ct = '05/06/2013')
```

Kết quả

Ngay_Ct	So_Ct	Dien_Giai	Ma_Dt
2013-05-31	PX001	Xuất hàng hóa bán đại lý	N001
2013-06-01	PX002	Xuất hàng hóa bán đại lý	N001
2013-06-05	PX004	Xuất bán hàng hóa	N004

- Toán tử IN: Sử dụng toán tử này để chỉ định một dãy điều kiện với bất kỳ giá trị nào trong dãy thỏa mãn. IN sử dụng một danh sách các giá trị được tách bởi dấu phẩy dưới ",", tất cả được đặt trong dấu ngoặc đơn.

Ví dụ:

Lập bảng kê xuất hàng chi tiết mặt hàng "Bánh trứng Custas (TP001)" hoặc "Bánh kem bơ (TP003)" sử dụng từ khóa IN.

```
SELECT Ma_Vt, So_Luong, Don_Gia,
     So_Luong * Don_Gia AS Thanh_Tien
  FROM CtCt
  WHERE Ma_Vt IN('TP001', 'TP003')
```

- Toán tử NOT: Phủ định biểu thức đứng ngay sau nó

Ví dụ:

+ Lập bảng kê chứng từ xuất hàng cho Cty TNHH ABC (N001) hoặc các chứng từ không phải cho công ty này nhưng được thực hiện vào ngày 05/06/2013 sử dụng từ khóa NOT.

```
SELECT Ngay_Ct, So_Ct, Dien_Giai, Ma_Dt
  FROM CT
  WHERE Ma_Dt = 'N001'
    OR (NOT(Ma_Dt = 'N001') AND Ngay_Ct = '05/06/2013')
```

- Toán tử LIKE và các ký tự đại diện: Sử dụng toán tử này để tìm dữ liệu gần đúng với giá trị tìm kiếm bằng cách kết hợp với các ký tự đại diện. Chỉ có thể áp dụng toán tử LIKE và các ký tự đại diện cho dữ liệu kiểu chuỗi.

+ Ví dụ về ký tự đại diện "%": Ký tự đại diện này đại diện cho phần còn lại của chuỗi.

Lập danh sách các khách hàng là loại hình doanh nghiệp "Cổ phần" trên địa bàn Hà Nội

```
SELECT RTRIM(Ten_Dt) + ' (' + RTRIM(Ma_Dt) + ')' AS Ten_Dt,
     So_Dt, Email
  FROM DmDt
```

```
  WHERE Ten_Dt LIKE N'%Cổ phần%' AND Dia_Chi = N'Hà Nội'
```

Kết quả:

Ten_Dt	So_Dt	Email
Cty Cổ phần CB thực phẩm miền bắc (N002)	046 3423 3438	

+ Ví dụ về ký tự đại diện "_": Ký tự đại diện này tương tự như ký tự "%" tuy nhiên nó chỉ đại diện cho một ký tự đơn.

Tìm những thành phẩm với mã có phần mở rộng là các số chạy nhỏ hơn 10.

```
SELECT Ma_Vt, Ten_Vt, Dvt, Quy_Cach
  FROM DmVt
  WHERE RTRIM(Ma_Vt) LIKE N'TP00_'
```

Kết quả:

Ma_Vt	Ten_Vt	Dvt	Quy_Cach
TP001	Bánh trứng Custas	Hộp	Hộp 20 cái
TP002	Bánh kem xốp	Gói	Gói 200 gram
TP003	Bánh kem bơ	Gói	Gói 150 gram
TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái

+ Ví dụ về ký tự đại diện "[ ]": Sử dụng để chỉ định một tập hợp các ký tự, một ký tự trong tập hợp phải thỏa mãn một ký tự tại một vị trí xác định trước (vị trí của ký tự đại diện).

Lập danh sách các công ty "TNHH" hoặc "Cổ phần" đóng trên địa bàn "Hà Nội"

```
SELECT RTRIM(Ten_Dt) + ' (' + RTRIM(Ma_Dt) + ')' AS Ten_Dt,
     So_Dt, Email
  FROM DmDt
  WHERE Ten_Dt LIKE N'%[Cổ phần, TNHH]%'
        AND Dia_Chi = N'Hà Nội'
```

Kết quả:

Ten_Dt	So_Dt	Email
Cty TNHH ABC (N001)	04 3640 0119	
Cty Cổ phần CB thực phẩm miền bắc (N002)	046 3423 3438	

Lưu ý: Không thể phủ nhận khả năng linh hoạt khi tìm kiếm với các ký tự đại diện, tuy nhiên việc tìm kiếm với chúng mất nhiều thời gian hơn cả so với những cách tìm kiếm đã giới thiệu.. sau đây là một số thủ thuật cần ghi nhớ khi sử dụng tìm kiếm bằng các ký tự đại diện:

- Không lạm dụng các ký tự đại diện nếu như có thể tìm kiếm bằng các toán tử khác.
- Nếu thực sự cần thiết mới sử dụng ký tự đại diện ở phần đầu của các mẩu tìm kiếm vì đây là cách tìm kiếm chậm nhất.
- Đặc biệt chú ý đến vị trí của ký tự đại diện vì nếu đặt sai vị trí bạn sẽ không có được kết quả như mong muốn.

2.3.4. Sắp xếp dữ liệu với ORDER BY

Trong những ví dụ ở các phần đã học kết quả truy vấn chưa được sắp xếp, chúng thường được hiển thị theo thứ tự trong bảng, đây có thể là trật tự dữ liệu được thêm vào bảng ban đầu. Tuy nhiên trật tự này có thể thay đổi nếu như bảng thường xuyên được cập nhật hoặc xóa, chúng ta có thể sử dụng câu truy vấn với ORDER BY để sắp xếp lại dữ liệu.

❖ ***Cú pháp:***

```
SELECT [DISTINCT] [TOP So_Ban_Ghi]
     <Danh_Sach_Truong>
  FROM <Ten_Bang>
  WHERE <Dieu_Kien_Loc>
  ORDER BY <Chi_Tieu_Sap_Xep> [ASC|DESC]
```

Giải thích:

- Chi_Tieu_Sap_Xep: Là danh sách tên cột hoặc thứ tự cột cần sắp xếp
- ASC: Chỉ định sắp xếp tăng dần
- DESC: Chỉ định sắp xếp giảm dần

Ví dụ:

- Lập danh sách thành phẩm mà công ty sản xuất, kết quả được sắp xếp theo thứ tự trong bảng chữ cái của tên thành phẩm.

```
SELECT Ma_Vt, Ten_Vt, Dvt, Quy_Cach
  FROM DmVt
  WHERE Ma_Vt LIKE 'TP%'
  ORDER BY Ten_Vt
```

Kết quả:

Ma_Vt	Ten_Vt	Dvt	Quy_Cach
TP003	Bánh kem bơ	Gói	Gói 150 gram
TP002	Bánh kem xốp	Gói	Gói 200 gram
TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái
TP001	Bánh trứng Custas	Hộp	Hộp 20 cái

Xem lại kết quả khi không sử dụng ORDER BY

Ma_Vt	Ten_Vt	Dvt	Quy_Cach
TP001	Bánh trứng Custas	Hộp	Hộp 20 cái
TP002	Bánh kem xốp	Gói	Gói 200 gram
TP003	Bánh kem bơ	Gói	Gói 150 gram
TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái

- Lập bảng kê chứng từ nhập xuất trong tháng 06 năm 2013, sắp xếp theo Ma_Dt và Ngay_Ct tăng dần.

```
SELECT Ngay_Ct, So_Ct, Ma_Dt, Dien_Giai
  FROM Ct
  WHERE MONTH(Ngay_Ct) = 6 AND YEAR(Ngay_Ct) = 2013
  ORDER BY Ma_Dt, Ngay_Ct
```

Kết quả:

Ngay_Ct	So_Ct	Ma_Dt	Dien_Giai
01/06/2013	PX002	N001	Xuất hàng hóa bán đại lý
01/06/2013	PN001	N002	Nhập vật tư phục vụ sản xuất
01/06/2013	PX003	N004	Xuất bán hàng hóa
05/06/2013	PX004	N004	Xuất bán hàng hóa
02/06/2013	PN002	N005	Nhập nguyên liệu

- Lập bảng kê chứng từ xuất bán thành phẩm, sắp xếp theo Ma_Vt và So_Luong giảm dần.

```
SELECT Ma_Vt, So_Luong, Don_Gia,
     So_Luong * Don_Gia AS Thanh_Tien
  FROM CtCt
  WHERE Ma_Vt LIKE 'TP%'
  ORDER BY Ma_Vt, So_Luong DESC
```

Kết quả:

Ma_Vt	So_Luong	Don_Gia	Thanh_Tien
TP001	120	120.000	14.400.000
TP001	78	13.000	1.014.000
TP001	75	110.000	8.250.000
TP001	35	11.500	402.500
TP002	45	23.500	1.057.500

TP004	44	34.000	1.496.000

2.3.5. Nhóm dữ liệu với mệnh đề GROUP BY

Với mệnh đề WHERE ở phần trước đã học chúng ta có thể thống kê xem có bao nhiêu lần vật tư có mã là 'VT001' được nhập xuất với câu lệnh như sau:

```
SELECT COUNT(*) AS So_Lan_Nx
  FROM CTCT
  WHERE Ma_Vt = 'VT001'
```

Tuy nhiên để thống kê xem mỗi vật tư được nhập xuất bao nhiêu lần chúng ta phải sử dụng mệnh đề GROUP BY.

❖ ***Cú pháp:***

```
SELECT <Danh_Sach_Truong>
  FROM <Ten_Bang>
  WHERE <Dieu_Kien_Loc>
  GROUP BY <Danh_Sach_Nhom>
  HAVING <Bieu_Thuc_Dieu_Kien>
  ORDER BY <Chi_Tieu_Sap_Xep>
```

Giải thích:

- <Danh_Sach_Nhom>: Danh sách cột, thứ tự cột cần nhóm dữ liệu
- <Bieu_Thuc_Dieu_Kien>: Biểu thức điều kiện lọc nhóm dữ liệu

Ví dụ:

- Thống kê xem mỗi nhà cung cấp, khách hàng đã phát sinh bao nhiêu chứng từ nhập xuất với công ty trong tháng 6 năm 2013.

```
SELECT Ma_Dt, COUNT(Ma_Dt) AS So_Ct_Nx
  FROM Ct
  WHERE MONTH(Ngay_Ct) = 6 AND YEAR(Ngay_Ct) = 2013
  GROUP BY Ma_Dt
```

Kết quả:

Ma_Dt	So_Ct_Nx
N001	1
N002	1
N004	2
N005	1

- Thống kê tổng số lượng nhập xuất kho của từng mặt hàng, kết quả sắp xếp tăng dần theo số lượng.

```
SELECT Ma_Vt, SUM(So_Luong) AS So_Luong
  FROM CtCt
  GROUP BY Ma_Vt
  ORDER BY So_Luong
```

Kết quả:

Ma_Vt	So_Luong
VT002	15
VT004	30
TP004	44
TP002	45
VT003	56
VT001	152
TP001	308

- Thống kê lượng nhập xuất hàng hóa theo từng kho và từng mặt hàng, kết quả sắp xếp theo mã kho và giảm dần theo số lượng.

```
SELECT Ma_Kho, Ma_Vt, SUM(So_Luong) AS So_Luong
  FROM CtCt
  GROUP BY Ma_Kho, Ma_Vt
  ORDER BY Ma_Kho, So_Luong DESC
```

Kết quả:

Ma_Kho	Ma_Vt	So_Luong
KTP	TP001	230
KTP	TP002	45
KTP	TP004	44
KVT	VT001	152
KVT	TP001	78
KVT	VT003	56
KVT	VT004	30
KVT	VT002	15

- Thống kê doanh số bán hàng theo từng mặt hàng, kết quả sắp xếp theo Ma_Vt và giảm dần theo doanh thu.

```
SELECT Ma_Vt, SUM(So_Luong * Don_Gia) AS Doanh_Thu
```

```
  FROM CtCt
  WHERE Ma_Vt LIKE 'TP%'
  GROUP BY Ma_Vt
  ORDER BY Doanh_Thu DESC
```

Kết quả:

Ma_Vt	Doanh_Thu
TP001	24.066.500
TP004	1.496.000
TP002	1.057.500

- Thống kê những mặt hàng có doanh số bán hàng lớn hơn 2 triệu, kết quả sắp xếp danh sách theo Ma_Vt.

```
SELECT Ma_Vt, SUM(So_Luong * Don_Gia) AS Doanh_Thu
  FROM CtCt
  WHERE Ma_Vt LIKE 'TP%'
  GROUP BY Ma_Vt
  HAVING SUM(So_Luong * Don_Gia) > 2000000
  ORDER BY Ma_Vt
```

Kết quả:

Ma_Vt	Doanh_Thu
TP001	24.066.500

- Lập danh sách những nhà cung cấp, khách hàng đã từng phát sinh từ 2 chứng từ nhập xuất trở lên với công ty. Kết quả sắp xếp giảm dần theo số chứng từ phát sinh.

```
SELECT Ma_Dt, COUNT(Ma_Dt) AS So_Ct_Nx
  FROM Ct
  GROUP BY Ma_Dt
  HAVING COUNT(Ma_Dt) >= 2
  ORDER BY So_Ct_Nx DESC
```

Kết quả

Ma_Dt	So_Ct_Nx
N001	2
N004	2

- Thống kê mặt hàng có mã ‘TP..’ bán chạy nhất.

```
SELECT TOP 1 Ma_Vt, SUM(So_Luong * Don_Gia) AS Doanh_Thu
  FROM CtCt
```

```
  WHERE Ma_Vt LIKE 'TP%'
  GROUP BY Ma_Vt
  ORDER BY Doanh_Thu DESC
```

Kết quả

Ma_Vt	Doanh_Thu
TP001	24.066.500

- Thống kê 2 mặt hàng có mã 'TP..' bán chậm nhất

```
SELECT TOP 2 Ma_Vt, SUM(So_Luong * Don_Gia) AS Doanh_Thu
  FROM CtCt
  WHERE Ma_Vt LIKE 'TP%'
  GROUP BY Ma_Vt
  ORDER BY Doanh_Thu ASC
```

Kết quả:

Ma_Vt	Doanh_Thu
TP002	1.057.500
TP004	1.496.000

➢ ***Lưu ý:***

- Các trường trong danh sách nhóm sau mệnh đề GROUP BY có thể không xuất hiện trong danh sách trường sau SELECT.

- Các trường không xuất hiện trong danh sách nhóm sau mệnh đề GROUP BY, nhưng xuất hiện trong danh sách trường sau SELECT thì phải được chỉ định bởi một hàm thống kê (phải là một trường tính toán). Một số hàm thống kế thường sử dụng.

Hàm	Giải thích
AVG()	Trả về giá trị trung bình của cột
COUNT()	Đếm số dòng trong cột
MAX()	Trả về giá trị lớn nhất của cột
MIN()	Trả về giá trị nhỏ nhất của cột
SUM()	Tính tổng giá trị của cột

- Sử dụng HAVING trong mệnh đề GROUP BY vì WHERE chỉ lọc dòng dữ liệu cụ thể không phải là nhóm dữ liệu.

- Biểu thức sau HAVING hỗ trợ tất cả các toán tử như biểu thức sau WHERE

- Khác biệt giữa HAVING và WHERE là WHERE lọc dữ liệu khi chưa nhóm còn HAVING lọc dữ liệu sau khi đã được nhóm.

- Chúng ta cũng có thể sử dụng ORDER BY cùng GROUP BY để sắp xếp nhóm dữ liệu

2.3.6. Query con

Tất cả các câu lệnh truy vấn (Query) mà chúng ta đã nghiên cứu qua các phần trước đều là các câu truy vấn đơn giản (các câu truy vấn từ các bảng cơ sở dữ liệu riêng lẻ). T-SQL còn cho phép bạn tạo ra các query con (SubQuery: Các query được nhúng vào query khác), để hiểu sâu về vấn đề này chúng ta nghiên cứu một ví dụ sau đây:

Từ bảng danh mục vật tư (DMVT) và bảng chi tiết chứng từ nhập xuất (CTCT) đã tạo lập ở các phần trước chúng ta cần lập danh sách những vật tư đã từng được nhập xuất. Đầu tiên hãy thực hiện ví dụ này bằng các Query đơn giản chúng ta làm như sau:

***Bước 1**:* Lấy mã những vật tư đã từng nhập xuất trong bảng CTCT

```
SELECT DISTINCT Ma_Vt FROM CTCT
```

Kết quả:

Ma_Vt
TP001
TP002
VT001
VT002

***Bước 2**:* Lấy danh sách vật tư có mã tương ứng trong danh sách vừa lọc

```
SELECT * FROM DMVT
  WHERE Ma_Vt IN('TP001', 'TP002', 'VT001', 'VT002')
```

Kết quả:

ID	Ma_Vt	Ten_Vt	Dvt	Quy_Cach	Ghi_Chu
1	TP001	Bánh trứng Custas	Hộp	Hộp 20 cái	NULL
2	TP002	Bánh kem xốp	Gói	Gói 200 gram	NULL
3	VT001	Bột mì loại 1	Kg	NULL	NULL
4	VT002	Hương liệu	Kg	NULL	NULL

Bây giờ có thể viết lại với Query con bằng cách kết hợp hai Query vừa thực hiện:

```
SELECT * FROM DMVT
  WHERE Ma_Vt IN(SELECT Ma_Vt FROM CTCT)
```

Kết quả cuối cùng cho tương tự như thực hiện bước 2

Với ví dụ trên chúng ta đã tìm hiểu cách lọc dữ liệu theo các Query con, ví dụ sau đây sẽ mô tả cách sử dụng Query con làm trường tính toán.

Từ bảng danh mục khách hàng, nhà cung cấp (DMDT) và bảng chứng từ nhập xuất (CT) hãy xem mỗi khách hàng, nhà cung cấp đã nhập xuất bao nhiêu chứng từ với công ty, query đơn giản chúng ta có thể viết như sau:

```
SELECT Ma_Dt, COUNT(*) AS So_Ct_Nx
  FROM CT
  GROUP BY Ma_Dt
```

Kết quả

Ma_Dt	So_Ct_Nx
N001	2
N002	1

Tuy nhiên với câu lệnh này bạn không có được các thông tin chi tiết về khách hàng, nhà cung cấp đó. Bây giờ hãy thử thực hiện với một query phức tạp hơn.

```
SELECT Ma_Dt, Ten_Dt, Dia_Chi,
    (
      SELECT COUNT(*)
        FROM CT
        WHERE DmDt.Ma_Dt = Ct.Ma_Dt
    ) AS So_Ct_Nx
  FROM DmDt
```

Kết quả

Ma_Dt	Ten_Dt	Dia_Chi	So_Ct_Nx
N001	Công ty TNHH ABC	Hà Nội	2
N002	Công ty Cổ phần CB thực phẩm miền bắc	Hà Nội	1

Một số ví dụ khác

- Thống kê tổng số lượng nhập xuất kho của từng mặt hàng, kết quả hiển thị bao gồm các thông tin: Ma_Vt, Ten_Vt, Dvt, So_Luong và sắp xếp tăng dần theo số lượng.

```
SELECT Ma_Vt, Ten_Vt, Dvt,
    (
      SELECT ISNULL(SUM(So_Luong), 0) AS So_Luong
        FROM CtCt WHERE CtCt.Ma_Vt = DmVt.Ma_Vt
    ) AS So_Luong
  FROM DmVt
  ORDER BY So_Luong
```

Kết quả:

Ma_Vt	Ten_Vt	Dvt	So_Luong
TP003	Bánh kem bơ	Gói	0
VT005	Sửa ông thọ	Thùng	0
VT002	Hương liệu	Kg	15
VT004	Socola nguyên liệu	Kg	30
TP004	Bánh Socola Vinasun	Hộp	44
TP002	Bánh kem xốp	Gói	45
VT003	Bột nở	Kg	56
VT001	Bột mì loại 1	Kg	152
TP001	Bánh trứng Custas	Hộp	308

- Thống kê doanh số bán hàng theo từng mặt hàng, kết quả hiển thị bao gồm các trường Ma_Vt, Ten_Vt, Dvt, Doanh_Thu được sắp xếp theo Ma_Vt và giảm dần theo doanh thu.

```
SELECT Ma_Vt, Ten_Vt, Dvt,
     (
       SELECT ISNULL(SUM(So_Luong * Don_Gia), 0) AS
Doanh_Thu
          FROM CtCt WHERE CtCt.Ma_Vt = DmVt.Ma_Vt
     ) AS Doanh_Thu
  FROM DmVt
  WHERE Ma_Vt LIKE 'TP%'
  ORDER BY Doanh_Thu DESC
```

Kết quả:

Ma_Vt	Ten_Vt	Dvt	Doanh_Thu
TP001	Bánh trứng Custas	Hộp	24.066.500
TP004	Bánh Socola Vinasun	Hộp	1.496.000
TP002	Bánh kem xốp	Gói	1.057.500
TP003	Bánh kem bơ	Gói	0

2.3.7. Các phép nối

Một trong những tính năng mạnh mẽ nhất của SQL là khả năng kết các bảng nhanh chóng trong các Query. Phép nối được thực hiện đơn giản bằng cách sử dụng câu lệnh

SELECT của SQL, việc tìm hiểu kỹ và thực hành tốt các phép nối là phần vô cùng quan trọng trong quá trình học SQL.

Để đảm bảo toàn vẹn, tránh dư thừa cơ sở dữ liệu luôn được chuẩn hóa vì vậy trong quá trình truy vấn dữ liệu thường xuyên phải lấy dữ liệu từ nhiều bảng khác nhau, với việc sử dụng các phép nối sẽ giúp người lập trình dễ dàng trích rút đầy đủ dữ liệu mà mình cần khai thác.

2.3.7.1. Tạo phép nối với mệnh đề WHERE

Với mệnh đề WHERE khi tạo kết nối phải chỉ định tất cả các bảng chứa dữ liệu cần truy vấn và cách chúng tạo ra quan hệ với nhau. Chúng ta tìm hiểu ví dụ sau:

Lập bảng kê chứng từ gồm các thông tin: Ngay_Ct, So_Ct, Dien_Giai, Ten_Dt, Ong_Ba, Dia_Chi.

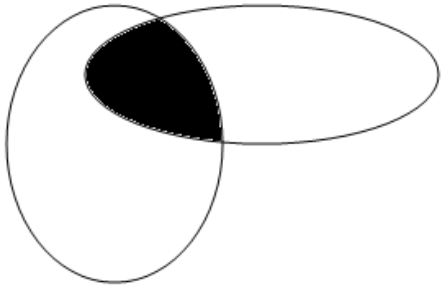
Phân tích ví dụ trên chúng ta thấy thông tin Ten_Dt được lấy từ bảng danh mục khách hàng, nhà cung cấp (DMDT), các thông tin còn lại lấy từ bảng chứng từ (CT), hai bảng này liên kết với nhau bằng trường Ma_Dt. Câu lệnh truy vấn như sau:

```
SELECT CT.Ngay_Ct, CT.So_Ct,
     CT.Dien_Giai, DMDT.Ten_Dt,
     CT.Ong_Ba, CT.Dia_Chi
  FROM DMDT, CT
  WHERE DMDT.Ma_Dt = CT.Ma_Dt
```

Kết quả:

Ngay_Ct	So_Ct	Dien_Giai	Ten_Dt	Ong_Ba	Dia_Chi
2013-01-06	PX001	Xuất hàng hóa bán đại lý	Công ty TNHH ABC	Lê Văn Khương	Hà Nội
2013-01-07	PX002	Xuất hàng hóa bán đại lý	Công ty TNHH ABC	Đào Thị Hạnh	Hà Nội
2013-01-06	PN001	Nhập vật tư phục vụ sản xuất	Công ty Cổ phần CB thực phẩm miền bắc	Tạ Thu Loan	Công ty

Như vậy kết quả truy vấn là phần dữ liệu có mối quan hệ giữa hai bảng cơ sở dữ liệu.



Một số ví dụ

- Lập bảng kê chứng từ nhập hàng chi tiết theo vật tư gồm có các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Ma_Dt, Ma_Vt, So_Luong, Don_Gia, Thanh_Tien.

```
SELECT Ct.Ma_Ct, Ct.Ngay_Ct,Ct.So_Ct,
     Ct.Dien_Giai, Ct.Ma_Dt,
     CtCt.Ma_Vt, CtCt.So_Luong,
     CtCt.Don_Gia,
     ISNULL(CtCt.Don_Gia * CtCt.So_Luong, 0) AS Thanh_Tien
  FROM Ct, CtCt
  WHERE Ct.Stt_Ct = CtCt.Stt_Ct AND Ct.Ma_Ct = 'PN'
  ORDER BY Ct.Ngay_Ct
```

Kết quả:

Ma_Ct	Ngay_Ct	So_Ct	Dien_Giai	Ma_Dt	Ma_Vt	So_Luong	Don_Gia	Thanh_Tien
PN	01/06/11	PN001	Nhập vật tư phục vụ sản xuất	N002	VT001	120	30.000	3.600.000
PN	01/06/11	PN001	Nhập vật tư phục vụ sản xuất	N002	VT002	15	120.000	1.800.000
PN	02/06/11	PN002	Nhập nguyên liệu	N005	VT004	30	75.000	2.250.000
PN	02/06/11	PN002	Nhập nguyên liệu	N005	VT001	32	31.000	992.000
PN	02/06/11	PN002	Nhập nguyên liệu	N005	VT003	56	120.000	6.720.000

- Lập bảng kê chứng từ theo tổng tiền (chi tiết theo chứng từ) bao gồm các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Ten_Dt, Tong_Tien.

```
SELECT MAX(Ct.Ma_Ct) AS Ma_Ct, MAX(Ct.Ngay_Ct) AS Ngay_Ct,
     MAX(Ct.Dien_Giai) AS Dien_Giai,
     MAX(DmDt.Ten_Dt) AS Ten_Dt,
     SUM(CtCt.Don_Gia * CtCt.So_Luong) AS Tong_Tien
  FROM Ct, CtCt, DmDt
  WHERE Ct.Stt_Ct = CtCt.Stt_Ct
       AND Ct.Ma_Dt = DmDt.Ma_Dt
```

```
GROUP BY CtCt.Stt_Ct
```

Kết quả:

Ma_Ct	Ngay_Ct	Dien_Giai	Ten_Dt	Tong_Tien
PX	31/05/11	Xuất hàng hóa bán đại lý	Cty TNHH ABC	15.457.500
PX	01/06/11	Xuất hàng hóa bán đại lý	Cty TNHH ABC	402.500
PN	01/06/11	Nhập vật tư phục vụ sản xuất	Cty Cổ phần CB thực phẩm miền bắc	5.400.000
PX	01/06/11	Xuất bán hàng hóa	Cty TNHH một thành viên cấp nước Yên Bái	9.746.000
PN	02/06/11	Nhập nguyên liệu	Cty Cổ phần Lạc Hồng	9.962.000
PX	05/06/11	Xuất bán hàng hóa	Cty TNHH một thành viên cấp nước Yên Bái	1.014.000

2.3.7.2. INNER JOIN (nội kết)

Tương tự như phép nối sử dụng mệnh đề WHERE, chúng ta có thể sử dụng INNER JOIN để chỉ ra phép kết nối rõ hơn. Viết lại ví dụ ở phần trên với INNER JOIN như sau:

```
SELECT CT.Ngay_Ct, CT.So_Ct,
     CT.Dien_Giai, DMDT.Ten_Dt,
     CT.Ong_Ba, CT.Dia_Chi
  FROM DMDT INNER JOIN CT ON DMDT.Ma_Dt = CT.Ma_Dt
```

Câu lệnh trên tương đối tường minh từ khóa INNER JOIN nằm giữa hai bảng kết nối sau FROM và biểu thức kết nối nằm sau từ khóa ON.

2.3.7.3. Sử dụng bí danh của bảng

Tương tự như các trường tính toán, khi sử dụng phép nối chúng ta cũng có thể tạo bí danh cho bảng, với việc sử dụng bí danh câu lệnh truy vấn sẽ ngắn ngọn và tường minh hơn, để tìm hiểu kỹ hơn về chủ đề này hãy viết lại các ví dụ phần trước với câu lệnh SQL có sử dụng bí danh.

Ví dụ:

Viết lại các ví dụ sau sử dụng bí danh bảng.

- Lập bảng kê chứng từ nhập hàng chi tiết theo vật tư gồm có các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Ma_Dt, Ma_Vt, So_Luong, Don_Gia, Thanh_Tien

```
SELECT Ct.Ma_Ct, Ct.Ngay_Ct,Ct.So_Ct,
     Ct.Dien_Giai, Ct.Ma_Dt,
     Ct0.Ma_Vt, Ct0.So_Luong, Ct0.Don_Gia,
     ISNULL(Ct0.Don_Gia * Ct0.So_Luong, 0) AS Thanh_Tien
  FROM Ct, CtCt AS Ct0
```

```
  WHERE Ct.Stt_Ct = Ct0.Stt_Ct AND Ct.Ma_Ct = 'PN'
  ORDER BY Ct.Ngay_Ct
```

- Lập bảng kê chứng từ theo tổng tiền (chi tiết theo chứng từ) bao gồm các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Ten_Dt, Tong_Tien.

```
SELECT MAX(Ct.Ma_Ct) AS Ma_Ct, MAX(Ct.Ngay_Ct) AS Ngay_Ct,
    MAX(Ct.Dien_Giai) AS Dien_Giai,
    MAX(Dt.Ten_Dt) AS Ten_Dt,
    SUM(Ct0.Don_Gia * Ct0.So_Luong) AS Tong_Tien
  FROM Ct, CtCt AS Ct0, DmDt AS Dt
  WHERE Ct.Stt_Ct = Ct0.Stt_Ct
      AND Ct.Ma_Dt = Dt.Ma_Dt
  GROUP BY Ct0.Stt_Ct
```

2.3.7.4. OUTER JOIN (Ngoại kết)

Với hai cách kết nối như đã trình bày ở trên kết quả dữ liệu truy vấn trả về là những dữ liệu có quan hệ với nhau, tuy nhiên trong thực tế chúng ta thường xuyên cần lấy cả những dữ liệu ở một bảng nào đó mà chúng chưa có quan hệ với các bảng còn lại, khi đó có thể sử dụng OUTER JOIN. Khi sử dụng OUTER JOIN bắt buộc phải kết hợp với một trong hai từ khóa là RIGHT và LEFT. Hãy cùng tìm hiểu ví dụ sau đây:

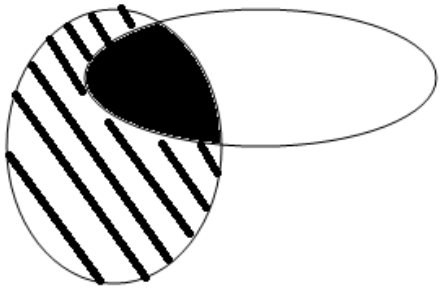
Hãy lập danh sách khách hàng, nhà cung cấp bao gồm thông tin mã, tên, địa chỉ và số lượng chứng từ nhập xuất (danh sách bao gồm cả những khách hàng, nhà cung cấp chưa có giao dịch).

```
SELECT dt.Ma_Dt, max(dt.Ten_Dt) AS Ten_Dt,
    MAX(dt.Dia_Chi) AS Dia_Chi,
    COUNT(ct.So_Ct) AS So_Ct_Nx
  FROM DmDt as dt LEFT OUTER JOIN CT ON dt.Ma_Dt = Ct.Ma_Dt
  GROUP BY dt.Ma_Dt
```

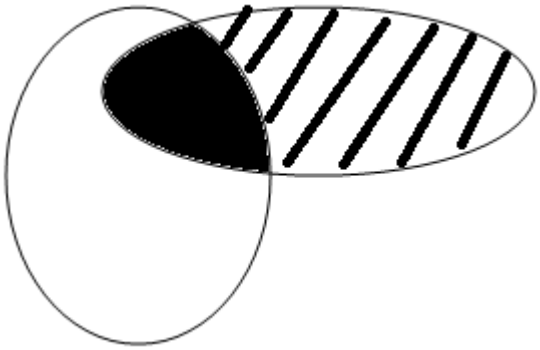
Kết quả

Ma_Dt	Ten_Dt	Dia_Chi	So_Ct_Nx
N001	Công ty TNHH ABC	Hà Nội	2
N002	Công ty Cổ phần CB thực phẩm miền bắc	Hà Nội	1
N003	Công ty liên doanh Việt Pháp	Hải Phòng	0
N004	Công ty TNHH một thành viên cấp nước Yên Bái	NULL	0

Như vậy toàn bộ dữ liệu có quan hệ giữ hai bảng và dữ liệu của bảng bên trái của OUTER JOIN được hiển thị trong kết quả truy vấn.



Trường hợp ngược lại chúng ta có thể dùng từ khóa RIGHT kết quả truy vấn sẽ lấy toàn bộ dữ liệu quan hệ giữa hai bảng và dữ liệu còn lại của bảng bên phải của OUTER JOIN.



Một số ví dụ

- Lập báo cáo tổng hợp nhập kho bao gồm các thông tin Ma_Vt, Ten_Vt, Dvt, So_Luong, Don_Gia, Thanh_Tien.

```
SELECT Ct0.Ma_Vt, Vt.Ten_Vt, Vt.Dvt,
     Ct0.So_Luong, Ct0.Don_Gia,
     Ct0.So_Luong * Ct0.Don_Gia AS Thanh_Tien
  FROM CtCt AS Ct0 LEFT OUTER JOIN DmVt AS Vt
     ON Ct0.Ma_Vt = Vt.Ma_Vt
     LEFT OUTER JOIN Ct ON Ct0.Stt_Ct = Ct.Stt_Ct
  WHERE Ct.Ma_Ct = 'PN'
```

Kết quả:

Ma_Vt	Ten_Vt	Dvt	So_Luong	Don_Gia	Thanh_Tien
VT001	Bột mì loại 1	Kg	120	30.000	3.600.000
VT002	Hương liệu	Kg	15	120.000	1.800.000
VT004	Socola nguyên liệu	Kg	30	75.000	2.250.000
VT001	Bột mì loại 1	Kg	32	31.000	992.000

VT003	Bột nở	Kg	56	120.000	6.720.000

- Lập bảng kê chứng từ nhập hàng chi tiết theo vật tư gồm có các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Ten_Dt, Ten_Vt, Dvt, So_Luong.

```
SELECT Ct.Ma_Ct, Ct.Ngay_Ct, Ct.Dien_Giai, Dt.Ten_Dt,
     Ct0.Ma_Vt, Vt.Ten_Vt, Vt.Dvt, Ct0.So_Luong
  FROM CtCt AS Ct0 LEFT OUTER JOIN Ct ON Ct0.Stt_Ct =
Ct.Stt_Ct
     LEFT OUTER JOIN DmDt AS Dt ON Ct.Ma_Dt = Dt.Ma_Dt
     LEFT OUTER JOIN DmVt AS Vt ON Ct0.Ma_Vt = Vt.Ma_Vt
  WHERE Ct.Ma_Ct = 'PN'
```

Kết quả:

Ma_Ct	Ngay_Ct	Dien_Giai	Ten_Dt	Ma_Vt	Ten_Vt	Dvt	So_Luong
PN	01/06/11	Nhập vật tư phục vụ sản xuất	Cty Cổ phần CB thực phẩm miền bắc	VT001	Bột mì loại 1	Kg	120
PN	01/06/11	Nhập vật tư phục vụ sản xuất	Cty Cổ phần CB thực phẩm miền bắc	VT002	Hương liệu	Kg	15
PN	02/06/11	Nhập nguyên liệu	Cty Cổ phần Lạc Hồng	VT004	Socola nguyên liệu	Kg	30
PN	02/06/11	Nhập nguyên liệu	Cty Cổ phần Lạc Hồng	VT001	Bột mì loại 1	Kg	32
PN	02/06/11	Nhập nguyên liệu	Cty Cổ phần Lạc Hồng	VT003	Bột nở	Kg	56

2.3.8. Kết hợp các query

Hầu hết các query SQL chứa một câu lệnh SELECT đơn vốn trả về dữ liệu từ một hoặc nhiều bảng. Tuy nhiên, SQL cũng cho phép bạn thực hiện nhiều câu query và kết quả trả về dưới dạng một tập hợp dữ liệu từ các query đơn.

Để kết hợp các Query đơn với nhau chúng ta dùng từ khóa UNION đặt giữa mỗi câu lệnh SELECT, cũng ở ví dụ phần trước chúng ta thêm một dòng tổng cộng để biết xem tổng số chứng từ nhập xuất của tất cả khách hàng nhà cung cấp.

Câu lệnh SQL như sau:

```
SELECT dt.Ma_Dt, max(dt.Ten_Dt) AS Ten_Dt,
     MAX(dt.Dia_Chi) AS Dia_Chi,
```

```
    COUNT(ct.So_Ct) AS So_Ct_Nx
  FROM DmDt as dt LEFT OUTER JOIN CT ON dt.Ma_Dt = Ct.Ma_Dt
  GROUP BY dt.Ma_Dt
UNION
SELECT '' AS Ma_Dt, N'Tổng cộng' AS Ten_Dt,
    '' AS Dia_Chi, COUNT(*) AS So_Ct_Nx
  FROM CT
```

Kết quả:

Ma_Dt	Ten_Dt	Dia_Chi	So_Ct_Nx
	Tổng cộng		3
N001	Công ty TNHH ABC	Hà Nội	2
N002	Công ty Cổ phần CB thực phẩm miền bắc	Hà Nội	1
N003	Công ty liên doanh Việt Pháp	Hải Phòng	0
N004	Công ty TNHH một thành viên cấp nước Yên Bái	NULL	0

❖ ***Lưu ý***:

- Một UNION phải có từ 2 câu lệnh SELECT trở lên, mỗi câu lệnh được tách bởi từ khóa UNION.
- Danh sách cột của các câu SELECT trong cùng một UNION phải tương ứng và cùng kiểu.
- Sử dụng UNION ALL khi muốn lấy tất cả các lần xuất hiện của các giá trị thỏa mãn.

2.4. CẬP NHẬT DỮ LIỆU (DATA MANIPULATION LANGUAGE)

2.4.1. Chèn dữ liệu

2.4.1.1. Chèn dữ liệu trực tiếp

❖ ***Cú pháp***:

```
INSERT INTO <Ten_Bang> [(Danh_Sach_Truong)]
  VALUE (<Danh_Sach_Gia_Tri>)
```

Giải thích:

- <Ten_Bang>: Tên bảng cần chèn dữ liệu
- [(Danh_Sach_Truong]): Danh sách trường cần chèn dữ liệu được liệt kê trong dấu ngoặc đơn, danh sách trường có thể bỏ trắng nếu chèn dữ liệu vào tất cả các dòng.
- <Danh_Sach_Gia_Tri>: Danh sách giá trị tương ứng với danh sách trường cần chèn dữ liệu.

Ví dụ:

Viết câu lệnh chèn dữ liệu trong danh sách sau vào bảng DmVt

Ma_Vt	Ten_Vt	Dvt	Quy_Cach
TP001	Bánh trứng Custas	Hộp	Hộp 20 cái
TP002	Bánh kem xốp	Gói	Gói 200 gram
TP003	Bánh kem bơ	Gói	Gói 150 gram
TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái

```
INSERT INTO DmVt
     (
       Ma_Vt,
       Ten_Vt,
       Dvt,
       Quy_Cach
     )
     VALUES
        (
          N'TP001',
          N'Bánh trứng Custas',
          N'Hộp',
          N'Hộp 20 cái'
        )
```

2.4.1.2. Chèn dữ liệu truy vấn

❖ ***Cú pháp***:

```
INSERT INTO <Ten_Bang> [(Danh_Sach_Truong)]
   SELECT <Danh_Sach_Truong_Truy_Van>
   FROM <Ten_Bang_Truy_Van>
   ...
```

Giải thích:

- Giá trị dữ liệu có thể được lấy từ một Query, danh sách trường trong câu Query này phải tương ứng với các trường cần chèn dữ liệu.
- Các tùy chọn và mệnh đề có thể sử dụng trong Query lấy dữ liệu như WHERE, GROUP BY.

2.4.1.3. Sao chép dữ liệu từ bảng khác

❖ ***Cú pháp***:

```
SELECT * INTO <Ten_Bang_Can_Chen_DL>
```

```
   FROM <Ten_Bang_Chua_DL>
```

Giải thích:

- <Ten_Bang_Can_Chen_DL>: Bảng cần chèn dữ liệu
- <Ten_Bang_Chua_DL>: Bảng chứa dữ liệu cần chèn
- Bất kỳ các tùy chọn và mệnh đề SELECT có thể sử dụng bao gồm WHERE và GROUP BY.

2.4.2. Cập nhật, xóa dữ liệu

2.4.2.1. Cập nhật chỉnh sửa dữ liệu

❖ ***Cú pháp**:*

```
UPDATE <Ten_Bang>
   SET  <Truong_Cap_Nhat1> = <Gia_Tri_Cap_Nhat1>,
      <Truong_Cap_Nhat2> = <Gia_Tri_Cap_Nhat2>,
      ....................................
      <Truong_Cap_NhatN> = <Gia_Tri_Cap_NhatN>
   WHERE <Dieu_Kien_Cap_Nhat>
```

Giải thích:

- <Ten_Bang>: Tên bảng cần cập nhật dữ liệu
- <Truong_Cap_Nhat>: Tên trường cần cập nhật dữ liệu
- <Gia_Tri_Cap_Nhat>: Giá trị mới cần cập nhật và trường tương ứng
- <Dieu_Kien_Cap_Nhat>: Biểu thức điều kiện cập nhật dữ liệu

***Ví dụ**:*

- Sửa lại tên cho vật tư có "Bánh trứng Custas" thành "Bánh trứng Custas loại 1" trong bảng DmVt.

```
UPDATE DmVt
   SET Ten_Vt = N'Bánh trứng Custas loại 1'
   WHERE Id = 1
```

- Sửa lại mã "Công ty TNHH ABC" từ "N001" thành "N009"
- Do mã số của "Công ty TNHH ABC" bị thay đổi ở phần trước hãy tìm và thay đổi tất cả thông tin này ở các bảng tương ứng.

2.4.2.2. Xóa dữ liệu

❖ ***Cú pháp**:*

```
DELETE FROM <Ten_Bang>
   WHERE <Dieu_Kien_Xoa>
```

Giải thích:

- <Ten_Bang>: Tên bảng cần xóa dữ liệu
- <Dieu_Kien_Xoa>: Biểu thức điều kiện xóa dữ liệu

Ví dụ:

- Do sai sót nhân viên kho nhập nhầm vật tư có mã "VT002" hãy xóa vật tư này khỏi DmVt.

```
DELETE FROM DmVt WHERE Ma_Vt = 'VT002'
```

- Do vật tư có mã "VT002" không tồn tại hãy tìm và xóa phát sinh của vật tư này trong các bảng tương ứng để đảm bảo toàn vẹn dữ liệu.

Lưu ý: khi thực thi câu lệnh UPDATE hoặc DELETE

- Nếu một câu lệnh cập nhật hoặc xóa dữ liệu không có mệnh đề WHERE nó sẽ áp dụng cho tất cả dữ liệu trong bảng.
- Trước khi thực hiện hai câu lệnh trên hãy sử dụng mệnh đề WHERE trong một lệnh SELECT để chắc chắn dữ liệu cần cập nhật hay xóa là chính xác.
- Không nên cập nhật hoặc xóa các dữ liệu có quan hệ với các bảng khác trong cùng một cơ sở dữ liệu.

2.5. BÀI TẬP THỰC HÀNH

1. Lập danh sách những mặt hàng chưa được bán trong tháng 06 năm 2013 bao gồm các thông tin Ma_Vt, Ten_Vt, Dvt.
2. Lập danh sách những mặt hàng bán chạy nhất trong tháng 06 năm 2013 bao gồm các thông tin Ma_Vt, Ten_Vt, Dvt, So_Luong, kết quả sắp xếp giảm dần theo số lượng.
3. Lập bảng kê chứng từ nhập kho tháng 06 năm 2013 bao gồm các thông tin Ma_Ct, Ngay_Ct, So_Ct, Dien_Giai, Tong_Tien, Ten_Dt.
4. Lập danh sách 02 khách hàng mua nhiều nhất trong tháng 06 năm 2013 (tính theo doanh số) bao gồm các thông tin Ma_Dt, Ten_Dt, Dia_Chi, So_Dt, Doanh_So.
5. Lập danh sách những khách hàng mua hàng có tổng trị giá lớn hơn 5 triệu trong tháng 06 năm 2013 để làm căn cứ tính chiết khấu, danh sách này giảm dần theo doanh số bán hàng. Kết quả hiển thị bao gồm các trường Ma_Dt, Ten_Dt, Dia_Chi, Doanh_So
6. Thông kê 2 loại vật tư nhập nhiều nhất trong tháng 06 bao gồm các thông tin Ma_Vt, Ten_Vt, Dvt, Tien_Nhap.
7. Lập danh sách những mặt hàng bán trong tháng 05 năm 2013 nhưng lại chưa được bán trong tháng 06 năm 2013, kết quả bao gồm các thông tin Ma_Vt, Ten_Vt, Dvt, So_Luong_T5, Don_Gia_T5, Thanh_Tien_T5.
8. Lập báo cáo tổng hợp nhập kho theo quy cách sau đây: Ma_Vt, Ten_Vt, Dvt, So_Luong_T5, So_Luong_T6.
9. Lập báo cáo nhập xuất vật tư đơn giản theo mẫu sau: Ma_Vt, Ten_Vt, Dvt, So_Luong_Nhap, So_Luong_Xuat.
10. Thống kê vật tư nhập theo kho với các thông tin sau: Ten_Kho, Ten_Vt, So_Luong, Don_Gia, Thanh_Tien.

Chương 3. TẠO CƠ SỞ DỮ LIỆU TRÊN SQL SERVER

3.1. CÁC LOẠI FILE LƯU TRỮ CƠ SỞ DỮ LIỆU

Là cơ sở dữ liệu được dùng cho dịch vụ **SQL Server Agent**, dịch vụ này sẽ tự động thực hiện các hành động mà người quản trị cơ sở dữ liệu đã đưa ra trong thời gian biểu như sao chép an toàn dữ liệu (**backup**), đồng bộ hóa dữ liệu (**replicate**), …

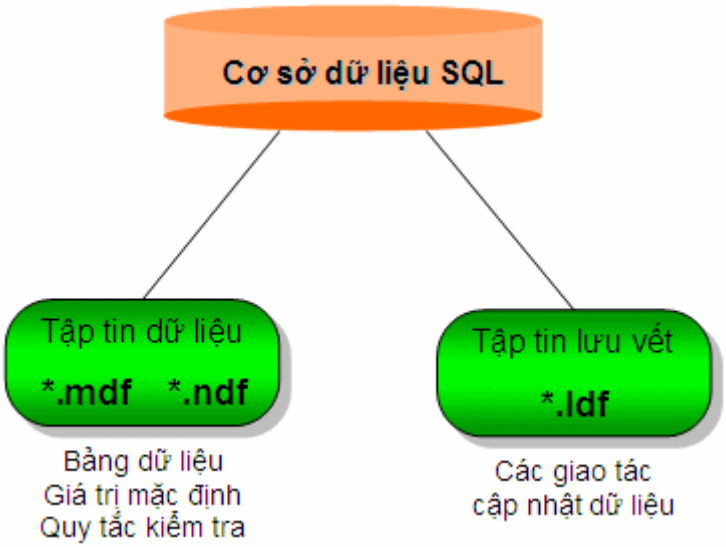
Bạn có thể hủy bỏ các cơ sở dữ liệu: **Pubs**, **NorthWind** trong trường hợp không muốn sử dụng nữa mà không làm ảnh hưởng đến hệ thống **Microsoft SQL Server**. Tuy nhiên, nếu dung lượng đĩa cứng bạn còn trống nhiều, bạn không nên hủy bỏ các cơ sở dữ liệu này.

3.1.1. Các tập tin vật lý lưu trữ cơ sở dữ liệu

Mặc dù phải quản lý nhiều đối tượng bên trong cơ sở dữ liệu nhưng Microsoft SQL Server chỉ tổ chức một vài tập tin dữ liệu (data files) vật lý để lưu trữ, đó là điều cải tiến từ phiên bản 7.0 trở lên của Microsoft SQL Server.

Một cơ sở dữ liệu trong **Microsoft SQL Server** tối thiểu phải bao gồm hai tập tin vật lý để lưu trữ dữ liệu.

- Một dùng để lưu trữ dữ liệu (**data file**).
- Một dùng để lưu vết các giao tác mà người dùng đã thực hiện (**transaction log file**).



Hình 3.1. Các loại tập tin lưu trữ dữ liệu của SQL Server

Các tập tin lưu trữ cơ sở dữ liệu bên trong **Microsoft SQL Server** được phân chia thành ba loại tập tin vật lý khác nhau.

3.1.2. Tập tin dữ liệu chính (Primary Data File)

Đây là tập tin chính dùng để lưu trữ các thông tin hệ thống của cơ sở dữ liệu và phần còn lại dùng lưu trữ một phần dữ liệu. Phần mở rộng của tập tin này thông thường là ***.MDF**.

3.1.3. Tập tin dữ liệu thứ yếu (Secondary Data Files)

Đây là tập tin dùng để lưu trữ các đối tượng dữ liệu không nằm trong tập tin dữ liệu chính. Loại tập tin này không bắt buộc phải có khi tạo mới cơ sở dữ liệu. Phần mở rộng của tập tin này thông thường là ***.NDF**.

3.1.4. Tập tin lưu vết (Log Files)

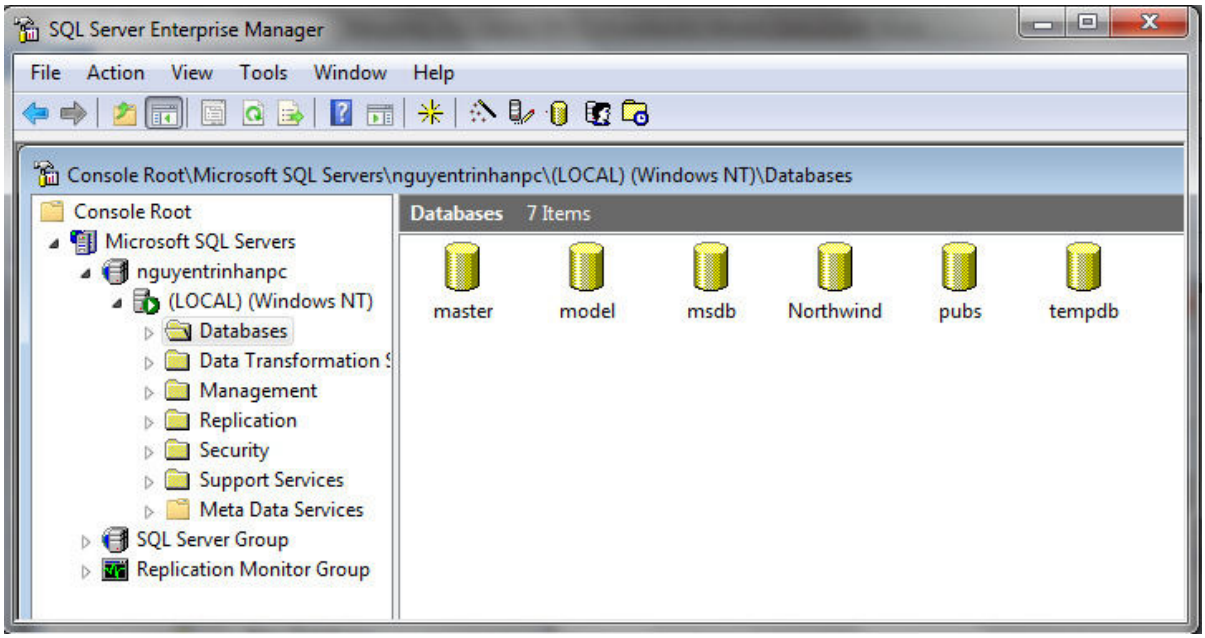
Đây là tập tin dùng để lưu vết các giao tác, là những hành động cập nhật dữ liệu (thêm, sửa, xóa) vào các bảng do người dùng tác động trên cơ sở dữ liệu. Tập tin này hỗ trợ cho phép bạn có thể hủy bỏ (**Rollback**) các thao tác cập nhật dữ liệu vừa mới thực hiện. Điều này giúp cho dữ liệu không hề bị thay đổi. Phần mở rộng của tập tin này thông thường là ***.LDF**.

Mặc định các tập tin dữ liệu của cơ sở dữ liệu **Microsoft SQL Server** sẽ được lưu trữ trong thư mục **C:\Program Files\Microsoft SQL Server\MSSQL\Data** hoặc đường dẫn mà bạn đã chỉ định lại trong quá trình cài đặt **Microsoft SQL Server**.

3.2. TẠO CƠ SỞ DỮ LIỆU

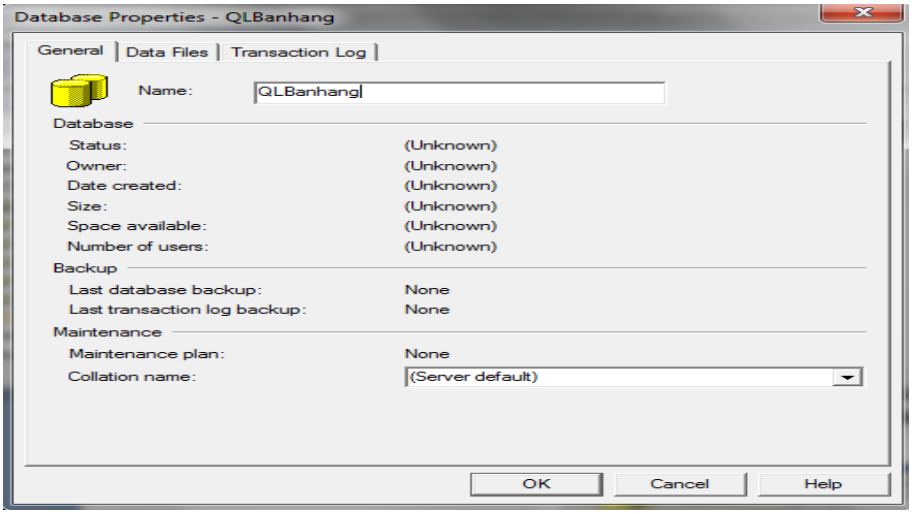
3.2.1. Sử dụng tiện ích Enterprise Manager:

- Khởi động tiện ích **Enterprise Manager**, chọn một **Microsoft SQL Server** đã được đăng ký quản trị trước đó. Click chuột phải trên đối tượng **Database**, chọn **New Database**,



Hình 3.2. Chức năng New Database

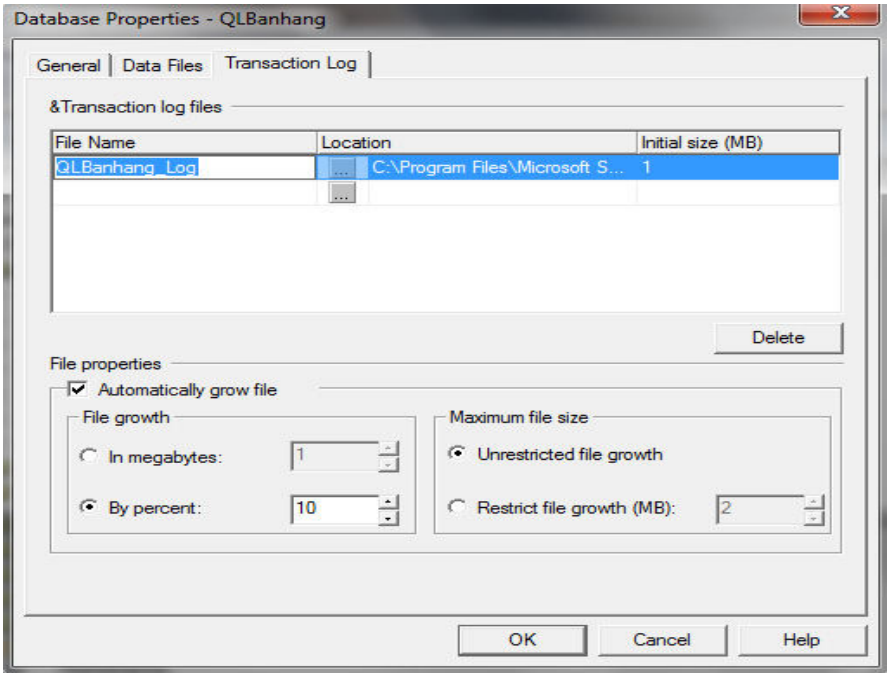
Xuất hiện hộp hội thoại:



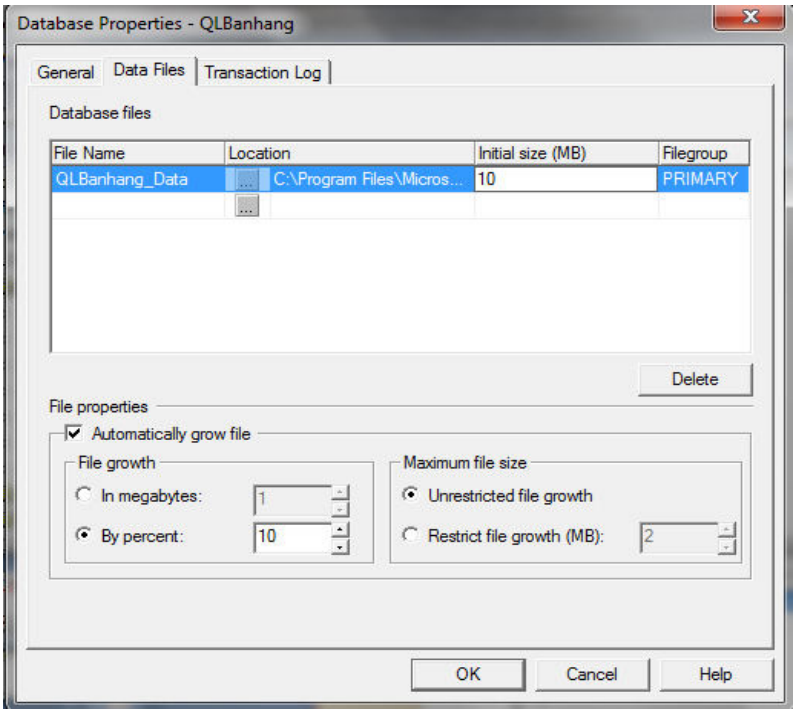
Hình 3.3. Các thuộc tính cơ sở dữ liệu - tab General

- Chọn tab **General**, nhập vào tên cơ sở dữ liệu muốn tạo, ví dụ **QLBanHang**, chỉ định kích thước ban đầu khởi tạo của tập tin dữ liệu chính, thay đổi các thuộc tính khác (nếu cần).

- Chọn tab **Transaction Log** để thay đổi các thuộc tính của tập tin lưu vết.



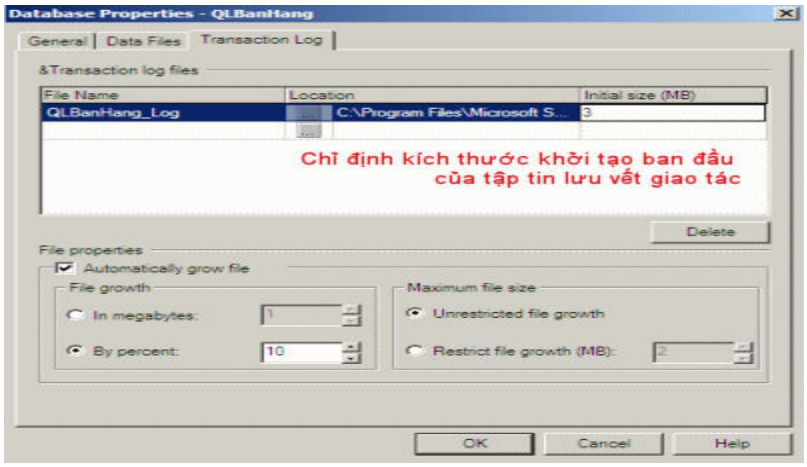
- Chọn tab **Data Files** để chỉ định kích thước ban đầu khi khởi tạo của tập tin dữ liệu chính, thay đổi các thuộc tính khác (nếu cần).



Hình 3.4. Các thuộc tính cơ sở dữ liệu - tab Data Files

- Chọn tab **Transaction Log** để thay đổi kích thước và các thuộc tính của tập tin lưu vết giao tác.

- Tại tab **Transaction Log** chỉ định kích thước ban đầu khi khởi tại của tập tin lưu vết giao tác, thay đổi các thuộc tính khác (nếu cần).



Hình 3.5. Các thuộc tính cơ sở dữ liệu - tab Transaction Log

Nhấn **OK** để hệ thống bắt đầu thực hiện việc khởi tạo cơ sở dữ liệu đã chỉ định.

Thời gian thực hiện việc tạo cơ sở dữ liệu tùy thuộc vào kích thước của cơ sở dữ liệu muốn tạo.

3.2.2. Sử dụng câu lệnh Transaction - SQL:

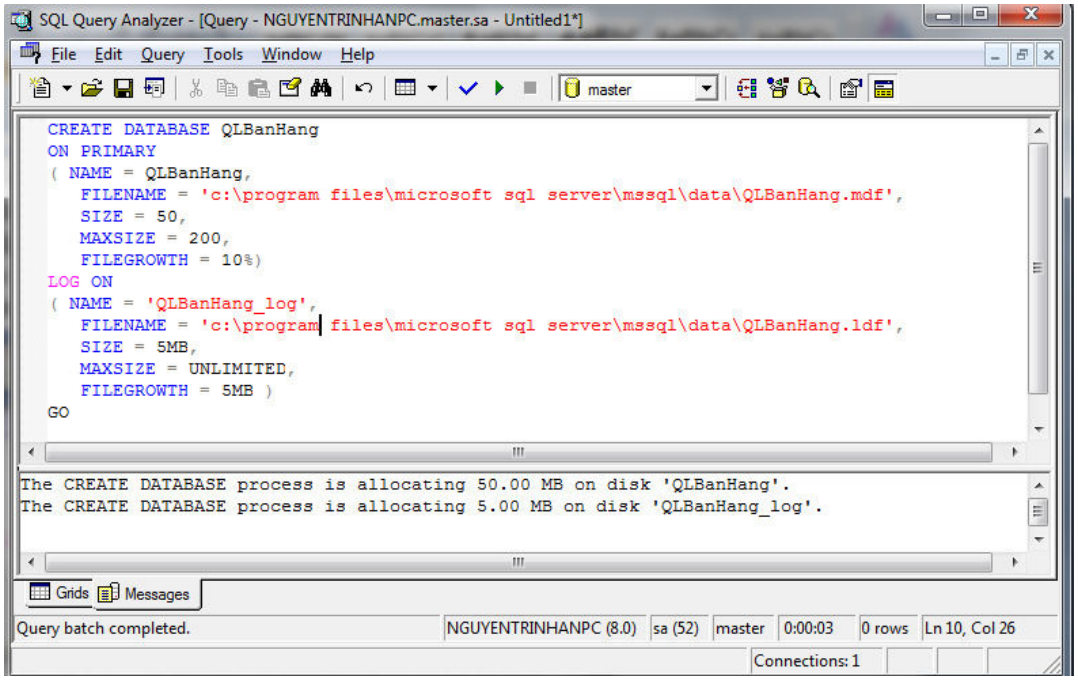
Ngoài ra, bạn có thể tạo mới một cơ sở dữ liệu bằng câu lệnh **CREATE DATABASE** được thực hiện trong tiện ích **Query Analyzer**.

- **Cú pháp:**

```
CREATE DATABASE database_name
[ ON [ < filespec > [ ,...n ] ]
[ , < filegroup > [ ,...n ] ]
]
[ LOG ON { < filespec > [ ,...n ] } ]
[ COLLATE collation_name ]
[ FOR LOAD | FOR ATTACH ]
< filespec > ::=
[ PRIMARY ]
( [ NAME = logical_file_name , ]
FILENAME = 'os_file_name'
[ , SIZE = size ]
[ , MAXSIZE = { max_size | UNLIMITED } ]
[ , FILEGROWTH = growth_increment ] ) [ ,...n ]
< filegroup > ::=
FILEGROUP filegroup_name < filespec > [ ,...n ]
```

Ví dụ

Tạo một cơ sở dữ liệu có tên là QLBanHang với kích thước ban đầu lúc khởi tạo của tập tin dữ liệu chính là 50MB, tự động tăng kích thước lên 10% khi dữ liệu bị đầy, kích thước tăng trưởng tập tin dữ liệu tối đa không quá 200MB, tập tin lưu vết với kích thước ban đầu lúc khởi tạo là 10MB, tự động tăng kích thước tập tin lên 5MB khi dữ liệu bị đầy, kích thước tăng trưởng tập tin không giới hạn. Bạn thực hiện câu lệnh như sau:



Hình 3.6. Tạo cơ sở dữ liệu QLBanHang bằng tiện ích Query Analyzer

Như vậy, bạn có hai cách để thực hiện các hành động trong **Microsoft SQL Server**: hoặc thực hiện các thao tác trong tiện ích **Enterprise Manager** hoặc thực hiện các câu lệnh **T-SQL** (**Transaction SQL**) trong tiện ích **Query Analyzer** để tạo ra các đối tượng bên trong **Microsoft SQL Server**. Để môi trường làm việc trở nên sinh động hơn, bạn nên phối hợp sử dụng cả hai tiện ích nêu trên.

3.3. XÓA, SỬA, ĐỔI TÊN, XEM THÔNG TIN CƠ SỞ DỮ LIỆU

3.3.1. Xóa cơ sở dữ liệu

Một cơ sở dữ liệu sau khi tạo xong sau một thời gian bạn không còn khai thác các dữ liệu bên trong đó thì bạn có thể hủy bỏ để cải thiện dung lượng đĩa cứng.

Tuy nhiên phải chắc rằng các thông tin dữ liệu trong cơ sở dữ liệu mà bạn dự định xóa sẽ không còn hữu ích về sau. Bởi vì bạn không thể khôi phục lại sau khi đã xóa.

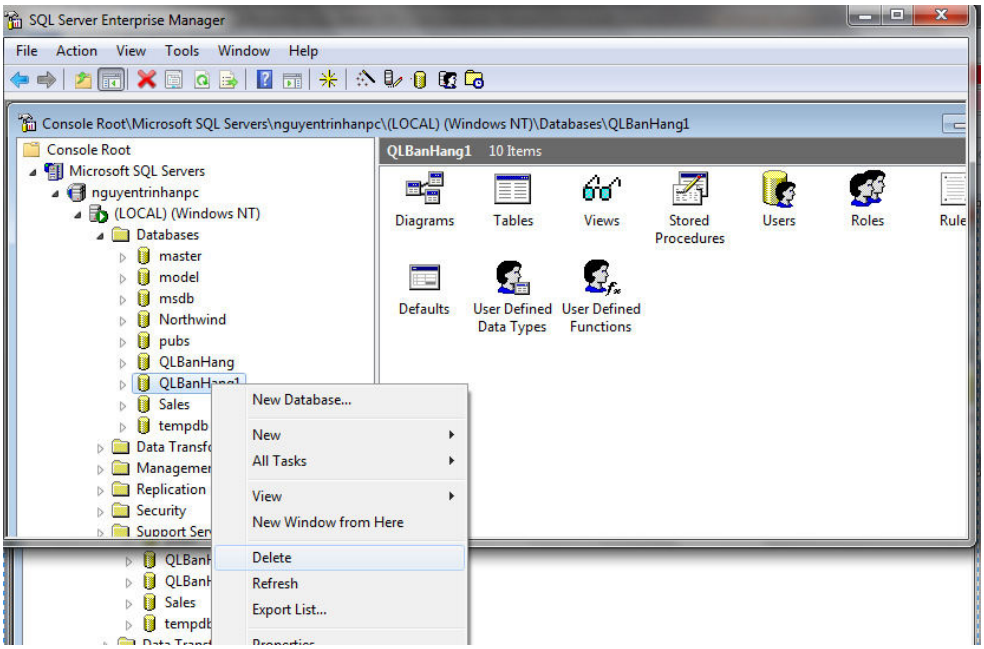
Để hủy bỏ cơ sở dữ liệu trong **Microsoft SQL Server**, bạn có thể sử dụng nhiều cách: hoặc sử dụng câu lệnh **T-SQL** (**Transaction SQL**) trong tiện ích **Query Analyzer** hoặc sử dụng chức năng từ thực đơn ngữ cảnh trong tiện ích **Enterprise Manager**.

Ví dụ

Hủy bỏ cơ sở dữ liệu QLBanHang đã tạo trong **Microsoft SQL Server**.

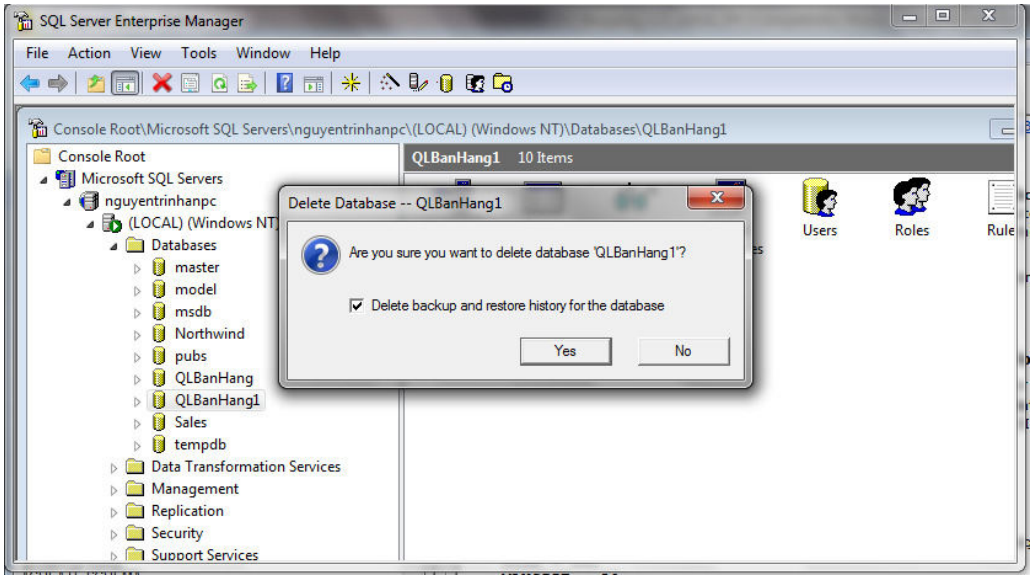
3.3.1.1. Sử dụng tiện ích Enterprise Manager

Khởi động tiện ích **Enterprise Manager** trong **Microsoft SQL Server**.



Hình 3.7. Xóa cơ sở dữ liệu QLBanHang bằng tiện ích Enterprise Manager

Chọn cơ sở dữ liệu QLBanHang, nhấn phím **Delete** hoặc click chuột phải tại cơ sở dữ liệu QLBanHang, chọn **Delete**, xuất hiện thông báo:



Hình 3.8. Xác nhận đồng ý hủy bỏ cơ sở dữ liệu

3.3.1.2. Sử dụng tiện ích Query Analyzer

Khởi động tiện ích **Query Analyzer** trong **Microsoft SQL Server**.

Xóa cơ sở dữ liệu QLBanHang bằng tiện ích Query Analyzer

Hệ thống **Microsoft SQL Server** không cho phép người dùng hủy bỏ các cơ sở dữ liệu hệ thống như **Master**, **Model**, **Tempdb** bởi vì các cơ sở dữ liệu luôn được hệ thống **Microsoft SQL Server** sử dụng.

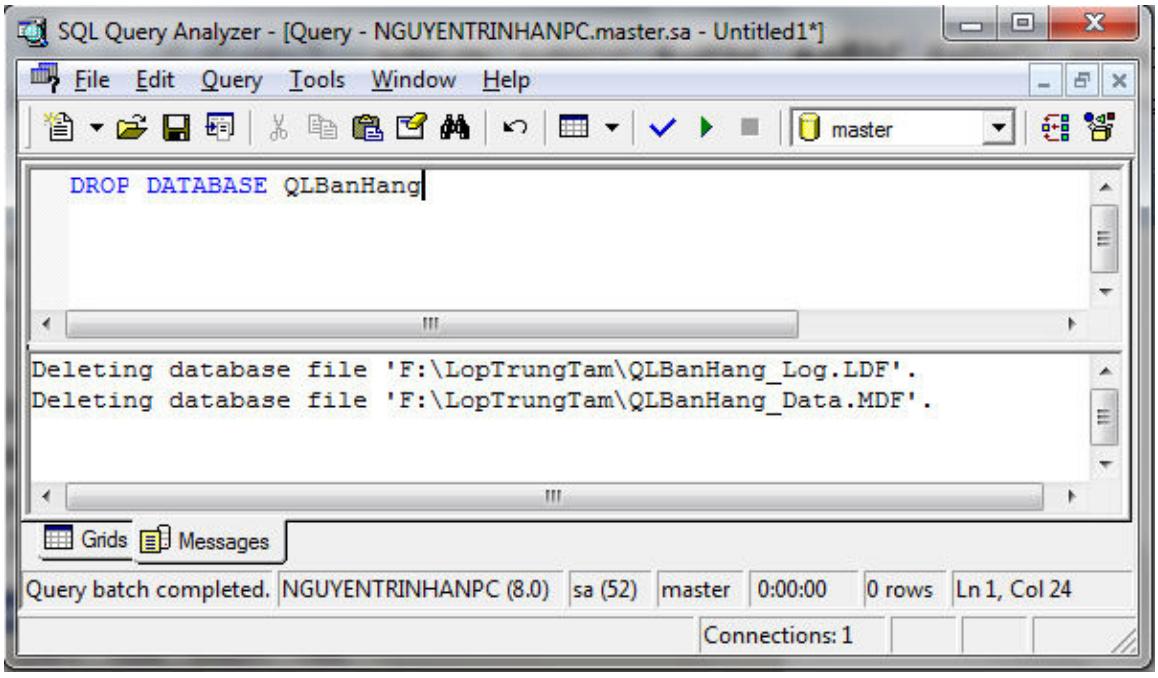
Việc hủy bỏ một cơ sở dữ liệu thành công phải đảm bảo không còn người dùng nào truy cập vào cơ sở dữ liệu đó. Nếu còn người dùng truy cập cơ sở dữ liệu mà bạn muốn xóa, hệ thống sẽ lỗi.

- ***Lưu ý***

Bạn không nên xóa cơ sở dữ liệu bằng cách sử dụng **Windows Explorer** hoặc **Windows Commander** để hủy bỏ các tập tin dữ liệu trong thư mục **C:\Program Files\Microsoft SQL Server\MSSQL\Data** vì thực hiện như thế sẽ ảnh hưởng đến hệ thống cơ sở dữ liệu **Microsoft SQL Server**.

Cú pháp:

DROP DATABASE *database_name* [**,**...*n*]



Hình 3.9. Hộp thoại SQL Query Analyer

3.3.2. Đổi tên Database (Rename)

Để đổi tên Database trong hệ quản trị cơ sở dữ liệu Sql Server 2008 và máy tính, các bước thực hiện như sau:

Bước 1: Chọn Database cần xóa trong Object Explorer.
Bước 2: Nhấp phải chuột vào Database này chọn Remane
Bước 3: Nhập tên mới và nhấp Ok để thực hiện đổi tên.

Chương 4. TẠO VÀ QUẢN LÝ BẢNG (TABLE)

4.1. TẠO BẢNG (TABLE)

4.1.1. Khái niệm về bảng

Bảng trong Microsoft SQL Server dùng cho việc lưu trữ các thông tin dữ liệu của những đối tượng, thực thể trong thế giới thực muốn được lưu trữ vào máy tính.

***Ví dụ**:*

Thông tin các khách hàng, nhà cung cấp, hóa đơn xuất hàng, hóa đơn nhập hàng, … các thông tin này sẽ được tổ chức thành các dòng (row) và các cột (column) mà thông thường định dạng của nó sẽ gần giống như một danh sách trong bảng tính Excel của các bạn thường dùng.

Mỗi dòng dữ liệu trong bảng thường phải là duy nhất do đó sẽ có một hoặc nhiều cột bên trong bảng sẽ tham gia làm khóa chính (**primary key**).

Giá trị dữ liệu tại các cột tham gia làm khóa chính là duy nhất không bị trùng lắp bên trong bảng.

Hầu như các bảng trong một cơ sở dữ liệu sẽ có các quan hệ với các bảng khác nhằm kiểm tra tính tồn tại dữ liệu (**foreign key**) và trao đổi, chia sẻ thông tin với nhau.

***Ví dụ**:*

Trong cơ sở dữ liệu quản lý bán hàng, bảng nhà cung cấp sẽ có quan hệ với bảng đơn đặt hàng để nói rằng việc đặt hàng cho những nhà cung cấp ứng với các loại vật tư nào.

❖ *Các thuộc tính của bảng:*

Trước khi thực hiện việc tạo cấu trúc bảng, bạn xem xét một số các thuộc tính liên quan đến bảng.

Mục đích của nhằm giúp bạn hiểu rõ về các thuộc tính liên quan đến bảng để có thể thực hiện việc tạo cấu trúc bảng được tốt hơn.

Các thuộc tính cơ bản của một bảng dữ liệu bao gồm:

• Tên bảng (table name):

Là tên của bảng dữ liệu do bạn quy định, độ dài không quá 128 ký tự. Tên bảng phải duy nhất bên trong phạm vi của người đã tạo ra nó trong một cơ sở dữ liệu.

Điều này có nghĩa là nếu người sử dụng ND1 đã tạo bảng KHACHHANG thì ND2 cũng được phép tạo ra bảng KHACHHANG khác trong cùng một cơ sở dữ liệu.

• Tên cột (column name):

Là tên của các cột bên trong bảng, tên của các cột bên trong bảng phải duy nhất

4.1.2. Tạo cấu trúc bảng dữ liệu

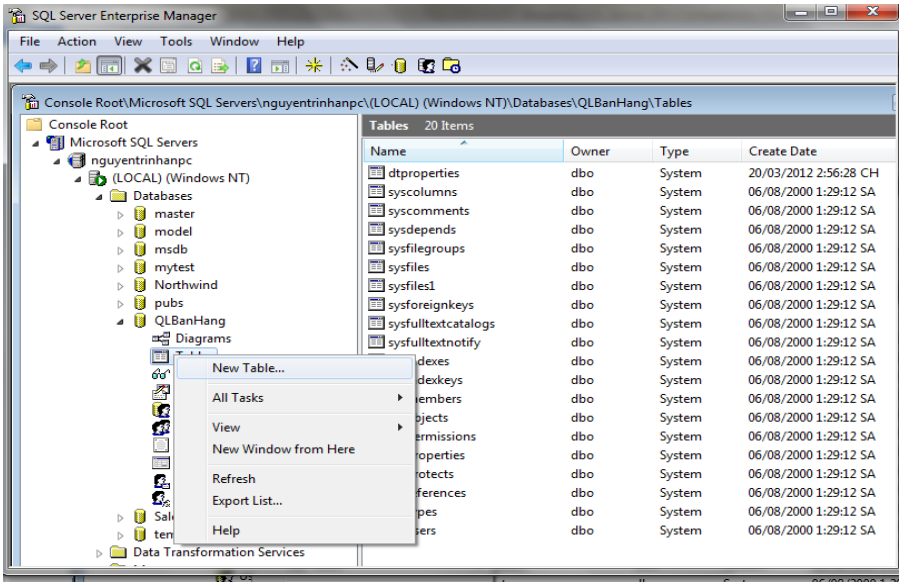
Để tạo cấu trúc bảng, bạn có hai cách thực hiện. Đầu tiên là tạo cấu trúc bảng bằng tiện ích Enterprise Manager.

Kể từ bây giờ xem như bạn đã đăng ký quản trị một Microsoft SQL Server và bên trong Microsoft SQL Server này, cơ sở dữ liệu quản lý bán hàng (QLBanHang) đã được tạo lập.

Các bảng dữ liệu và những đối tượng khác ở các bài hướng dẫn kế tiếp sẽ được tạo ra bên trong cơ sở dữ liệu QLBanHang này.

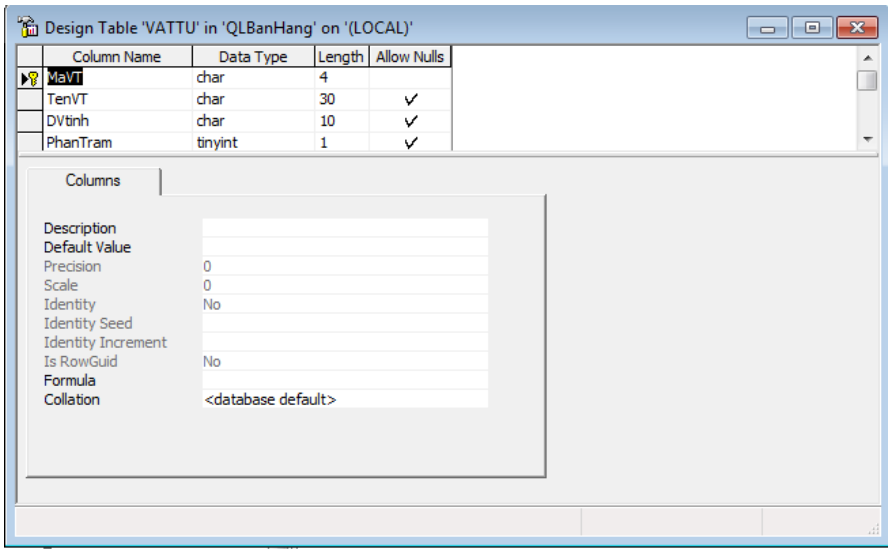
Các bước thực hiện việc tạo bảng dữ liệu trong **Enterprise Manager**:

• Trong ứng dụng **Enterprise Manager**, mở rộng cơ sở dữ liệu QLBanHang để thấy các đối tượng dữ liệu bên trong. Click chuột phải trên đối tượng **Tables**, chọn **New Table** trong thực đơn tắt.



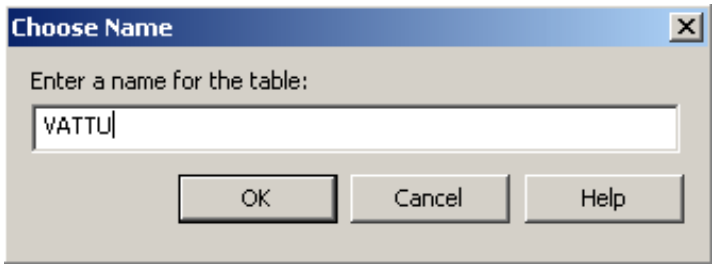
Hình 4.1. Chọn chức năng New Table để tạo mới bảng

• Trong màn hình thiết kế cấu trúc bảng (design table), lần lượt nhập vào tên các cột bên trong bảng, chọn lựa các kiểu dữ liệu tương ứng thích hợp và chỉ định các thuộc tính cần thiết cho các cột bên trong bảng. Sau cùng định nghĩa khóa chính cho bảng.



Hình 4.2. Thiết kế cấu trúc bảng

• Lưu lại cấu trúc bảng vừa định nghĩa kết thúc quá trình tạo cấu trúc bảng bằng tiện ích **Enterprise Manager**.



Hình 4.3. Chỉ định tên bảng mới

Ngoài ra bạn cũng có thể tạo cấu trúc bảng bằng câu lệnh **CREATE TABLE**, cú pháp đầy đủ của câu lệnh này rất phức tạp và rất dài. Do đó, sẽ hướng dẫn bạn từng thành phần nhỏ bên trong câu lệnh **CREATE TABLE** để bạn từng bước làm quen với các cú pháp của câu lệnh này. Sau đó bạn có thể kết hợp các thành phần này lại trong một câu lệnh **CREATE TABLE** để xây dựng cấu trúc bảng dữ liệu hoàn chỉnh.

4.1.2.1. Tạo cấu trúc bảng đơn giản:

Với cú pháp **CREATE TABLE** bên dưới cho phép bạn tạo ra cấu trúc bảng gồm: tên bảng, tên các cột cùng với kiểu dữ liệu tương ứng và chỉ định giá trị dữ liệu tại các cột bên trong bảng không được phép bỏ trống. Mặc định giá trị dữ liệu tại một cột không tham gia làm khóa chính của bảng được phép bỏ trống dữ liệu khi thêm mới.

Cú pháp:

```
CREATE TABLE Tên_bảng
(
      Tên_cột1  Kiểu_dữ_liệu [NOT NULL],
      Tên_cột2  Kiểu_dữ_liệu [NOT NULL],
      [...]
)
```

Trong đó:

• Tên bảng, tên cột và kiểu dữ liệu đã được mô tả trong phần các thuộc tính của bảng.

• Từ khóa **NOT NULL** chỉ định không cho phép dữ liệu tại cột được phép bỏ trống.

***Ví dụ**:*

Để tạo bảng có tên là VATTU (vật tư) gồm có những cột và kiểu dữ liệu mô tả như trên, chỉ có dữ liệu tại cột phần trăm (Phantram) là được phép bỏ trống. Bạn thực hiện câu lệnh **CREATE TABLE** như sau:

```
CREATE TABLE VATTU
(
      Mavtu     CHAR(4)       NOT NULL,
      Tenvtu    CHAR(30)      NOT NULL,
      Dvtinh    CHAR(10)      NOT NULL,
      Phantram  TinyInt
)
```

4.1.2.2. Tạo cấu trúc bảng có giá trị mặc định:

Với cú pháp **CREATE TABLE** bên dưới cho phép bạn tạo ra cấu trúc bảng với định nghĩa giá trị mặc định cho cột. Giá trị mặc định là những giá trị sẽ được gán vào bên trong cột dữ liệu của bảng trong trường hợp dữ liệu tại cột đó bỏ trống khi thêm dòng dữ liệu mới vào bảng.

- **Cú pháp**:

```
CREATE TABLE Tên_bảng
(
      Tên_cột1  Kiểu_dữ_liệu DEFAULT Giá_trị | Hàm,
      Tên_cột2  Kiểu_dữ_liệu [NOT NULL],
      [...]
)
```

Trong đó:

• **Giá trị:** là giá trị cụ thể sẽ được gán vào cột.

• **Hàm:** là tên hàm cùng với tham số (nếu có), tuy nhiên kiểu dữ liệu mà hàm trả về phải cùng với kiểu dữ liệu mà cột sẽ lưu trữ.

***Ví dụ**:*

Để tạo bảng có tên VATTU (vật tư) gồm có những cột và kiểu dữ liệu như mô tả trên, có thêm yêu cầu là tạo giá trị mặc định tại cột phần trăm (Phantram) là 20. Bạn thực hiện câu lệnh **CREATE TABLE** như sau:

```
CREATE TABLE VATTU
(
      Mavtu     CHAR(4)        NOT NULL,
      Tenvtu    CHAR(30)       NOT NULL,
      Dvtinh    CHAR(10)       NOT NULL,
      Phantram  TinyInt  DEFAULT 20
)
```

4.2. THAY ĐỔI CẤU TRÚC BẢNG (TABLE)

Trong thực tế việc thay đổi cấu trúc bảng vẫn thường được thực hiện đối với các bảng đã có chứa nhiều dữ liệu. Trong trường hợp này bạn tuyệt đối không hủy bỏ bảng hiện có và tạo lại cấu trúc mới bởi vì làm như thế là bạn sẽ mất đi tất cả các dữ liệu đang hiện có bên trong bảng.

Bạn có thể thay đổi cấu trúc bảng bằng tiện ích **Enterprise Manager** trong màn hình thiết kế bảng. Tuy nhiên, trong bài này sẽ hướng dẫn bạn các tính năng mà câu lệnh **ALTER TABLE** sẽ mang đến khi bạn thực hiện việc thay đổi cấu trúc bảng.

4.2.1. Thêm một cột mới trong bảng:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn thêm vào một hoặc nhiều cột mới vào trong bảng hiện đang có. Tên các cột mới phải khác tên các cột hiện đang có bên trong bảng. Mặc nhiên dữ liệu của các cột mới thêm vào phải được phép bỏ trống.

- **Cú pháp**:

```
ALTER TABLE Tên_bảng
      ADD Tên_Cột Kiểu_dữ_Liệu[,…]
```

Trong đó:

- **Tên cột:** tên của cột mới được thêm vào bảng.
- **Kiểu dữ liệu:** kiểu dữ liệu tương ứng của cột mới.

***Ví dụ**:*

Để thêm vào bảng DONDH (đơn đặt hàng) một cột ngày dự kiến nhận hàng có kiểu dữ liệu là ngày. Bạn thực hiện câu lệnh **ALTER TABLE** như sau:

- **Cú pháp**

```
ALTER TABLE DONDH
      ADD NGAYDKNH   DATETIME
```

4.2.2. Hủy bỏ cột hiện có bên trong bảng:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn hủy bỏ một hoặc nhiều cột hiện có bên trong bảng. Nên nhớ việc hủy bỏ các cột đồng nghĩa với các dữ liệu mà cột hiện đang lưu trữ cũng sẽ bị mất theo. Do đó cần thận trọng khi sử dụng câu lệnh này:

❖ **Cú pháp**:

```
ALTER   TABLE Tên_Bảng
        DROP    COLUMN Tên_cột [,…]
```

Trong đó:

• **Tên cột:** tên cột sẽ bị hủy bỏ ra khỏi bảng.

Ví dụ:

Để hủy bỏ cột ngày dự kiến nhận hàng trong bảng DONDH (đơn đặt hàng) vừa thêm ở ví dụ trên, bạn thực hiện câu lệnh **ALTER TABLE** như sau:

```
ALTER TABLE DONDH
  DROP COLUMN Ngaydknh
```

Lưu ý:

Đối với những cột nào đã có định nghĩa các quy tắc kiểm tra tòan vẹn dữ liệu thông qua các constraint thì bạn không thể hủy bỏ. Trong các trường này để có thể hủy bỏ các cột đó, trước tiên bạn phải hủy bỏ các constraint có liên quan đến chúng.

4.2.3. Sửa đổi kiểu dữ liệu của cột:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn sửa đổi kiểu dữ của cột hiện có trong bảng. Đối với các kiểu dữ liệu dạng text, ntext hoặc image thì không thể đổi sang kiểu dữ liệu khác.

Thông thường bạn rất ít khi sửa đổi kiểu dữ liệu của các cột một cách trực tiếp mà thay vào đó bạn thêm một cột mới với kiểu dữ liệu như mong muốn vào bảng, sau đó cập nhật dữ liệu của cột hiện có sang cột mới vừa thêm vào, kiểm tra việc dữ liệu sau khi cập nhật có đúng theo như mong muốn không. Cuối cùng là hủy bỏ cột dữ liệu cũ và đổi tên cột dữ liệu mới thành tên của cột dữ liệu cũ trước đó.

Cú pháp:

```
ALTER TABLE Tên_Bảng
        ALTER COLUMN Tên_cột Kiểu_dữ_liệu_mới
```

Ví dụ:

Để sửa lại kiểu dữ liệu và tăng độ dài lưu trữ của cột đơn vị tính lên 20 ký tự trong bảng VATTU (vật tư), bạn thực hiện câu lệnh **ALTER TABLE** như sau:

```
ALTER TABLE VATTU
  ALTER COLUMN Dvtinh VARCHAR(20)
```

Lưu ý:

Giống như việc hủy bỏ các cột, với những cột đã có định nghĩa các quy tắc kiểm tra toàn vẹn dữ liệu thông qua các **constraint** thì không thể thay đổi kiểu dữ liệu của chúng.

4.2.4. Tắt bỏ quy tắc kiểm tra toàn vẹn dữ liệu:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn tạm thời bỏ qua việc kiểm tra toàn vẹn dữ liệu thông qua các đối tượng constraint bên trong bảng bằng cách tắt các quy tắc. Các quy tắc mà bạn chỉ định sẽ không còn được áp dụng dùng để kiểm tra tính toàn vẹn dữ liệu cho đến khi nào bạn bật cho nó hoạt động trở lại. Tuy nhiên các quy tắc này sẽ hoàn toàn không bị xóa đi ra khỏi cơ sở dữ liệu.

- **Cú pháp**:

```
ALTER TABLE Tên_bảng
        NOCHECK CONSTRAINT ALL | Tên_constraint [,…]
```

Trong đó:

• **Tên constraint:** tên của các constraint phải có tồn tại bên trong cơ sở dữ liệu mà bạn muốn tạm ngưng việc kiểm tra tính toàn vẹn dữ liệu trên đó.

• **Từ khóa ALL:** được sử dụng khi bạn muốn tạm ngưng việc kiểm tra tính toàn vẹn dữ liệu cho tất cả các constraint có liên quan đến bảng đã được định nghĩa trước đó.

- **Lưu ý:**

Bạn tạm thời tắt việc kiểm tra dữ liệu trong cột số lượng đặt hàng của bảng CTDONDH (chi tiết đặt hàng) nằm trong miền giá trị từ 10 đến 50 khi người dùng thêm hoặc sửa dữ liệu trong bảng CTDONDH, bạn thực hiện câu lệnh **ALTER TABLE** như sau:

```
ALTER TABLE CTDONDH
  NOCHECK CONSTRAINT CHK_CTDONDH_SLDAT
```

- **Lưu ý:**

Bạn chỉ được phép tắt các quy tắc kiểm tra toàn vẹn dữ liệu của các constraint là khóa ngoại (**FOREIGN KEY**), miền giá trị (**CHECK**) và giá trị mặc định (**DEFAULT**). Hai loại constraint còn lại là khóa chính (**PRIMARY KEY**) và duy nhất (**UNIQUE**) hệ thống Microsoft SQL Server hoàn toàn không cho phép tắt là bởi vì hệ thống cần phải kiểm tra tính duy nhất của dữ liệu bên trong bảng có định nghĩa khóa chính.

4.2.5. Bật lại quy tắc kiểm tra toàn vẹn dữ liệu:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn bật lại việc áp dụng các quy tắc kiểm tra toàn vẹn dữ liệu thông qua các đối tượng constraint bên trong bảng sau khi bạn tạm thời đã tắt đi trước đó.

- **Cú pháp**:

```
ALTER TABLE Tên_Bảng
        CHECK CONSTRAINT ALL | Tên_Constraint [,…]
```

Trong đó:

Tên constraint: tên của các constraint phải tồn tại trong cơ sở dữ liệu mà bạn muốn bật trở lại sau khi đã tắt tạm thời.

Từ khóa ALL: được sử dụng khi muốn bật việc kiểm tra toàn vẹn dữ liệu cho toàn bộ tất cả các constraint liên quan đến bảng.

Ví dụ:

Bạn bật lại việc kiểm tra toàn vẹn dữ liệu trong cột số lượng đặt hàng của bảng CTDONDH (chi tiết đặt hàng) đã bị tắt tạm thời ở ví dụ trên, bạn thực hiện lệnh **ALTER TABLE** như sau:

```
ALTER TABLE CTDONDH
        CHECK CONSTRAINT CHK_CTDONDH_SLDAT
```

4.2.6. Đổi tên cột, tên bảng dữ liệu:

Trong thực tế, việc thay đổi tên cột hoặc tên bảng dữ liệu là rất hạn chế bởi vì chúng sẽ làm ảnh hưởng rất nhiều đến các đoạn chương trình có tham chiếu đến tên cột hoặc tên bảng ở đâu đó trong ứng dụng. Bạn chỉ thực hiện việc này khi thật sự cần thiết.

Để có thể đổi tên cột hoặc tên bảng, bạn có thể vào trực tiếp tiện ích **Enterprise Manager**. Tuy nhiên, bạn cần biết thêm về **Procdure Store** hệ thống (system stored procedure) có tên là sp_rename.

Các **Procdure Store** hệ thống là một tập hợp các thủ tục do Microsoft SQL Server tạo ra và cung cấp cho người sử dụng thực hiện một số xử lý sẵn có bên trong Microsoft SQL Server.

- **Cú pháp**:

```
EXEC sp_rename 'Tên_Bảng[. Tên_Cột]', 'Tên_Mới'
        [, 'COLUMN']
```

Trong đó:

• **EXE:** lệnh dùng để thực thi (execute) các **Procdure Store** bên trong Microsoft SQL Server.

• **Tên bảng:** tên của bảng mà bạn sẽ đổi tên hoặc tên bảng chứa tên cột mà bạn muốn đổi tên.

• **Tên cột:** tên cột mà bạn muốn đổi tên. Khi muốn đổi tên một cột trong bảng thì bạn phải chỉ ra đầy đủ tên bảng chứa tên cột muốn đổi tên.

• **Tên mới:** tên mới của cột hoặc tên mới của bảng sau khi đổi.

• **COLUMN:** từ khóa chỉ được sử dụng khi thay đổi tên cột.

Ví dụ:

Để thay đổi tên cột họ tên nhà cung cấp trong bảng NHACC (nhà cung cấp) từ cột cũ Tennhacc thành cột mới là Hotenncc, bạn thực hiện câu lệnh như sau:

```
EXEC sp_rename 'NHACC.TenNhaCC', 'HoTenNCC'
```

Để thay đổi tên bảng hiện có của NHACC thành tên mới là NHACCAP, bạn thực hiện câu lệnh như sau:

```
EXEC sp_rename 'NHACC', 'NHACCAP'
```

4.3. XÓA BẢNG (TABLE)

Trong thực tế thì lệnh xóa bỏ cấu trúc bảng rất ít được thực hiện bởi vì nó rất nguy hiểm, khi đó nó sẽ làm mất toàn bộ các dòng dữ liệu bên trong đó. Vì thế bạn nên hạn chế sử dụng câu lệnh này.

❖ **Cú pháp**:

```
DROP TABLE Danh_sách_tên_các_bảng
```

Trong đó:

• **Danh sách tên bảng:** là danh sách tên các bảng mà bạn muốn hủy bỏ, tên các bảng muốn hủy bỏ được ngăn cách nhau bởi dấu phẩy (,). Thông thường muốn hủy bỏ các bảng có quan hệ trong cơ sở dữ liệu, bạn bắt buộc phải hủy bỏ các bảng bên nhánh quan hệ N trước và sau đó mới đến hủy bỏ các bảng bên nhánh 1.

Ví dụ:

Để hủy bỏ bảng VATTU (vật tư) vừa tạo ở trên, bạn thực hiện câu lệnh **DROP TABLE** như sau:

```
DROP TABLE VATTU
```

Lưu ý:

Trong các ví dụ kế tiếp, nếu bảng nào đã có tồn tại trong cơ sở dữ liệu rồi thì trước khi tạo lại cấu trúc mới của bảng đó, bạn phải ra lệnh hủy bỏ bảng hiện có rồi mới thực hiện lệnh tạo lại bảng với cấu trúc mới. Nếu không thì Microsoft SQL Server sẽ thông báo rằng bảng đã có trong cơ sở dữ liệu rồi.

4.4. QUẢN LÝ BẢNG

4.4.1. Tính toàn vẹn dữ liệu trong cơ sở dữ liệu

Toàn vẹn dữ liệu là việc đặt ra các quy tắc trong một cơ sở dữ liệu nhằm kiểm tra các giá trị của dữ liệu trước khi được lưu trữ phải đảm bảo tính chính xác và hợp lý bên trong một cơ sở dữ liệu.

Nếu các giá trị dữ liệu nào vi phạm các quy tắc đặt ra thì các dữ liệu đó sẽ không được lưu vào bảng. Các quy tắc này phần lớn được xây dựng dựa vào các quy tắc hiện hữu đang tồn tại trong công việc kinh doanh.

Ví dụ:

Trong cơ sở dữ liệu QLBanHang (quản lý bán hàng) bạn có thể đưa ra một số quy tắc như sau:

• Số lượng đặt hàng phải dương (miền giá trị).

• Ngày nhập hàng phải sau ngày đặt hàng.

• Số hóa đơn giao hàng không bị cấp trùng số (khóa chính).

• Đơn đặt hàng phải gởi cho một nhà cung cấp có trong danh sách nhà cung cấp (khóa ngoại).

• Số lượng bán hàng phải dương và còn đủ số lượng hiện có trong kho hàng.

Điều gì xảy ra nếu dữ liệu bên trong các bảng có vi phạm các quy tắc ở trên mà vẫn được lưu trữ vào bên trong cơ sở dữ liệu? Do đó, trong các hệ cơ sở dữ liệu quan hệ lớn ngày nay, hệ thống sẽ giúp cho người sử dụng có thể định nghĩa ra các quy tắc thông qua các từ khóa hoặc thuộc tính có liên quan đến cơ sở dữ liệu nhằm đảm bảo dữ liệu khi lưu trữ vào cơ sở dữ liệu phải chính xác và hợp lý.

Bên trong cơ sở dữ liệu của Microsoft SQL Server việc kiểm tra tính toàn vẹn dữ liệu sẽ thông qua hai đối tượng quản lý, đó là các **constraint** và **trigger**. Cả hai đối tượng này đều được liên kết trực tiếp vào bảng dữ liệu.

Các loại quy tắc kiểm tra tính toàn vẹn dữ liệu bao gồm:

4.4.1.1. Kiểm tra duy nhất dữ liệu:

Loại ràng buộc toàn vẹn này cho phép bạn có thể kiểm tra tính duy nhất của dữ liệu bên trong bảng. Điều này ngăn cản việc người sử dụng tình cờ nhập trùng lại các giá trị dữ liệu bên trong bảng. Bạn có thể sử dụng hai thành phần **PRIMARY KEY** hoặc **UNIQUE** trong câu lệnh **CREATE TABLE** để thực hiện việc kiểm tra tính duy nhất của dữ liệu.

Điểm khác biệt chính giữa **PRIMARY KEY** và **UNIQUE** là sự xuất hiện của các thành phần trong câu lệnh **CREATE TABLE**. Thành phần **PRIMARY KEY** cho phép bạn tạo ra cấu trúc bảng có chứa khóa chính. Do một bảng chỉ có một khóa chính, tuy nhiên khóa chính được phép định nghĩa có nhiều cột tham gia. Vì thế thành phần **PRIMARY KEY** chỉ xuất hiện một lần duy nhất khi tạo cấu trúc bảng.

Thành phần **UNIQUE** cho phép bạn kiểm tra tính duy nhất của các cột không tham gia làm khóa chính của bảng. Thành phần **UNIQUE** được phép xuất hiện nhiều lần khi tạo cấu trúc bảng nếu cần kiểm tra tính duy nhất của các cột không làm khóa chính.

❖ **Cú pháp**:

```
[ CONSTRAINT Tên_constraint ]
      PRIMARY KEY (Danh_sách_cột_khóa_chính)
```

Trong đó:

• **Tên constraint:** phải là duy nhất trong cơ sở dữ liệu. Thông thường quy định tên constraint gồm có 3 phần. Bắt đầu bằng các chữ PRK, kế tiếp là tên bảng và cuối cùng là tên cột áp dụng quy tắc kiểm tra duy nhất dữ liệu của các cột khóa chính.

• **Danh sách cột khóa chính:** là danh sách tên các cột tham gia làm khóa chính, tên các cột được ngăn cách nhau bởi dấu phẩy (,).

❖ **Cú pháp**:

```
[ CONSTRAINT Tên_constraint ]
        UNIQUE (Danh_sách_các_cột)
```

Trong đó:

• **Tên constraint:** phải là duy nhất trong cơ sở dữ liệu. Thông thường quy định tên constraint gồm có 3 phần. Bắt đầu bằng các chữ UNQ, kế tiếp là tên bảng và cuối cùng là tên cột áp dụng quy tắc kiểm tra duy nhất dữ liệu của các cột không tham gia làm khóa chính.

• **Danh sách cột:** là danh sách tên các cột cần kiểm tra duy nhất, tên các cột được ngăn cách nhau bởi dấu phẩy (,).

Lưu ý:

Mặc định tên các constraint sẽ do hệ thống Microsoft SQL Server tạo ra nhằm đảm bảo tính duy nhất bên trong cơ sở dữ liệu. Tuy nhiên bạn có thể chủ động đặt tên cho các constraint khi tạo ra chúng bằng cách đưa thêm từ khóa **CONSTRAINT** trong các thành phần định nghĩa các kiểm tra ràng buộc toàn vẹn dữ liệu.

***Ví dụ**:*

Tạo bảng VATTU kiểm tra dữ liệu của cột mã vật tư phải là duy nhất. Trường hợp chỉ định cột mã vật tư làm khóa chính của bảng. Bạn thực hiện câu lệnh **CREATE TABLE** như sau:

```
CREATE TABLE VATTU
(
        Mavtu     CHAR(4)
                  PRIMARY KEY (Mavtu),
        Tenvtu    CHAR(30),
        DvTinh    CHAR(10) NOT NULL,
        Phantram  TinyInt DEFAULT 20
)
```

Với cú pháp như trên bảng VATTU có sử dụng một constraint loại **PRIMARY KEY** dùng định nghĩa khóa chính của bảng là cột Mavtu. Tuy nhiên tên của constraint sẽ do hệ thống Microsoft SQL Server tạo ra.

Trong trường hợp nếu bạn muốn chỉ định tên của constraint do bạn định nghĩa thì bạn thực hiện câu lệnh **CREATE TABLE** có sử dụng từ khóa **CONSTRAINT** như sau:

```
CREATE TABLE VATTU
(
      Mavtu     CHAR(4)         NOT NULL,
            CONSTRAINT PRK_VATTU_Mavtu
               PRIMARY KEY (Mavtu),
      Tenvtu    CHAR(30) ,
      DvTinh    CHAR(10) NOT NULL,
      Phantram  TinyInt  DEFAULT 20
)
```

Khi mà các thành phần của đối tượng constraint xuất hiện ngay phía sau tên cột như hai ví dụ ở trên thì ràng buộc toàn vẹn dữ liệu sẽ được kiểm tra trên cột dữ liệu. Bạn nên chọn cách này khi thực hiện việc kiểm tra các ràng buộc toàn vẹn dữ liệu chỉ trên một cột bên trong bảng dữ liệu.

Trong trường hợp nếu bạn đặt các thành phần của đối tượng constraint bên dưới tất cả các cột dữ liệu bên trong bảng thì ràng buộc toàn vẹn dữ liệu sẽ được kiểm tra trên bảng. Bạn nên chọn cách này khi thực hiện việc kiểm tra các ràng buộc toàn vẹn dữ liệu trên nhiều cột có liên quan bên trong bảng dữ liệu.

***Ví dụ**:*

Để tạo bảng có tên CTDONDH (chi tiết đơn đặt hàng) gồm có những cột như: số đặt hàng có kiểu dữ liệu là chuỗi và chiều dài 4 ký tự, mã vật tư có kiểu dữ liệu là chuỗi và chiều dài 4 ký tự, số lượng đặt có kiểu số nguyên. Dữ liệu tại các cột không được phép trống. Khóa chính gồm có 2 cột là số đặt hàng và mã vật tư. Bạn thực hiện câu lệnh **CREATE TABLE** như sau:

```
CREATE TABLE CTDONDH
(
      Sodh      CHAR(4),
      Mavtu     CHAR(4),
      SlDat     SMALLINT
      PRIMARY KEY (Sodh, Mavtu)
)
```

Hoặc muốn đặt tên của constraint là PRK_CTDONDH_SodhID

```
CREATE TABLE CTDONDH
(
      Sodh      CHAR(4),
      Mavtu     CHAR(4),
      SlDat     SMALLINT
      CONSTRAINT PRK_CTDONDH_SodhID
      PRIMARY KEY (Sodh, Mavtu)
)
```

Lưu ý:

Khi sử dụng các loại ràng buộc toàn vẹn dữ liệu được kiểm tra trên cột dữ liệu thì cần nhớ rằng dấu phẩy (,) luôn được đặt ở vị trí sau cùng của thành phần **constraint** chứ không được đặt ở vị trí phía sau tên kiểu dữ liệu của cột.

Ngược lại khi sử dụng các loại ràng buộc toàn vẹn dữ liệu được kiểm tra trên bảng thì không cần có thêm dấu phẩy (,) ở vị trí phía sau của cột dữ liệu cuối cùng bên trong bảng.

Đôi khi những điều lưu ý này sẽ làm cho bạn cảm thấy khó nhớ. Do vậy để đơn giản khi tạo cấu trúc bảng, bạn thực hiện hai bước:

Bước 1: Tạo cấu trúc bảng đơn giản bằng lệnh **CREATE TABLE** gồm có tên bảng, tên các cột và các kiểu dữ liệu mong muốn.

Bước 2: Thêm các loại constraint tương ứng bằng lệnh **ALTER TABLE ADD CONSTRAINT** để áp dụng các kiểm tra ràng buộc toàn vẹn dữ liệu cho bảng dữ liệu.

4.4.1.2. Kiểm tra tồn tại dữ liệu:

Loại ràng buộc toàn vẹn này cho phép bạn có thể kiểm tra tính tồn tại của dữ liệu (khóa ngoại), bắt buộc phải có bên một bảng khác, còn gọi là bảng tham chiếu. Điều này ngăn cản việc người sử dụng nhập một giá trị dữ liệu không có trong một bảng dữ liệu khác. Bạn có thể sử dụng thành phần **FOREIGN KEY** trong câu lệnh **CREATE TABLE** để thực hiện việc kiểm tra tính tồn tại của dữ liệu.

Trong một bảng được phép có nhiều khóa ngoại, tuy nhiên các cột tham chiếu phải được định nghĩa là khóa chính trong các bảng tham chiếu trước đó. Vì thế thành phần **FOREIGN KEY** được phép xuất hiện nhiều lần khi tạo cấu trúc bảng nếu bạn cần kiểm tra tính tồn tại của nhiều cột khác nhau trong các bảng khác nhau.

Cú pháp:

```
[ CONSTRAINT Tên_constraint ]
      FOREIGN KEY (Danh_sách_cột_khóa_ngoại)
      REFERENCES Tên_bảng_tham_chiếu
        (Danh_sách_cột_tham_chiếu)
```

Trong đó:

• **Tên constraint:** phải là duy nhất trong cơ sở dữ liệu. Thông thường quy định tên constraint gồm có 3 phần. Bắt đầu bằng các chữ: FRK, kế tiếp là tên bảng và cuối cùng là tên cột áp dụng quy tắc kiểm tra tồn tại dữ liệu.

• **Danh sách cột khóa ngoại:** là danh sách tên các cột tham gia làm khóa ngoại, tên các cột được ngăn cách nhau bởi dấu phẩy (,).

• **Tên bảng tham chiếu:** là tên của bảng tham chiếu để kiểm tra tính tồn tại dữ liệu bên bảng tham chiếu.

• **Danh sách cột tham chiếu:** là danh sách tên các cột tham chiếu trong bảng tham chiếu, tên các cột ngăn cách nhau bởi dấu phẩy (,). Lưu ý rằng danh sách các cột khóa ngoại và danh sách các cột tham chiếu phải tương xứng nhau.

***Ví dụ**:*

Tạo lại bảng CTDONDH có bổ sung thêm hai ràng buộc toàn vẹn dữ liệu khóa ngoại: số đặt hàng tham chiếu qua bảng DONDH, nghĩa là giá trị dữ liệu tại cột số đặt hàng phải tồn tại trong bảng DONDH và mã vật tư tham chiếu qua bảng VATTU, nghĩa là giá trị dữ liệu tại cột mã vật tư phải tồn tại trong bảng VATTU. Bạn thực hiện câu lệnh **CREATE TABLE** như sau:

```
create Table CTDONDH
(
    SODH      INT,
    MaVTu     Char(4),
    SLDat          SmallInt
    PRIMARY KEY (SODH, MAVTU),
    FOREIGN KEY (SODH) REFERENCES DONDH (SODH),
    FOREIGN KEY (MAVTU) REFERENCES VATTU (MAVTU))
```

Hoặc muốn đặt tên cho các constraint liên quan

```
CREATE TABLE CTDONDH111
(
    SODH      INT,
    MaVTu     Char(4),
    SLDat          SmallInt
    CONSTRAINT PRK_CTDONDH_SODH PRIMARY KEY (SODH,
    MAVTU),
    CONSTRAINT FRK_CTDONDH_SODH FOREIGN KEY (SODH)
    REFERENCES DONDH (SODH),
    CONSTRAINT   FRK_CTDONDH_MAVTU   FOREIGN   KEY
    (MAVTU) REFERENCES VATTU (MAVTU))
```

4.4.1.3. Kiểm tra miền giá trị:

Loại ràng buộc toàn vẹn này cho phép bạn có thể kiểm tra miền giá trị của các cột dữ liệu bên trong bảng. Điều này ngăn cản việc người sử dụng nhập một giá trị dữ liệu vượt ra khỏi phạm vi mà bạn quy định trước đó. Bạn có thể sử dụng thành phần **CHECK** trong câu lệnh **CREATE TABLE** để thực hiện việc kiểm tra miền giá trị của dữ liệu.

Có thể thực hiện việc kiểm tra để so sánh giá trị của một cột bên trong bảng với một giá trị cụ thể hoặc một hàm tính toán của Microsoft SQL Server mà kiểu trả về của hàm phải khớp với kiểu dữ liệu của cột. Ngoài ra bạn cũng có thể kiểm tra giá trị của nhiều cột có liên quan trong cùng một bảng dữ liệu.

Cú pháp:

```
[ CONSTRAINT Tên_constraint ]
    CHECK (Biểu_thức_luận_lý)
```

Trong đó:

• **Tên constraint:** phải là duy nhất trong cơ sở dữ liệu. Thông thường quy định tên constraint gồm có 3 phần. Bắt đầu bằng các chữ: CHK, kế tiếp là tên bảng và cuối cùng là tên cột áp dụng quy tắc kiểm tra miền giá trị dữ liệu.

• **Biểu thức luận lý:** là một biểu thức chỉ định quy tắc kiểm tra dữ liệu trong bảng. Thông thường biểu thức luận lý sẽ là biểu thức so sánh.

***Ví dụ**:*

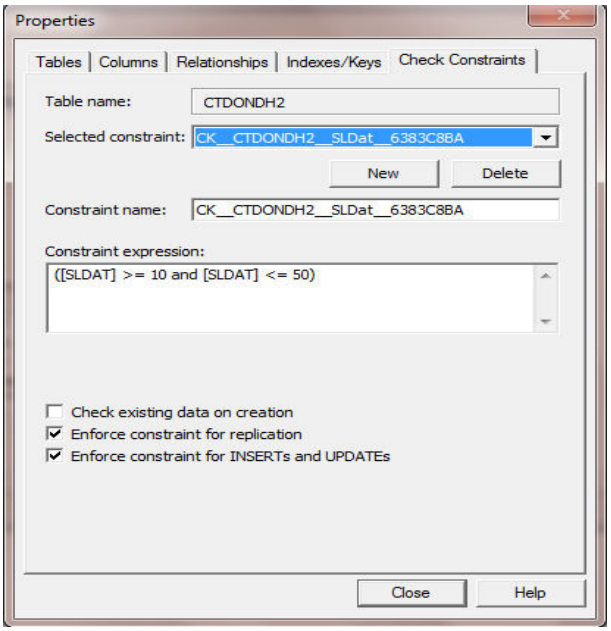
Trong bảng CTDONDH (chi tiết đơn đặt hàng) kiểm tra quy tắc giá trị dữ liệu của cột số lượng đặt tối thiểu là 10 và không lớn hơn 50. Có nghĩa là cột số lượng đặt >=10 và số lượng đặt <=50, giá trị mặc định sẽ là 10 khi người sử dụng không cung cấp giá trị cột số lượng đặt.

```
create Table CTDONDH2
(
  SODH      INT,
  MaVTu     Char(4),
  SLDat     SmallInt Default 20
  CHECK(SLDAT BETWEEN 10 AND 50)
  PRIMARY KEY (SODH, MAVTU),
  FOREIGN KEY (SODH) REFERENCES DONDH (SODH),
  FOREIGN KEY (MAVTU) REFERENCES VATTU (MAVTU))
```

Hoặc muốn đặt tên cho các constraint liên quan

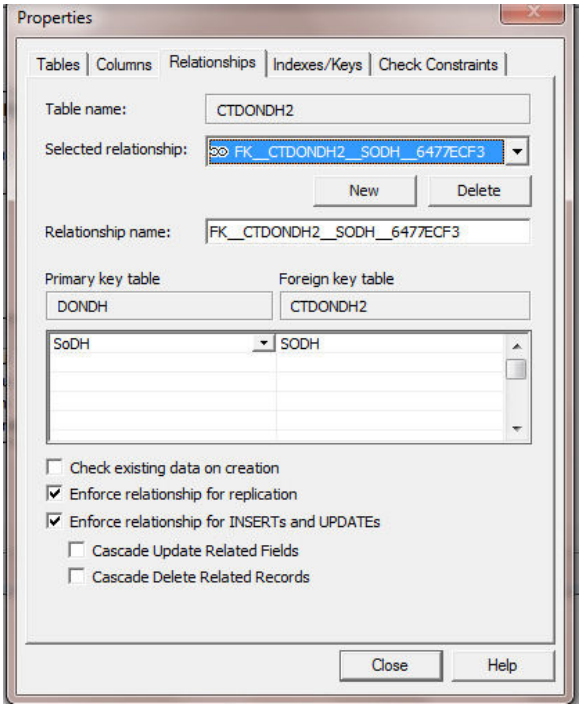
```
create Table CTDONDH
(
  SODH      INT,
  MaVTu     Char(4),
  SLDat     SmallInt Default 20
  CONSTRAINT CHK_CTDONDH_SLDAT
       CHECK(SLDAT BETWEEN 10 AND 50),
  CONSTRAINT PRK_CTDONDH_SODH
       PRIMARY KEY (SODH, MAVTU),
  CONSTRAINT FRK_CTDONDH_SODH
       FOREIGN KEY (SODH) REFERENCES DONDH (SODH),
  CONSTRAINT FRK_CTDONDH_MAVTU
       FOREIGN KEY (MAVTU) REFERENCES VATTU (MAVTU))
```

Khi muốn xem tên của các constraint và các quy tắc liên quan được định nghĩa trong cấu trúc bảng, bạn chọn chức năng **Properties** sau khi nhấn vào tên bảng trong màn hình mô hình quan hệ dữ liệu (**diagram**).



Hình 4.4. Hộp thoại thuộc tính mô hình quan hệ

Tương tự, bạn có thể di chuyển qua các tab Relationships hoặc Indexs/Keys để thấy được các quy tắc kiểm tra dữ liệu về tính duy nhất, tính tồn tại của các dữ liệu bên trong bảng.



Hình 4.5. Hộp thoại thuộc tính mô hình quan hệ

Thay vì tạo cấu trúc bảng bằng duy nhất một câu lệnh CREATE TABLE với nhiều thành phần của các đối tượng constraint khó nhớ bên trong đó, bạn có thể tạo ra cấu trúc bảng đơn giản và sau đó thêm vào các constraint mới trong bảng dùng để thực hiện quy tắc kiểm tra cho các loại ràng buộc toàn vẹn dữ liệu trên bảng.

4.4.1.4. Thêm vào constraint mới trong bảng:

Với cú pháp **ALTER TABLE** tổng quát bên dưới cho phép bạn thêm vào các loại quy tắc kiểm tra toàn vẹn dữ liệu thông qua các đối tượng constraint bên trong bảng.

❖ **Cú pháp**:

```
ALTER TABLE Tên_bảng
        ADD [ CONSTRAINT Tên_constraint ]
                LOẠI Các_tham_số
        [,...]
```

Trong đó:

• **Tên constraint:** phải là duy nhất bên trong cơ sở dữ liệu. Thông thường quy định tên của một constraint bao gồm 3 phần nhỏ. Bắt đầu bằng các chữ: PRK, FRK, UNQ, CHK, DEF chỉ các loại constraint tương ứng sẽ là khóa chính, khóa ngoại, duy nhất, miền giá trị, giá trị mặc định, kế sau đó là tên của bảng và cuối cùng là tên cột sẽ áp dụng quy tắc kiểm tra dữ liệu.

• **Loại:** là các từ khóa tương ứng cho các thành phần theo từng loại constraint như kiểm tra tính duy nhất (**PRIMARY KEY, UNIQUE**), kiểm tra khóa ngoại (**FOREIGN KEY**), kiểm tra miền giá trị (**CHECK**), giá trị mặc định (**DEFAULT**).

• **Các tham số:** là các tham số cần thiết đi kèm theo với các từ khóa của từng loại constraint tương ứng.

❖ **Cú pháp** chi tiết từng loại constraint:

```
ALTER TABLE Tên_bảng
        ADD [ CONSTRAINT Tên_constraint ]
                PRIMARY KEY (Danh_sách_cột_khóa_chính)
          Hoặc
                UNIQUE (Danh_sách_các_cột)
          Hoặc
                FOREIGN KEY (Danh_sách_cột_khóa_ngoại
                REFERENCES Tên_bảng_tham_chiếu _
                                (Danh_sách_cột_tham_chiếu)
          Hoặc
                CHECK (Biểu_thức_luận_lý)
          Hoặc
                DEFAULT Giá_trị_mặc_định FOR Tên_cột
```

***Ví dụ**:*

Tạo cấu trúc bảng NHACC (nhà cung cấp) với khóa chính là cột mã nhà cung cấp. Trước tiên bạn tạo cấu trúc bảng đơn giản.

```
create tabLe NHACC
(
     MANHACC       CHAR(3) NOT NULL,
     TENNHACC CHAR(30) NOT NULL,
     DIACHI        CHAR(50) NOT NULL,
     DIENTHOAICHAR(15)
)
```

Sau đó bạn thêm vào các quy tắc kiểm tra duy nhất của dữ liệu tại khóa chính của bảng là cột mã nhà cung cấp bằng lệnh **ALTER TABLE ADD CONSTRAINT**.

```
ALTER TABLE NHACC
     ADD CONSTRAINT PRK_NHACC_MANHACC
          PRIMARY KEY(MANHACC)
```

Bạn có thể thêm một quy tắc mới dùng để kiểm tra tính duy nhất dữ liệu tại cột địa chỉ của bảng nhà cung cấp và đưa vào giá trị mặc định cho cột điện thoại của bảng nhà cung cấp là "Chưa có" bằng các câu lệnh như sau:

```
ALTER TABLE NHACC
     ADD CONSTRAINT UNQ_NHACC_DIACHI
          UNIQUE (DIACHI)
     CONSTRAINT DEF_NHACC_DIENTHOAI
          DEFAULT 'Chưa có' FOR DIENTHOAI
```

Lưu ý:

Trong lệnh **ALTER TABLE ADD CONSTRAINT** khi cần định nghĩa giá trị mặc định cho một cột dữ liệu bên trong bảng thì bắt buộc bạn phải có thêm từ khóa **FOR** tên cột để chỉ định giá trị mặc định được tạo ra sẽ áp dụng cho cột dữ liệu nào.

Sử dụng các câu lệnh **CREATE TABLE** để tạo ra cấu trúc các bảng VATTU (vật tư), NHACC (nhà cung cấp), DONDH (đơn đặt hàng) và CTDONDH (chi tiết đơn đặt hàng).

Bảng VATTU có các quy tắc kiểm tra toàn vẹn dữ liệu:

- Khóa chính là cột mã vật tư.
- Giá trị dữ liệu duy nhất tại cột tên vật tư.
- Giá trị mặc định cho cột phần trăm là 20.
- Kiểm tra phạm vi giá trị của phần trăm từ 5 đến 100.

```
CREATE TABLE VATTU
(
```

```
    MAVTU     CHAR(4)   NOT NULL,
    TENVTU    CHAR(3) NOT NULL,
    DVTINH    CHAR(10) NOT NULL,
    PHANTRAM TINYINT
    CONSTRAINT DEF_VATTU_PHANTRAM DEFAULT 20,
    CONSTRAINT PRK_VATTU_MAVTU
        PRIMARY KEY(MAVTU),
    CONSTRAINT UNQ_VATTU_TENVTU
        UNIQUE(TENVTU),
    CONSTRAINT UNQ_VATTU_PHANTRAM
        CHECK (PHANTRAM BETWEEN 5 AND 100)
)
```

Hoặc đầu tiên tạo bảng có cấu trúc đơn giản và sau đó thêm vào các ràng buộc toàn vẹn dữ liệu.

```
CREATE TABLE VATTU
(
    MAVTU          CHAR(4)   NOT NULL,
    TENVTU    CHAR(3)   NOT NULL,
    DVTINH         CHAR(10)  NOT NULL,
    PHANTRAM TINYINT
)
GO
ALTER TABLE VATTU6
    ADD CONSTRAINT     PRK_VATTU_MAVTU6
        PRIMARY KEY(MAVTU),
    CONSTRAINT     DEF_VATTU_PHANTRAM DEFAULT 20 FOR
PHANTRAM,
    CONSTRAINT     UNQ_VATTU_TENVTU
        UNIQUE(TENVTU),
    CONSTRAINT     CHK_VATTU_PHANTRAM
        CHECK (PHANTRAM BETWEEN 5 AND 100)
```

Bảng NHACC có các quy tắc kiểm tra toàn vẹn dữ liệu:

- Khóa chính là cột mã nhà cung cấp.
- Giá trị dữ liệu duy nhất tại cột địa chỉ.
- Giá trị mặc định cho cột điện thoại là "Chưa có".

```
CREATE TABLE NHACC
(
     MANHACC   CHAR(3)   NOT NULL,
     TENNHACC  CHAR(30)  NOT NULL,
     DIACHI    VARCHAR(50)    NOT NULL,
     DIENTHOAICHAR(15)  NOT NULL
     CONSTRAINT     DEF_NHACC_DIENTHOAI
          DEFAULT 'Chưa có',
     CONSTRAINT PRK_NHACC_MANHACC
          PRIMARY KEY (MANHACC),
     CONSTRAINT UNQ_NHACC_DIACHI
          UNIQUE(DIACHI)
)
```

Hoặc đầu tiên tạo bảng có cấu trúc đơn giản và sau đó thêm vào các ràng buộc toàn vẹn dữ liệu.

```
CREATE TABLE NHACC
(
     MANHACC   CHAR(3)   NOT NULL,
     TENNHACC  CHAR(30)  NOT NULL,
     DIACHI    VARCHAR(50)    NOT NULL,
     DIENTHOAICHAR(15)  NOT NULL
)
GO
ALTER TABLE NHACC
     ADD CONSTRAINT PRK_NHACC_MANHACC
          PRIMARY KEY (MANHACC),
     CONSTRAINT     DEF_NHACC_DIENTHOAI
          DEFAULT 'Chưa có' FOR DIENTHOAI,
     CONSTRAINT UNQ_NHACC_DIACHI
          UNIQUE(DIACHI)
```

Bảng DONDH có các quy tắc kiểm tra toàn vẹn dữ liệu:

• Khóa chính là cột số đơn đặt hàng.

• Giá trị mặc định cho các cột ngày đặt hàng và ngày dự kiến nhận hàng là ngày hiện hành.

• Ngày dự kiến nhận hàng phải sau ngày đặt hàng.

• Kiểm tra tính tồn tại dữ liệu của cột mã nhà cung cấp bên bảng NHACC.

```
CREATE TABLE DONDH
(
 SODH CHAR(4),
 NGAYDH    DATETIME   NOT NULL
 CONSTRAINT DEF_DONDH_NGAYDH
      DEFAULT GETDATE(),
 NGAYDKNH DATETIME NOT NULL
 CONSTRAINT    DEF_DONDH_NGAYDKNH
      DEFAULT GETDATE(),
 MANHACC   CHAR(3)   NOT NULL
 CONSTRAINT    PRK_DONDH_SODH
      PRIMARY KEY(SODH),
 CONSTRAINT    CHK_DONDH_NGAYDKNH
      CHECK (NGAYDH<=NGAYDKNH),
 CONSTRAINT    FRK_DONDH_MANHACC
      FOREIGN KEY (MANHACC) REFERENCES NHACC(MANHACC)
)
```

Hoặc đầu tiên tạo bảng có cấu trúc đơn giản và sau đó thêm vào các ràng buộc toàn vẹn dữ liệu.

```
CREATE TABLE DONDH
(
 SODH CHAR(4),
 NGAYDH    DATETIME   NOT NULL,
 NGAYDKNH DATETIME NOT NULL,
 MANHACC   CHAR(3)   NOT NULL
)
GO
ALTER TABLE DONDH
 ADD
 CONSTRAINT    PRK_DONDH_SODH
      PRIMARY KEY(SODH),
 CONSTRAINT DEF_DONDH_NGAYDH
      DEFAULT GETDATE() FOR NGAYDH,
 CONSTRAINT    DEF_DONDH_NGAYDKNH
      DEFAULT GETDATE()FOR NGAYDKNH,
 CONSTRAINT    CHK_DONDH_NGAYDKNH
      CHECK (NGAYDH<=NGAYDKNH),
```

```
    CONSTRAINT    FRK_DONDH_MANHACC
        FOREIGN KEY (MANHACC) REFERENCES NHACC(MANHACC)
```

Bảng CTDONDH có các quy tắc kiểm tra toàn vẹn dữ liệu:

• Khóa chính là cột số đặt hàng và mã vật tư.

• Giá trị mặc định cho cột số lượng đặt là 10.

• Kiểm tra phạm vi giá trị của số lượng đặt từ 10 đến 50.

• Kiểm tra tính tồn tại dữ liệu của cột số đặt hàng trong bảng DONDH và cột mã vật tư trong bảng VATTU.

```
CREATE TABLE CTDONDH
(
    SODH CHAR(4),
    MAVTU    CHAR(4),
    SLDAT    SMALLINT
    CONSTRAINT    DEF_CTDONDH_SLDAT
        DEFAULT 10,
    CONSTRAINT PRK_CTDONDH_SODH
        PRIMARY KEY(SODH, MAVTU),
    CONSTRAINT FRK_CTDONDH_SODH
        FOREIGN KEY (SODH) REFERENCES DONDH(SODH),
    CONSTRAINT FRK_CTDONDH_MAVTU
        FOREIGN KEY (MAVTU) REFERENCES VATTU(MAVTU),
    CONSTRAINT CHK_CTDONDH_SLDAT
        CHECK(SLDAT BETWEEN 10 AND 50)
)
```

Hoặc đầu tiên tạo bảng có cấu trúc đơn giản và sau đó thêm vào các ràng buộc toàn vẹn dữ liệu.

```
CREATE TABLE CTDONDH
(
 SODH CHAR(4),
 MAVTU    CHAR(4),
 SLDAT    SMALLINT
)
GO
ALTER TABLE CTDONDH
 ADD
```

```
CONSTRAINT    DEF_CTDONDH_SLDAT
     DEFAULT 10 FOR SLDAT,
CONSTRAINT PRK_CTDONDH_SODH
     PRIMARY KEY(SODH, MAVTU),
CONSTRAINT FRK_CTDONDH_SODH
     FOREIGN KEY (SODH) REFERENCES DONDH(SODH),
CONSTRAINT FRK_CTDONDH_MAVTU
     FOREIGN KEY (MAVTU) REFERENCES VATTY(MAVTU),
CONSTRAINT CHK_CTDONDH_SLDAT
     CHECK(SLDAT BETWEEN 10 AND 50)
```

- **Lưu ý:**

Khi thực *hiện tạo bảng có cấu trúc đơn giản và sau đó thêm vào các ràng buộc toàn vẹn dữ liệu thì bạn phải luôn luôn thêm từ khóa **NOT NULL** phía sau các cột tham gia làm khóa chính của bảng khi tạo cấu trúc bằng lệnh **CREATE TABLE**.*

4.4.1.5. Hủy bỏ constraint đã có trong bảng:

Với cú pháp **ALTER TABLE** bên dưới cho phép bạn hủy bỏ các quy tắc kiểm tra tòan vẹn dữ liệu thông qua các đối tượng constraint có bên trong bảng đã tạo trước đó. Các quy tắc mà bạn hủy bỏ sẽ không còn thực hiện việc kiểm tra tính toàn vẹn dữ liệu bên trong của bảng cho đến khi bạn phải tạo mới lại nó.

Muốn hủy bỏ được các đối tượng constraint bắt buộc bạn phải ghi nhớ được tên của các constraint lúc tạo ra nó. Hoặc xem lại trong màn hình thuộc tính của bảng. Do vậy khi tạo ra các quy tắc để kiểm tra các ràng buộc toàn vẹn dữ liệu bạn nên chủ động đặt tên cho các đối tượng constraint bên trong cơ sở dữ liệu Microsoft SQL Server.

- **Cú pháp**:

```
ALTER TABLE Tên_bảng
     DROP CONSTRAINT Tên_constraint [, ...]
```

Trong đó:

• Tên constraint: phải tồn tại trong cơ sở dữ liệu mà bạn muốn hủy bỏ. Do đó đối với tên của các constraint mà bạn chủ động đặt khi sử dụng từ khóa **CONSTRAINT** thì sẽ giúp bạn dễ dàng gợi nhớ hơn tên của các constraint do hệ thống tự đặt.

- **Ví dụ:**

Hủy bỏ quy tắc kiểm tra số lượng đặt hàng trong bảng CTDONDH có giới hạn từ 10 đến 50.

```
ALTER TABLE CTDONDH
     DROP CONSTRAINT CHK_CTDONDH_SLDAT
```

4.5. TẠO MỘT DATABASE DIAGRAM

Đối với người sử dụng đã quen thuộc với loại cơ sở dữ liệu Microsoft Access, màn hình Relationship trong Microsoft Access sẽ giúp cho bạn hiển thị và tạo lập các mối quan hệ dữ liệu giữa các bảng trong tập tin cơ sở dữ liệu MDB.

4.5.1. Khái niệm về mô hình quan hệ dữ liệu:

Thực ra việc tạo mối quan hệ giữa các bảng là nhằm trao đổi và chia sẻ thông tin qua lại giữa các bảng với nhau, đồng thời cũng giúp kiểm tra tính tồn tại dữ liệu (khóa ngoại).

Riêng trong môi trường cơ sở dữ liệu của Microsoft SQL Server khi đến giai đoạn này thì bạn chỉ biết được việc tạo cấu trúc các bảng có thể tạo thêm các mối quan hệ dữ liệu ngầm định giữa các bảng thông qua quy tắc kiểm tra ràng buộc về khóa ngoại (foreign key constraint). Tuy nhiên các mối quan hệ này không thể hiện một cách trực quan để giúp cho người sử dụng dễ dàng quan sát.

Thấy được sự thiếu sót này trong các phiên bản cũ trước đây. Vì thế kể từ phiên bản 7.0 trở lên, Microsoft đã đưa một đối tượng mới trong cơ sở dữ liệu của Microsoft SQL Server, đó chính là đối tượng mô hình quan hệ dữ liệu (diagram). Trong mô hình quan hệ dữ liệu này, bạn có thể dễ dàng nhìn thấy mối quan hệ dữ liệu giữa các bảng có liên quan với nhau, thực hiện một số các hành động tác động trực tiếp ngay trong các bảng như là tạo cấu trúc bảng mới, xóa cấu trúc bảng đã có, chỉ định khóa chính cho bảng, thêm các bảng hiện có vào bên trong mô hình quan hệ dữ liệu.

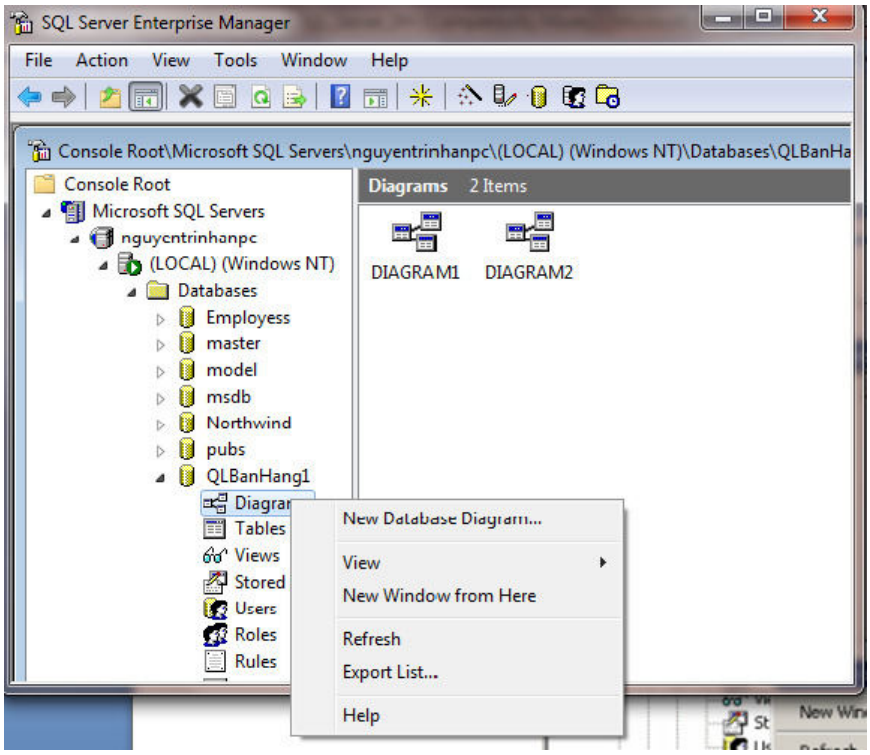
4.5.2. Tạo mới mô hình quan hệ dữ liệu:

Đối tượng mô hình quan hệ dữ liệu chỉ được tạo ra và hiển thị trong tiện ích **Enterprise Manager** nhằm hệ thống Microsoft SQL Server kiểm tra các ràng buộc dữ liệu khóa ngoại trong cơ sở dữ liệu và giúp cho người dùng dễ dàng thấy được các mối liên kết dữ liệu giữa các bảng một cách trực quan hơn.

Thông thường các mối kết hợp bên trong mô hình quan hệ dữ liệu sẽ được tự động tạo ra khi bạn chèn các bảng vào mô hình này. Để tạo mới mô hình quan hệ dữ liệu trong một cơ sở dữ liệu bạn lần lượt thực hiện các bước sau:

• Bước 1:

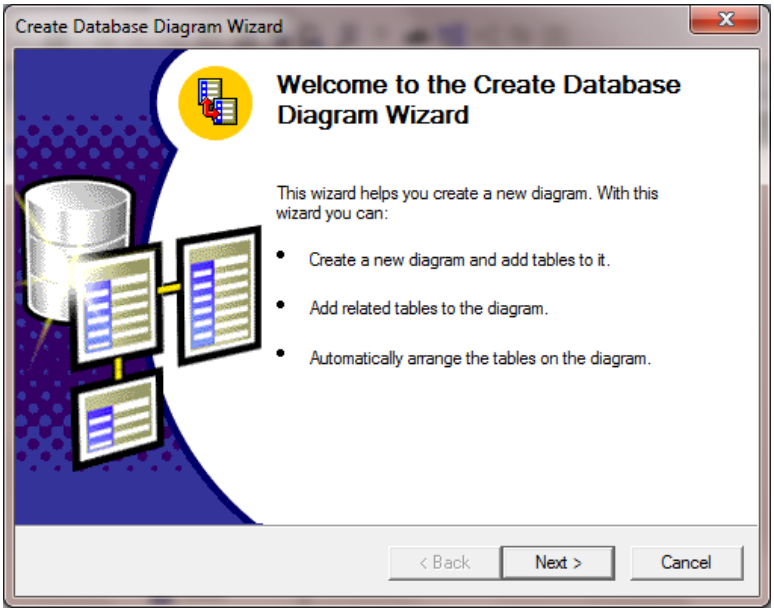
Khởi động tiện ích **Enterprise Manager**, chọn cơ sở dữ liệu có chứa các bảng cần tạo mới mô hình quan hệ dữ liệu. Chọn chức năng **New Database Diagram** trong thực đơn tắt sau khi click chuột phải trên đối tượng **Diagrams**.



Hình 4.6. Tạo mới mô hình quan hệ dữ liệu

• Bước 2:

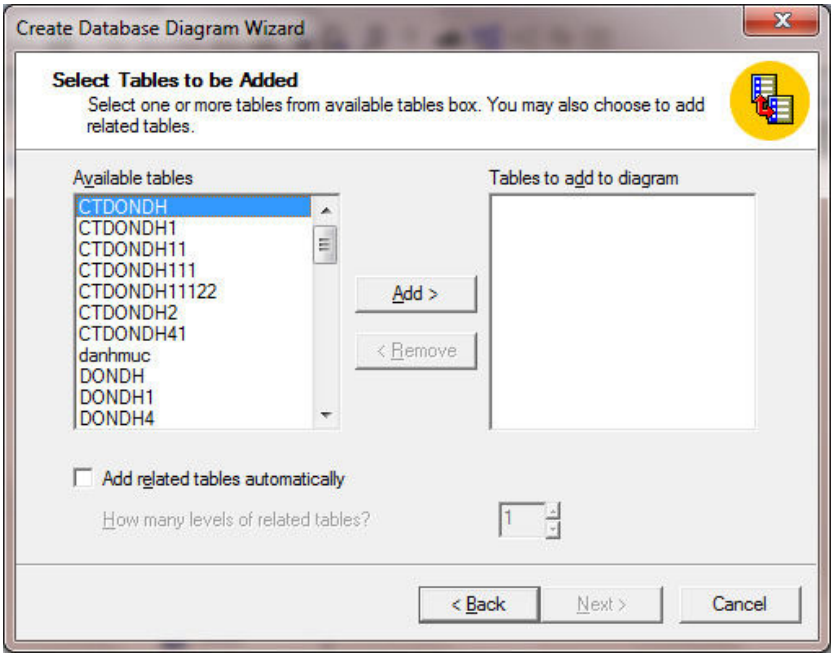
Trong màn hình **Welcome** việc tạo mới mô hình quan hệ dữ liệu bằng trình trợ giúp thông minh, hệ thống hiển thị cho bạn biết các bước sẽ thực hiện trong quá trình tạo mới mô hình quan hệ dữ liệu. Nhấn **Next** để tiếp tục.



Hình 4.7. Màn hình tạo mới mô hình quan hệ dữ liệu

• Bước 3:

Trong màn hình chọn các bảng (**Select Tables**), lần lượt chọn các bảng trong danh sách bảng đang có trong cơ sở dữ liệu (**Available tables**)) mà bạn muốn đưa chúng vào mô hình quan hệ dữ liệu, sau đó nhấn **Add**. Nhấn **Next** để tiếp tục.

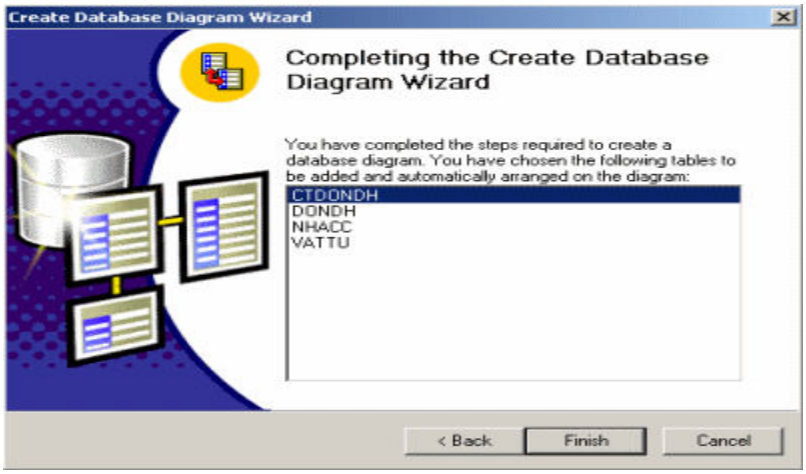


Hình 4.8. Chọn các bảng đưa vào mô hình quan hệ dữ liệu

Ngược lại khi muốn bỏ đi các bảng có trong mô hình quan hệ dữ liệu thì bạn sẽ nhấn vào nút **Remove** sau khi đã chọn các bảng cần xóa trong danh sách bên phải (**Tables to add to diagram**).

• Bước 4:

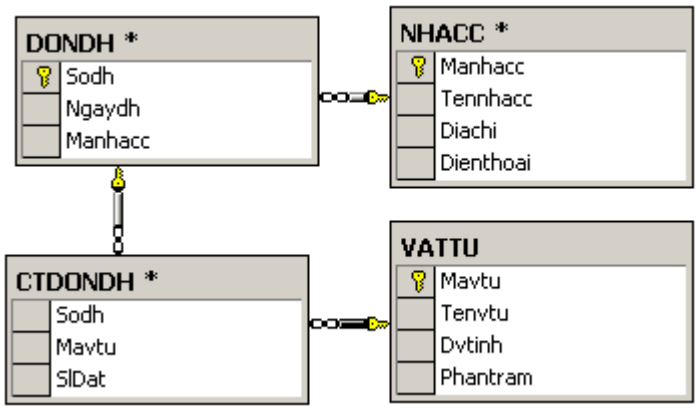
Trong màn hình kết thúc (**Completing**) hệ thống sẽ hiển thị danh sách các bảng mà bạn đã chọn ở bước 3, các bảng này sẽ xuất hiện đầy đủ trong mô hình quan hệ dữ liệu. Nếu bạn thấy còn thiếu bảng nào cần bổ sung thêm thì bạn có thể nhấn nút **Back** để quay lại bước 3. Nhấn nút **Finish** để kết thúc quá trình tạo mới mô hình quan hệ dữ liệu.



Hình 4.9. Kết thúc quá trình tạo mới mô hình quan hệ dữ liệu

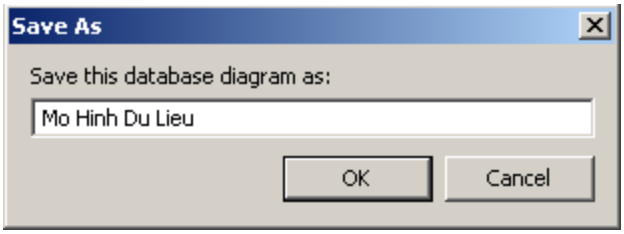
Sau một thời gian ngắn, hệ thống sẽ tự động tạo ra các mối liên kết giữa các bảng đã chọn và sắp xếp vào bên trong mô hình quan hệ dữ liệu.

Bên trong cơ sở dữ liệu Microsoft SQL Server, một mô hình quan hệ dữ liệu đã được tạo lập theo mô hình bên dưới.



Hình 4.10. Mối liên kết giữa các bảng trong mô hình quan hệ dữ liệu

Cuối cùng bạn nhấn vào biểu tượng nút **Save** trên thanh công cụ và gõ vào tên của mô hình quan hệ dữ liệu dùng để lưu lại mô hình quan hệ dữ liệu vừa mới tạo ở các bước trên.



Hình 4.11. Nhập tên mô hình quan hệ dữ liệu

4.6. BÀI TẬP THỰC HÀNH

1. Tạo các bảng dưới đây bằng công cụ trên giao diện hoặc bằng câu lệnh SQL

Bảng danh mục đối tượng (Lưu khách hàng hoặc nhà cung cấp)

Tên bảng: DMDT

Khóa chính: Ma_Dt

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	Id	Int	Trường tăng tự động, xác định bản ghi duy nhất
2	**Ma_Dt**	Nchar(16)	Trường khóa, lưu mã đối tượng
3	Ten_Dt	Nvarchar(96)	Tên đối tượng
4	Dia_Chi	Nvarchar(64)	Địa chỉ
5	So_Dt	char(20)	Số điện thoại
6	Fax	char(20)	Số Fax
7	Email	Nvarchar(64)	Thư điện tử
8	Ghi_Chu	Ntext	Thông tin ghi chú

Bảng danh mục vật tư

Tên bảng: DMVT

Khóa chính: Ma_Vt

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	Id	Int	Trường tăng tự động, xác định bản ghi duy nhất
2	**Ma_Vt**	Nchar(16)	Trường khóa, lưu mã vật tư, hàng hóa
3	Ten_Vt	Nvarchar(96)	Tên vật tư hàng hóa
4	Dvt	Nvarchar(10)	Đơn vị tính
5	Quy_Cach	Nvarchar(96)	Quy cách sản phẩm
6	Ghi_Chu	Ntext	Thông tin ghi chú

Bảng danh mục kho

Tên bảng: DMKHO

Khóa chính: Ma_Kho

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	Id	Int	Trường tăng tự động, xác định bản ghi duy nhất
2	**Ma_Kho**	Nchar(16)	Trường khóa, lưu mã kho
3	Ten_Kho	Nvarchar(96)	Tên kho hàng

4	Ghi_Chu	Ntext	Thông tin ghi chú

Bảng tồn kho đầu kỳ

Tên bảng: TONDK

Khóa chính: Id

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	**Id**	Int	Trường tăng tự động, xác định bản ghi duy nhất
2	Ma_Kho	Nchar(16)	Mã kho tồn
3	Ma_Vt	Nchar(16)	Mã vật tư hàng hóa tồn kho
4	So_Luong	Numeric(15, 3)	Số lượng tồn
5	Don_Gia	Numeric(18, 0)	Giá tồn kho
6	Ghi_Chu	Ntext	Thông tin ghi chú

Bảng chứng từ

Tên bảng: CT

Khóa chính: Stt_Ct

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	**Stt_Ct**	Char(10)	Là trường khóa chính lưu số thứ tự chứng từ
2	Ma_Ct	Char(3)	Mã chứng từ (PX, PN)
3	Nhom_Ct	Tinyint	Nhóm chứng từ (1: Nhập, 2: Xuất)
4	So_Ct	Char(10)	Số chứng từ
5	Ngay_Ct	DateTime	Ngày chứng từ
6	Ma_Dt	Char(16)	Mã khách hàng hoặc nhà cung cấp
7	Ong_Ba	Nvarchar(32)	Người thực hiện chứng từ
8	Dia_Chi	Nvarchar(96)	Địa chỉ
9	Dien_Giai	Nvarchar(128)	Diễn giải chứng từ
10	Dia_Diem	Nvarchar(96)	Địa điểm nhập, xuất hàng

Bảng chứng từ chi tiết

Tên bảng: CTCT

Khóa chính: Stt_Ct, Stt_Dong

Stt	Trường	Kiểu & độ rộng	Diễn giải
1	**Stt_Ct**	Char(10)	Là trường khóa chính lưu số thứ tự chứng từ
2	**Stt_Dong**	Char(3)	Số thứ tự từng dòng chi tiết chứng từ

3	Ma_Vt	Nchar(16)	Mã vật tư hàng hóa
4	So_Luong	Numeric(15, 3)	Số lượng vật tư hàng hóa nhập, xuất
5	Don_Gia	Numeric(18, 0)	Đơn giá
6	Ma_Kho	Numeric(18, 0)	Mã kho nhập, xuất vật tư hàng hóa

2. Thêm các dữ liệu mẫu dưới đây vào bảng tương ứng

Bảng tồn kho đầu kỳ (TONDK)

Id	Ma_Vt	Ma_Kho	So_Luong	Don_Gia	Ghi_Chu
1	TP001	KTP	10	125.000	
2	VT004	KVT	20	75.000	
3	VT003	KVT	4	110.000	

Bảng danh mục đối tượng (DMDT)

Id	Ma_Dt	Ten_Dt	Dia_Chi	So_Dt	Fax	Email
1	N001	Cty TNHH ABC	Hà Nội	04 3640 0119		
3	N002	Cty Cổ phần CB thực phẩm miền bắc	Hà Nội	046 3423 3438		
4	N003	Cty liên doanh Việt Pháp	Hải Phòng	033 6534 391		
5	N004	Cty TNHH một thành viên cấp nước Yên Bái	Yên Bái	02183 543 443		
6	N005	Cty Cổ phần Lạc Hồng	Bắc Ninh	0240 362 552		
7	N006	Cty XNK Á Châu	Hải Dương	0320 3111 411		

Bảng danh mục vật tư, hàng hóa (DMVT)

Id	Ma_Vt	Ten_Vt	Dvt	Quy_Cach
1	TP001	Bánh trứng Custas	Hộp	Hộp 20 cái
2	TP002	Bánh kem xốp	Gói	Gói 200 gram
3	VT001	Bột mì loại 1	Kg	
4	VT002	Hương liệu	Kg	

5	VT003	Bột nở	Kg	
6	TP003	Bánh kem bơ	Gói	Gói 150 gram
7	TP004	Bánh Socola Vinasun	Hộp	Hộp 6 cái
8	VT004	Socola nguyên liệu	Kg	
9	VT005	Sửa ông thọ	Thùng	

Bảng chứng từ (CT)

Stt Ct	Ma_Ct	Nhom_Ct	So_Ct	Ngay_Ct	Ma_Dt	Ong_Ba	Dia_Chi	Dien_Giai
1	PX	1	PX001	31/05/11	N001	Lê Văn Khương	Hà Nội	Xuất hàng hóa bán đại lý
2	PX	1	PX002	01/06/11	N001	Đào Thị Hạnh	Hà Nội	Xuất hàng hóa bán đại lý
3	PN	2	PN001	01/06/11	N002	Tạ Thu Loan	Công ty	Nhập vật tư phục vụ sản xuất
4	PX	1	PX003	01/06/11	N004	Phan Thế Anh	Hà Nội	Xuất bán hàng hóa
5	PN	2	PN002	02/06/11	N005	Tạ Thị Minh	Hà Tây	Nhập nguyên liệu
6	PX	1	PX004	05/06/11	N004	Phan Thế Anh	Hà Nội	Xuất bán hàng hóa

Bảng chi tiết chứng từ (CTCT)

Stt_Dong	Stt_Ct	Ma_Vt	Ma_Kho	So_Luong	Don_Gia
1	1	TP001	KTP	120.000	120,00
2	1	TP002	KTP	23.500	45,00
1	2	TP001	KTP	11.500	35,00
1	3	VT001	KVT	30.000	120,00
2	3	VT002	KVT	120.000	15,00
1	4	TP001	KTP	110.000	75,00
2	4	TP004	KTP	34.000	44,00
1	5	VT004	KVT	75.000	30,00
2	5	VT001	KVT	31.000	32,00

3	5	VT003	KVT	120.000	56,00

Chương 5. TẠO VIEW (BẢNG ẢO)

5.1. KHÁI NIỆM VIEW

Bảng ảo thực chất là một đối tượng mà bên trong nó chỉ lưu trữ duy nhất một câu lệnh **SELECT** dùng để chỉ định các cột, các dòng dữ liệu bên dưới các bảng dữ liệu mà nó chọn lựa ra để hiển thị cho người sử dụng xem hoặc cập nhật.

Với nguyên tắc này, bạn có thể hiển thị ra đúng các thông tin tối thiểu mà người sử dụng cần dùng, không cần thiết phải hiển thị ra tất cả các thông tin hiện đang được lưu trữ bên trong bảng (đáp ứng được tính bảo mật thông tin).

Ngoài ra còn giúp những người sử dụng dễ dàng truy xuất đến các thông tin mà họ đang cần, khi đó đơn giản sẽ thông qua việc thực hiện các truy vấn trực tiếp đến các bảng ảo mà không cần quan tâm các thông tin này đang được lưu trữ trong những bảng dữ liệu nào (đáp ứng được tính dễ sử dụng).

Trong thực tế, bạn thường tạo ra các bảng ảo để lưu trữ các thông tin cho các loại báo cáo đơn giản hoặc dữ liệu của các màn hình nhập liệu phức tạo có liên kết dữ liệu với nhiều bảng khác hoặc các màn hình tra cứu thông tin cho các người sử dụng.

Lưu ý:

Bảng ảo hoàn toàn không lưu trữ dữ liệu một cách riêng lẻ. Các dữ liệu được hiển thị trong bảng ảo sẽ được lấy từ bên dưới dữ liệu của các bảng cơ sở (underlying table) trong cơ sở dữ liệu hiện hành.

Tuy nhiên bạn vẫn có thể cập nhật (thêm, sửa, xóa) dữ liệu trong các bảng ảo như là đang cập nhật dữ liệu trong các bảng cơ sở.

5.2. SỬ DỤNG T – SQL

Cú pháp:

```
CREATE VIEW Tên_bảng_ảo
[(Tên_các_cột)]
[WITH ENCRYPTION]
AS
        Câu_lệnh_SELECT
[WITH CHECK OPTION]
```

Trong đó:

• **Tên bảng ảo:** tên của bảng ảo muốn tạo mới.

• **Tên các cột:** danh sách tên các cột sẽ được sử dụng về sau bên trong bảng ảo khi tham chiếu đến các cột trong bảng ảo. Thông thường được sử dụng trong bảng ảo có sử dụng các hàm tính toán, các biểu thức tính toán, hoặc các cột trùng tên trong các bảng khác nhau.

• **Từ khóa WITH ENCRYPTION:** dùng để mã hóa nội dung câu lệnh **SELECT** bên trong bảng ảo. Không ai có thể biết được nội dung của câu lệnh **SELECT** trong bảng ảo là gì.

• **Câu lệnh SELECT:** câu lệnh truy vấn chọn lựa dữ liệu từ một hoặc nhiều bảng có liên kết để hiển thị dữ liệu trong bảng ảo. Một số từ khóa có trong câu lệnh **SELECT** chuẩn sẽ không được dùng kèm theo trong khi tạo bảng ảo như là: **ORDER BY** (dùng để sắp xếp dữ liệu), **COMPUTE** (thống kê dữ liệu cuối cùng), **COMPUTE BY** (thống kê dữ liệu theo từng nhóm), **SELECT INFO** (sao chép cấu trúc và dữ liệu sang bảng dữ liệu mới). Thông thường bạn nên thực hiện câu lệnh **SELECT** này trước để xem kết quả đúng như mong muốn hay không trước khi đưa nó lồng vào câu lệnh **CREATE VIEW**.

• **Từ khóa WITH CHECK OPTION:** dùng để ngăn cản các thao tác cập nhật dữ liệu (thêm, sửa) tác động trực tiếp vào bảng ảo có làm ảnh hưởng đến dữ liệu đối với các bảng ảo có sử dụng mệnh đề **WHERE** trong câu lệnh **SELECT**.

***Ví dụ**:*

Để tạo bảng ảo đơn giản có tên là vw_DONDH_NHACC dùng để hiển thị thông tin tất cả các cột trong bảng DONDH (đơn đặt hàng) và hai cột địa chỉ và tên nhà cung cấp trong bảng NHACCAP (nhà cung cấp), bạn thực hiện câu lệnh **CREATE VIEW** như sau:

```
CREATE     view vw_DONDH_NHACC1
as
 Select DonDH.*, NhaCCAP.DiaChi As [Địa chỉ],
      NhaCCAP.HoTenNCC as [Tên Nhà cung cấp]
 from DonDh inner join NhaCCAP
           on DonDh.MaNhaCC = NhaCCAP.MaNhaCc
```

Khi muốn xem nội dung của câu lệnh **SELECT** bên trong bảng ảo, bạn có thể sử dụng **Procdure Store** hệ thống có tên sp_helptext để xem, cú pháp của nó khá đơn giản được minh họa trực tiếp bằng ví dụ bên dưới.

***Ví dụ**:*

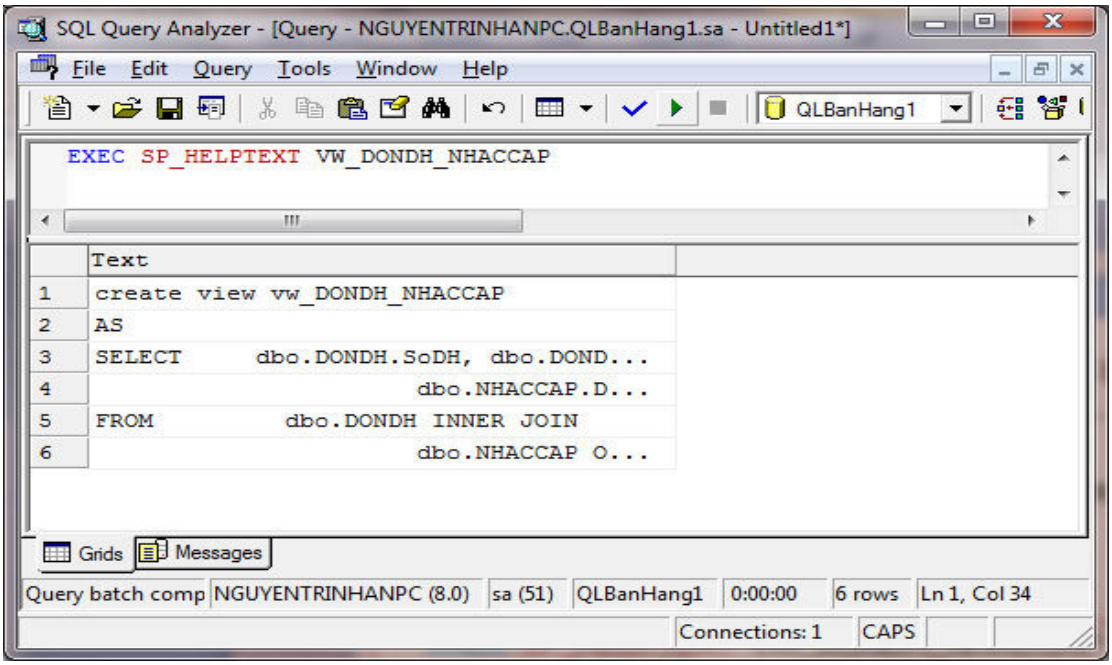
Để xem nội dung câu lệnh **SELECT** của bảng ảo vừa được tạo ở ví dụ trên, bạn gọi thực hiện thủ tục hệ thống sp_helptext như sau:

```
EXEC SP_HELPTEXT VW_DONDH_NHACCAP
```

Hoặc

```
EXEC SP_HELPTEXT VW_DONDH_NHACCAP
```

Kết quả trả về:



Hình 5.1. Kết quả thực thi xem câu lệnh SELECT

Tuy nhiên khi bạn sử dụng từ khóa **WITH ENCRYPTION** bên trong câu lệnh **CREATE VIEW** lúc tạo ra bảng ảo thì khi đó nội dung của câu lệnh **SELECT** sẽ được mã hóa và bạn không thể nào xem được nội dung của câu lệnh **SELECT** cho đến khi phải hủy bỏ và tạo lại bảng bảo bằng lệnh **CREATE VIEW** mà không sử dụng từ khóa **WITH ENCRYPTION**.

Ví dụ:

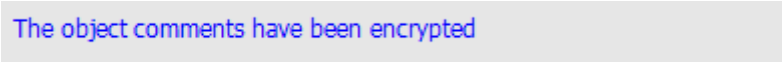
Để tạo bảng ảo có tên là vw_DONDH_NHACC giống ví dụ trên nhưng có bổ sung thêm cột điện thoại và từ khóa **WITH ENCRYPTION** để chủ động mã hóa nội dung câu lệnh **SELECT** bên trong bảng ảo, bạn thực hiện câu lệnh như sau:

```
DROP VIEW VW_DONDH_NHACCAP
GO
create view vw_DONDH_NHACCAP
WITH ENCRYPTION
as
 Select DonDH.*, NhaCCAP.DiaChi As [Địa chỉ], Dienthoai
As [Điện thoại],
      NhaCCAP.HoTenNCC as [Tên Nhà cung cấp]
 from DonDh inner join NhaCCAP
          on DonDh.MaNhaCC = NhaCCAP.MaNhaCc
```

Sau đó thực hiện lại câu lệnh để xem nội dung câu lệnh **SELECT** bên trong bảng ảo.

```
EXEC SP_HELPTEXT VW_DONDH_NHACCAP
```

Kết quả trả về là một thông báo: "Nội dung đối tượng đã được mã hóa".



Khi thực hiện các bảng ảo dùng để tính toán dữ liệu cho các thống kê, bạn thường sử dụng các hàm tính toán SUM, MIN, MAX, COUNT, AVG đi kèm theo câu lệnh **SELECT**. Tuy nhiên dữ liệu của các bảng ảo dạng này chỉ có tính chất thống kê hoặc hiển thị dữ liệu cho người sử dụng xem mà không cho phép các hành động cập nhật dữ liệu trực tiếp ngay trên bảng ảo. Để chỉ định đến các cột dữ liệu tổng hợp này bạn có thể sử dụng hai cách: hoặc đặt tên các cột ngay sau lệnh **CREATE VIEW** hoặc sử dụng bí danh để chỉ tên cột trong câu lệnh **SELECT**.

***Ví dụ**:*

Để tạo một bảng ảo có tên là vw_TINH_TONGSLDAT dùng để tính tổng số lượng đặt hàng của các vật tư. Dữ liệu hiển thị gồm các cột: mã vật tư, tên vật tư và tổng số lượng đặt, bạn thực hiện câu lệnh **CREATE VIEW** để tạo bảng ảo như sau:

```
CREATE VIEW vw_TinhSLDAT(MaVTu, TenVtu, TongSlDat)
 as
      SELECT CTDH.MaVTu, TenVtu, SUM(SlDat)
           From CTDONDH CTDH INNER JOIN VATTU VT
                ON CTDH.MAVTU = VT.MAVTU
           GROUP BY CTDH.MaVtu, tenVTu
```

Hoặc

```
      create view vw_TinhSLDAT
           as
           Select CTDH.MaVTu, TenVtu, SUM(SlDat)As
Tong_sldat
           From CTDONDH CTDH INNER JOIN VATTU VT
                ON CTDH.MAVTU = VT.MAVTU
                GROUP BY CTDH.MaVtu, TenVTu
```

Để hiểu rõ về từ khóa **WITH CHECK OPTION** bên trong câu lệnh **CREATE VIEW**, bạn xem xét các ví dụ bên dưới:

***Ví dụ**:*

Để tạo một bảng ảo có tên là vw_VATTU_TIVI gồm các cột dữ liệu của bảng VATTU: mã vật tư, tên vật tư, đơn vị tính nhưng các dòng dữ liệu chỉ lọc ra các loại hàng ti vi. Bạn thực hiện câu lệnh **CREATE VIEW** như sau:

```
CREATE VIEW vw_VATTU_TIVI
 as
      Select MaVTu, TenVtu, DVTinh
      From VATTU
      WHERE MaVTu Like 'TV%'
Go
```

Kế tiếp bạn sử dụng lệnh **INSERT** để thêm một hàng hóa mới thuộc loại máy hát nhạc vào bảng ảo. Câu lệnh này được thực hiện thành công.

```
INSERT INTO VW_VATTU_TIVI
      VALUES ('MH01','Máy hát Sony đời IK-2002','Bộ')
```

Nhưng khi xem lại dữ liệu của bảng ảo bằng câu lệnh **SELECT** bên dưới bạn sẽ ngạc nhiên vì khi đó sẽ không thấy hiển thị của mẫu tin vừa thêm mới ở trên.

```
SELECT *
FROM VW_VATTU_TIVI
```

Tuy nhiên khi xem dữ liệu của bảng VATTU, bạn vẫn thấy rằng dữ liệu đã được lưu trữ bên dưới bảng VATTU.

```
SELECT *
FROM VATTU
```

Điều này sẽ làm cho một số người sử dụng rất khó hiểu. Lý do mà dữ liệu không được hiển thị bên trong bảng ảo bởi vì mẫu tin mà bạn thêm vào đã không thỏa điều kiện lọc bên trong mệnh đề **WHERE** của câu lệnh **SELECT** trong bảng ảo. (mã của hàng hóa mới thêm vào không bắt đầu bằng chữ TV mà là chữ MH).

Do đó để tránh những hành động có thể làm cho người sử dụng khó hiểu, Microsoft SQL Server cho phép bạn ngăn cản các mẫu tin không thỏa điều kiện **WHERE** được cập nhật vào bảng ảo bằng mệnh đề **WITH CHECK OPTION**. Trở lại ví dụ trên nhưng bây giờ bạn có thêm từ khóa **WITH CHECK OPTION** trong câu lệnh tạo ra bảng ảo.

```
Drop View vw_VatTu_TiVi
go
create view vw_VatTu_TiVi
As
      Select MaVTu, TenVTu, DVTinh
      From VatTu
      Where MaVTu Like 'TV%'
      With Check Option
```

Kế tiếp khi bạn sử dụng lệnh **INSERT** để thêm một hàng hóa mới thuộc thoại máy hát nhạc vào bảng ảo.

```
insert into vw_VatTu_TiVi
      Values('MH02', 'Máy hát Sony đời AK - 2002', 'Bộ')
```

Hoặc sửa đổi mã của hàng hóa ti vi Sony 14 inches thành "TI14’ bằng câu lệnh **UPDATE** như bên dưới.

```
Update vw_VATTU_TIVI
 SET Mavtu = ‘TI14’
 WHERE Mavtu = ‘TV14’
```

Mặc dù dữ liệu và cú pháp các câu lệnh này hoàn toàn đúng nhưng nó sẽ không được thực hiện thành công bởi vì mã vật tư của nó đã vi phạm mệnh đề **WHERE** (không bắt đầu bằng chữ TV).

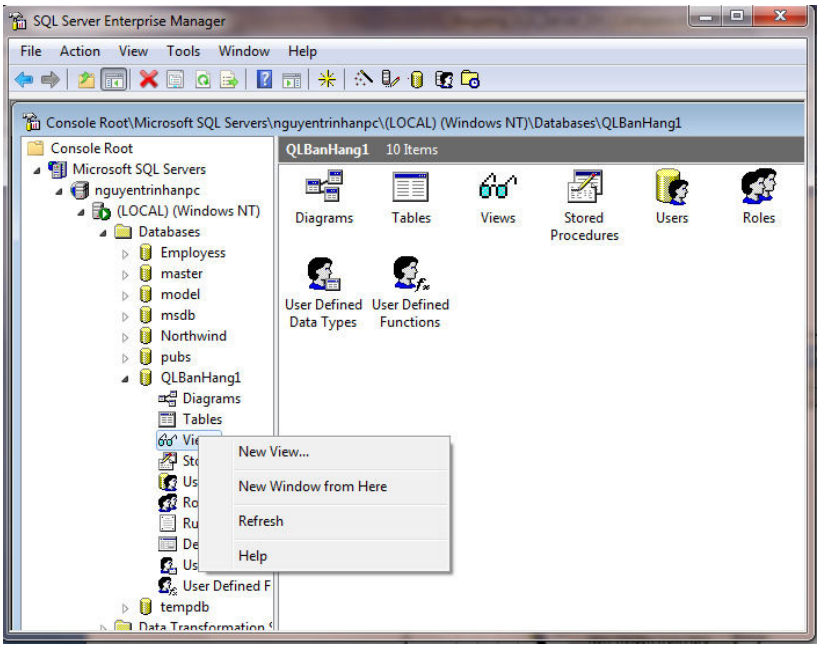
Tóm lại: khi sử dụng mệnh đề **WITH CHECK OPTION** trong câu lệnh **CREATE VIEW** dùng để ngăn cản các hành động cập nhật dữ liệu trong bảng ảo không thỏa điều kiện trong mệnh đề **WHERE** của câu lệnh **SELECT**.

5.3. SỬ DỤNG ENTERPRISE MANAGER

Để tạo bảng ảo, bạn có thể sử dụng tiện ích **Enterprise Manager**. Các bước tạo bảng ảo như sau:

Bước 1:

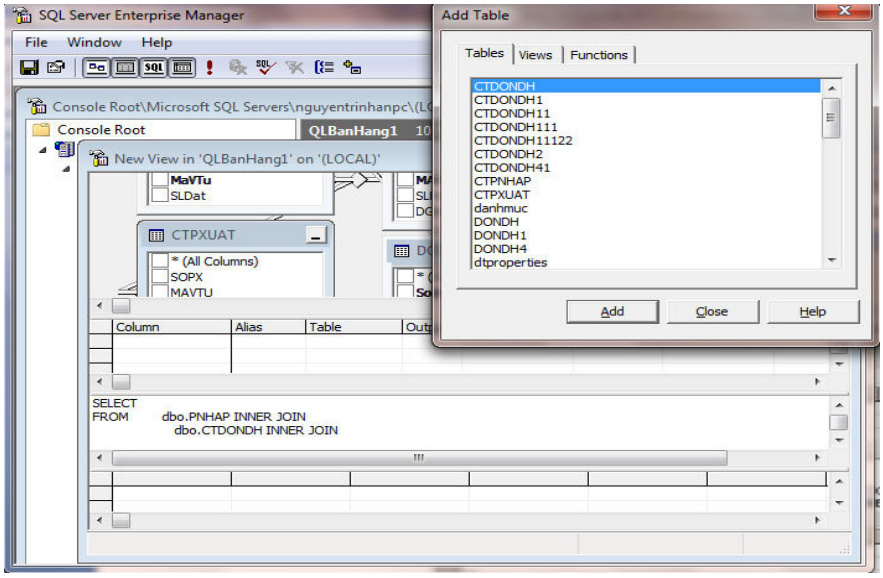
Khởi động tiện ích **Enterprise Manager**. Chọn chức năng **New View …** trong thực đơn tắt sau khi nhấn chuột phải trên đối tượng **Views**.



Hình 5.2. Tạo mới bảng ảo trong Enterprise Manager

Bước 2:

Trong màn hình thiết kế dữ liệu bảng ảo, nhấn vào biểu tượng **Add Table** trên thanh công cụ để đưa các bảng dữ liệu làm dữ liệu nguồn cho bảng ảo.

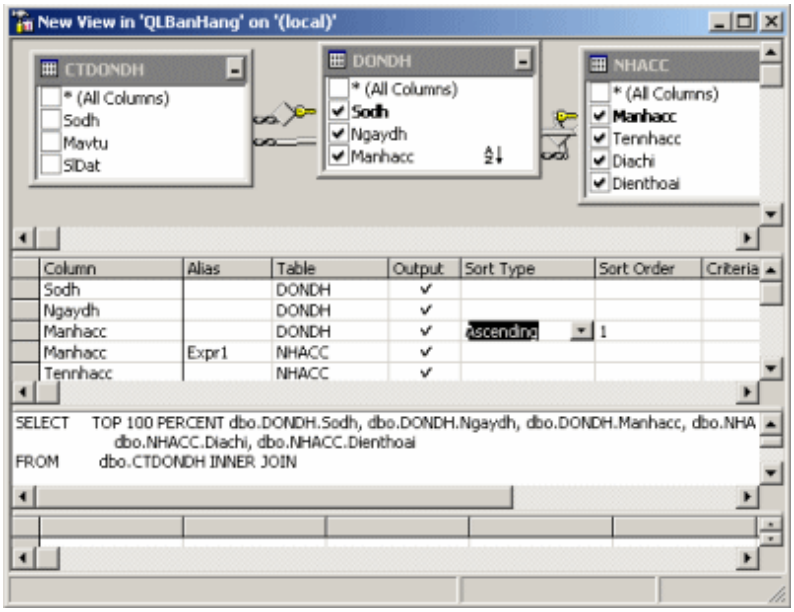


Hình 5.3. Thiết kế dữ liệu bảng ảo

Màn hình này có cách trình bày gần giống như màn hình tạo truy vấn bằng ví dụ (**QBE – Query by Example**) trong Microsoft Access, nó sẽ giúp cho bạn phát sinh nội dung của câu lệnh **SELECT** bằng các hành động kéo thả chuột của chính bạn trên màn hình này.

Bước 3:

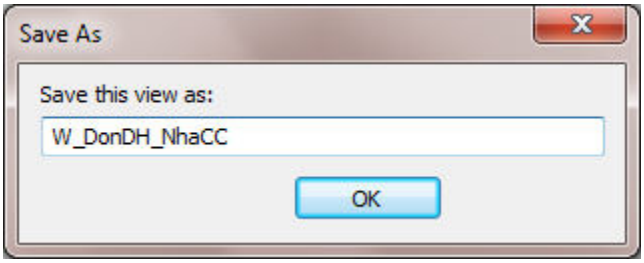
Trong màn hình chọn các dự liệu cho bảng ảo, bằng cách chọn vào ô kiểm tra (check box) phía trước tên của các cột hoặc thao tác kéo thả (drag drop) để chọn các cột muốn hiển thị dữ liệu trong bảng ảo. Sửa lại bí danh (alias) các cột để gợi nhớ.



Hình 5.4. Chỉ định các cột hiển thị dữ liệu bên trong bảng ảo

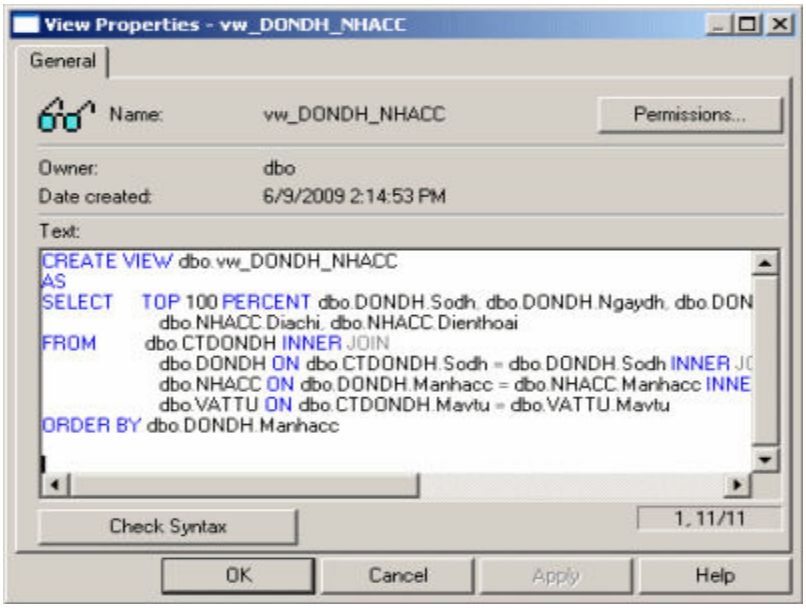
Bước 4:

Nhấn vào biểu tượng **Save** trên thanh công cụ và gõ vào tên của bảng ảo, nhấn **OK** để kết thúc quá trình tạo bảng ảo bằng tiện ích **Enterprise Manager**.



Hình 5.5. Nhập tên bảng ảo muốn lưu lại

Sau khi tạo xong bảng ảo, bạn cũng có thể quay lại để sửa đổi nội dung câu lệnh **SELECT** trong bảng ảo bằng cách chọn chức năng **Design View** để quay lại màn hình thiết kế dữ liệu bảng ảo trước đó hoặc chọn chức năng **Properties** để có thể sửa trực tiếp câu lệnh **SELECT** bên trong bảng ảo. Các chức năng này hiển thị trong thực đơn tắt sau khi nhấn chuột phải trên tên của bảng ảo cần sửa đổi.



Hình 5.6. Hiển thị câu lệnh SELECT trong bảng ảo

5.4. BÀI TẬP THỰC HÀNH

1. Tạo View thẻ kho (TheKhoView) với các thông tin sau đây: Ma_Ct, Nh_Ct, So_Ct, Ngay_Ct, Ma_Vt, Ten_Vt, Ma_Kho, Ten_Kho, So_Luong, Don_Gia, Thanh_Tien lấy dữ liệu từ bảng CT, CTCT và các bảng danh mục liên quan.
2. Tạo View nhật ký chung (NhatKyChungView) với các thông tin sau đây: Ma_Ct, Nh_Ct, So_Ct, Ngay_Ct, Ong_Ba, Dia_Chi, Dien_Giai, Ma_Dt, Ten_Dt, Ma_Vt, Ten_Ct, So_Luong, Don_Gia, Thanh_Tien, Ma_Kho, Ten_Kho. Dữ liệu lấy từ bảng CT, CTCT và các bảng danh mục liên quan.

Chương 6. STORE PROCEDURE

6.1. KHÁI NIỆM STORE PROCEDURE

Giống như ý nghĩa của việc sử dụng thủ tục dùng chung trong những ngôn ngữ lập trình khác, **Procdure Store** trong Microsoft SQL Server dùng để tạo ra những xử lý thường dùng bên trong ứng dụng và nhằm để chia nhỏ các xử lý theo mô hình thiết kế xử lý top-down, một cách làm giúp chia nhỏ một xử lý phức tạp thành các xử lý nhỏ dần, đơn giản hóa các xử lý phức tạp.

6.1.1. Thủ tục nội là gì ?

Procdure Store thật sự là một tập hợp chứa các dòng lệnh, các biến và các cấu trúc điều khiển bên trong ngôn ngữ Transaction-SQL dùng để thực hiện một hành động nào đó, tất cả các nội dung của **Procdure Store** sẽ được lưu trữ tại cơ sở dữ liệu của Microsoft SQL Server.

Các nét đặc trưng của một **Procdure Store** cũng hoàn toàn giống các thủ tục trong các ngôn ngữ lập trình khác: tên **Procdure Store**, tham số truyền giá trị vào và tham số đón nhận giá trị trả ra. Ngoài ra bên trong một **Procdure Store** bạn cũng được phép gọi thực thi một **Procdure Store** khác đã có trước đó. Phạm vi hoạt động của các **Procdure Store** do người dùng tạo ra chỉ có tính cục bộ bên trong một cơ sở dữ liệu lưu trữ thủ tục đó.

Một nét riêng biệt của **Procdure Store** là nó có thể được gọi thực hiện trong môi trường không phải là Microsoft SQL Server. Khi xây dựng giao diện màn hình trên các ngôn ngữ lập trình khác nhau, bạn vẫn có thể gọi thực hiện các **Procdure Store** một cách dễ dàng.

Ngoài ra do **Procdure Store** được lưu trữ vật lý trong cơ sở dữ liệu của Microsoft SQL Server, nên các **Procdure Store** sẽ được thực thi khá nhanh bởi vì nội dung bên trong **Procdure Store** đã được phân tích cú pháp các lệnh khi chúng được tạo mới. Lần đầu tiên khi **Procdure Store** được gọi thực hiện thì nội dung các lệnh bên trong **Procdure Store** sẽ được biên dịch và lưu lại trên bộ nhớ, kể từ các lần kế tiếp thì **Procdure Store** sẽ được thực thi càng nhanh hơn (vì các mã lệnh đã được lưu lại trên bộ nhớ). Đây cũng là một trong những lý do mà tại sao chúng ta nên sử dụng **Procdure Store** trong Microsoft SQL Server để thực thi các xử lý tính toán.

6.1.2. Các Procdure Store hệ thống:

Trong Microsoft SQL Server cung cấp cho chúng ta một số lớn các **Procdure Store** hệ thống dùng để thực hiện các xử lý trong việc quản trị cơ sở dữ liệu. Một **Procdure Store** hệ thống luôn luôn được bắt đầu bằng chữ **sp_** và hầu hết tất cả các **Procdure Store** hệ thống được lưu trữ bên trong cơ sở dữ liệu **Master**.

Việc làm quen và sử dụng nhuần nhuyễn các thủ tục hệ thống sẽ giúp bạn hiểu rõ thêm về cách tổ chức cấu trúc các bảng hệ thống bên dưới Microsoft SQL Server. Ngoài

ra còn giúp bạn biết được các xử lý bí ẩn bên dưới tiện ích Enterprise Manager sau khi bạn thực hiện các hành động chọn lựa trong tiện ích đồ họa này.

***Ví dụ**:*

Có thể tạo mới một người dùng có tên là "TTTH", với mật khẩu "T3HNVC" cho phép truy cập vào cơ sở dữ liệu **NorthWind** của Microsoft SQL Server bằng thủ tục hệ thống **sp_addlogin** như sau:

```
EXEC sp_addlogin 'TTTH', 'T3HNVC','NorthWind'
```

Các bài liên quan đến **Procdure Store** hệ thống chỉ hướng dẫn bạn thực hiện việc triển khai một cơ sở dữ liệu trong Microsoft SQL Server. Do đó sẽ không đề cập đến các **Procdure Store** hệ thống của Microsoft SQL Server mà chỉ trình bày cách thức tạo ra một thủ tục do người dùng định nghĩa để thực hiện các xử lý riêng biệt cho ứng dụng của bạn. Trong các bài kế tiếp, khi đề cập đến cụm từ **Procdure Store** thì xem như nói đến các **Procdure Store** do người dùng định nghĩa.

6.1.3. Các lợi ích khi sử dụng Procdure Store:

Nên xem xét thật kỹ các lợi ích khi sử dụng **Procdure Store** được mô tả bên dưới để có thể áp dụng vào đúng các trường hợp thực tế trong khi xây dựng các ứng dụng theo mô hình khách chủ.

• **Tốc độ xử lý của các Procdure Store sẽ rất nhanh** bởi vì bản thân nội dung của các **Procdure Store** được lưu trữ và thực hiện ngay tại máy chủ. Hơn thế nữa, các dữ liệu cũng được lưu trữ trên cùng máy chủ nên không cần mất thời gian để truyền dữ liệu qua hệ thống mạng cho các xử lý bên trong **Procdure Store** bởi vì các xử lý và dữ liệu cùng được lưu trữ trên cùng một vị trí vật lý là máy chủ.

• **Việc tổ chức và phân chia các xử lý thành hai nơi khác nhau:** tại máy chủ hoặc tại máy trạm sẽ giúp các bạn có thể làm giảm thời gian xây dựng ứng dụng. Bởi vì bản thân một **Procdure Store** trong Microsoft SQL Server có thể được gọi thực thi nhiều lần trên các màn hình khác nhau trong các ngôn ngữ lập trình khác nhau: Visual Basic, C++, Delphi, …

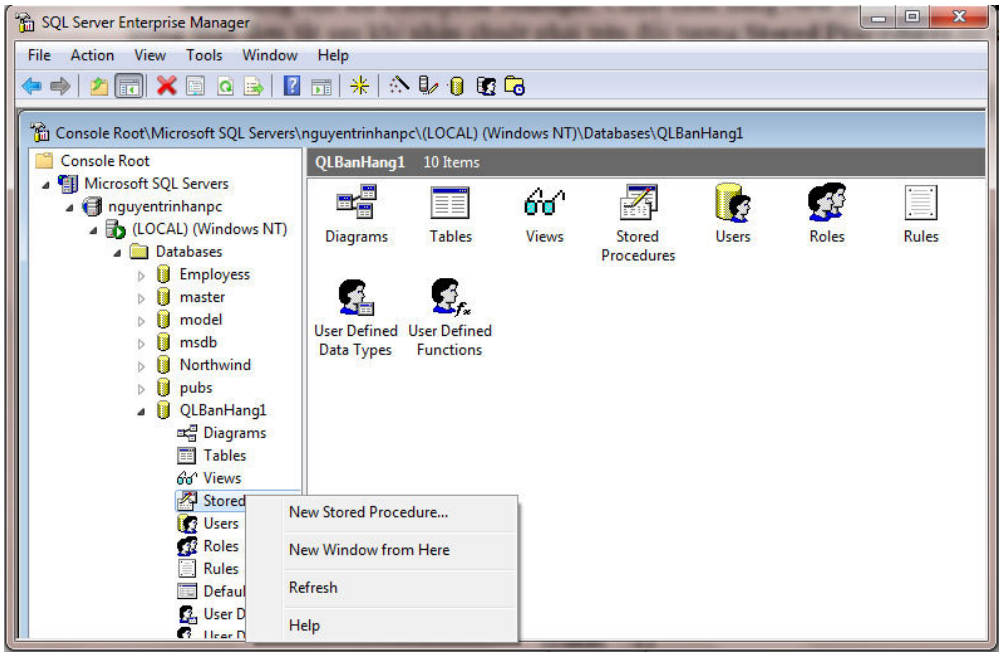
• **Giống như những lợi ích khi sử dụng đối tượng bảng ảo (View)**: bạn có thể sử dụng **Procdure Store** để phân cấp quyền hạn (security) cho những người sử dụng với các hành động cập nhật dữ liệu như: thêm, sửa, xóa dữ liệu hoặc xem chỉ có quyền xem nội dung của các báo cáo.

6.2. TẠO STORE PROCEDURE BẰNG LỆNH CREATE

Giống như các đối tượng khác trong Microsoft SQL Server đã trình bày trước đây, bạn có hai cách để có thể tạo mới một **Procdure Store**. Các bước bên dưới sẽ hướng dẫn các bạn cách thức tạo mới một **Procdure Store** bằng tiện ích Enterprise Manager.

Bước 1:

Khởi động tiện ích Enterprise Manager. Chọn chức năng **New Stored Procedure …** trong thực đơn tắt sau khi nhấn chuột phải trên đối tượng **Stored Procedures** để tạo mới một **Procdure Store** đơn giản.

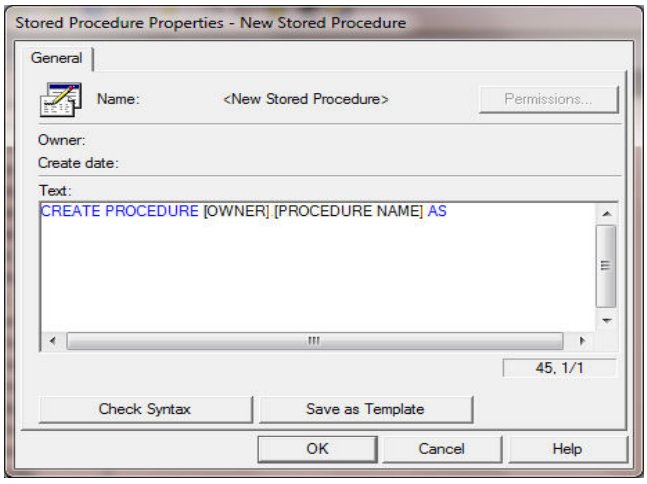


Hình 6.1. Hộp thoại tạo thủ tục trong Enterprise Manager

Bước 2:

Trong màn hình định nghĩa **Procdure Store** mới, lần lượt chỉ định tên của **Procdure Store** và các câu lệnh cần thiết bên trong nội dung của **Procdure Store** dùng để xử lý, tính toán theo một yêu cầu cụ thể nào đó. Nhấn vào nút **Check Syntax** để hệ thống kiểm tra

Cú pháp các lệnh bên trong **Procdure Store** có hợp lệ hay không ? Sau cùng nhấn nút OK để lưu lại nội dung của **Procdure Store** vừa mới tạo.



Hình 6.2. Hộp thoại kiểm tra thủ tục

➢ **Nhận xét:**

Trong màn hình minh họa bên trên phần tên thủ tục và phần nội dung thủ tục được ngăn cách nhau bởi từ khóa **AS**. Xử lý bên trong thủ tục này dùng để tính ra tên của vật tư nào kèm với số lượng với doanh số bán ra là cao nhất.

Ngoài ra bạn cũng có thể tạo mới **Procdure Store** bằng lệnh **CREATE PROCEDURE** trong tiện ích Query Analyzer, cú pháp như sau:

Cú pháp:

```
CREATE PROC[EDURE] Tên_thủ_tục
AS
[DECLARE Biến_cục_bộ]
      Các_lệnh
```

• **Tên thủ tục:** tên **Procdure Store** được tạo mới, tên **Procdure Store** này phải là duy nhất trong một cơ sở dữ liệu.

• **Biến cục bộ:** là những biến cục bộ được sử dụng tính toán tạm thời bên trong thủ tục, những biến này chỉ có phạm vi cục bộ bên trong **Procdure Store**.

• **Các lệnh:** các lệnh bên trong **Procdure Store** dùng để xử lý tính toán theo một yêu cầu nào đó.

***Ví dụ**:*

Để tạo thủ tục tính ra vật tư nào có doanh số bán cao nhất trong tháng 01/2013. Bạn thực hiện câu lệnh **CREATE PROCEDURE** như sau:

```
CREATE PROCEDURE spud_MaxSLVattu_201301
 AS
Declare
 @sTenVTu Char(50),
 @nMaxSL Int
Select
 @sTenVTu = RTrim(TenVTu),
 @nMaxSL = SLXuat
From
 CTPXuat  CTPX
INNER JOIN VATTU VT ON VT.MAVTU = CTPX.MAVTU
JOIN PXUAT PX ON PX.SOPX = CTPX.SOPX
WHERE CONVERT(CHAR(7), NGAYXUAT, 21) = '2013-01'
 AND SLXUAT = (SELECT MAX(SLXUAT)
 FROM CTPXUAT
 INNER JOIN PXUAT ON PXUAT.SOPX = CTPXUAT.SOPX
```

```
    WHERE CONVERT(CHAR(7), NGAYXUAT,21)= '2013-01')
PRINT @sTenVTu + 'có doanh thu số bán cao nhất'
PRINT 'Với số lượng là:' + cast (@nMaxSL as Char(10))
```

Nhận xét:

Trong ví dụ trên, có sử dụng hai biến cục bộ dùng để lưu trữ tên của vật tư có số lượng bán ra nhiều nhất trong tháng 01/2013 sau đó dùng nó để hiển thị ở cuối thủ tục.

6.3. THI HÀNH STORE PROCEDURE

Bạn có thể gọi thực hiện **Procdure Store** bằng lệnh **EXECUTE** trong tiện ích Query Analyzer. cú pháp bên dưới mô tả việc gọi thực hiện một **Procdure Store** đơn giản, không có tham số gởi vào hoặc đón nhận giá trị trả về gì cả.

- **Cú pháp**:

EXEC[UTE] Tên_Thủ_Tục

Trong đó:

• **Tên thủ tục:** tên **Procdure Store** đã được tạo trước đó mà bạn muốn gọi thực thi.

Ví dụ:

Để thực hiện thủ tục spud_MaxSLVattu_201301 đã được tạo ra trong ví dụ trước đó. Bạn thực hiện lệnh **EXECUTE** như sau:

```
EXECUTE spud_MaxSLVattu_201301
```

Kết quả trả về:

```
Đầu VCD Sony 3 đĩa có doanh số bán cao nhất
Với số lượng là : 10
```

6.4. CÁC PHÁT BIỂU ĐIỀU KHIỂN

6.4.1. Biểu thức Case

- **Các câu lệnh truy vấn dữ liệu – Biểu thức CASE**

Biểu thức CASE trong Transaction-SQL vô cùng hữu ích. Hoạt động của biểu thức CASE rất đơn giản chỉ là thực hiện việc so sánh một biểu thức bất kỳ với hàng loạt các giá trị chỉ định trước đó, nếu bạn là người lập trình trong môi trường Visual Basic thì biểu thức CASE của Transaction-SQL gần giống như cấu trúc điều khiển Select Case.

Tuy nhiên biểu thức CASE hoàn toàn không phải là một cấu trúc điều khiển, điều này có nghĩa là nó chỉ được sử dụng lồng vào các câu lệnh khác mà không thể thực hiện đơn lẻ như các cấu trúc điều khiển khác. Biểu thức CASE có thể sử dụng ở hai dạng khác nhau.

- ***Cú pháp CASE dạng đơn giản:***

```
CASE Biểu_thức
      WHEN Giá_trị_1 THEN Biểu_thức_kết_quả_1
      [WHEN Giá_trị_2 THEN Biểu_thức_kết_quả_2
        ...]
      [ELSE Biểu_thức_kết_quả_N]
END
```

Trong đó:

• **Biểu thức:** biểu thức tính toán hoặc tên cột dữ liệu của bảng được dùng để so sánh.

• **Giá trị 1, giá trị 2:** là các giá trị cụ thể để so sánh bằng (=) với biểu thức.

• **Biểu thức kết quả 1, biểu thức kết quả 2:** là các biểu thức sẽ được trả về khi việc so sánh của biểu thức bằng với các giá trị so sánh tương ứng.

• **Biểu thức kết quả N:** là biểu thức sẽ được trả về khi tất cả các trường hợp so sánh đều không bằng với các giá trị đưa ra.

Ví dụ:

Để hiển thị danh sách các vật tư có trong bảng VATTU theo từng loại hàng, có đếm tổng số các vật tư theo từng loại hàng. Bạn sử dụng lệnh SELECT FROM có kết hợp biểu thức CASE đơn giản như sau:

```
SELECT LOAI =
 CASE LEFT(MAVTU, 2)
      WHEN 'DD' THEN 'Đầu DVD'
      WHEN 'VD' THEN 'Đầu VCD'
      WHEN 'TV' THEN 'Ti vi'
      WHEN 'TL' THEN 'Tủ laïnh'
      WHEN 'BI' THEN 'Bia lon'
      ELSE 'Chưa phân loại'
 END, MAVTU, TENVTU, DVTINH
FROM VATTU
ORDER BY LEFT(MAVTU, 2)
COMPUTE COUNT(MAVTU) BY LEFT(MAVTU,2)
```

Kết quả truy vấn trả về:

```
LOAI        MAVTU     TENVTU                  DVTINH
-------------------------------------------------------------------------
Bia lon     BI01      Bia lon Tiger           Thùng
        cnt
        ============
        1
Đầu DVD     DD01      Đầu DVD Hitachi 1 đĩa   Bộ
Đầu DVD     DD02      Đầu DVD Hitachi 3 đĩa   Bộ
        cnt
        ============
        2
Loa thùng   LO01      Loa Panasonic 1000W     Bộ
        cnt
        ============
        1
Tủ lạnh     TL15      Tủ lạnh Sanyo 150 lít   Cái
Tủ lạnh     TL90      Tủ lạnh Sanyo 90 lít    Cái
        cnt
        ============
        2
Tivi        TV14      Tivi Sony 14 inches     Cái
Tivi        TV21      Tivi Sony 21 inches     Cái
Tivi        TV29      Tivi Sony 29 inches     Cái
        cnt
        ============
        3
Đầu VCD     VD01      Đầu VCD Sony 1 đĩa      Bộ
Đầu VCD     VD02      Đầu VCD Sony 3 đĩa      Bộ
        cnt
        ============
        2
(17 row(s) affected)
```

- ***Cú pháp CASE dạng tìm kiếm:***

```
CASE
      WHEN Bt_logic_1 THEN Biểu_thức_kết_quả_1
      [WHEN Bt_logic_2 THEN Biểu_thức_kết_quả_2
        ...]
      [ELSE Biểu_thức_kết_quả_N]
END
```

Trong đó:

• **Biểu thức logic1, biểu thức logic2:** là các biểu thức luận lý dùng để thực hiện các phép so sánh trong biểu thức CASE.

• **Biểu thức kết quả 1, biểu thức kết quả 2:** là các biểu thức sẽ được trả về khi một trong các biểu thức luận lý so sánh có kết quả là đúng.

• **Biểu thức kết quả N:** là biểu thức sẽ được trả về khi tất cả các biểu thức logic so sánh đưa ra đều sai.

***Ví dụ**:*

Để hiển thị danh sách các vật tư có trong bảng VATTU, thông tin bổ sung thêm chuỗi ghi chú, tùy thuộc vào giá trị của cột tỷ lệ phần trăm giá bán. Bạn sử dụng lệnh SELECT FROM có kết hợp biểu thức CASE tìm kiếm như sau:

```
SELECT GHICHU =
 CASE
      WHEN PHANTRAM < 20 THEN 'Lời ít'
      WHEN PHANTRAM BETWEEN 20 AND 40 THEN 'Lời nhiều'
      ELSE      'Rất lời'
 END, MAVTU, TENVTU, DVTINH
FROM VATTU
ORDER BY PHANTRAM
```

Kết quả truy vấn trả về:

GHICHU	TENVTU	DVTINH	PHANTRAM
Lời ít	Loa Panasonic 100W	Bộ	10
Lời ít	Tivi Sony 21 inches	Cái	15
Lời ít	Tivi Sony 29 inches	Cái	15
Lời nhiều	Tủ lạnh Sanyo 90 lít	Cái	20
Lời nhiều	Tivi Sony 14 inches	Cái	20
Lời nhiều	Tủ lạnh Sanyo 150 lít	Cái	25
Lời nhiều	Đầu VCD Sony 1 đĩa	Bộ	30
Lời nhiều	Đầu VCD Sony 3 đĩa	Bộ	30
Lời nhiều	Đầu DVD Hitachi 1 đĩa	Bộ	40
Lời nhiều	Đầu DVD Hitachi 3 đĩa	Bộ	40
Rất lời	Bia lon Tiger	Thùng	60

(11 row(s) affected)

Thực tế thì nhưng người lập trình trong môi tường Transaction-SQL thường sử dụng biểu thức CASE tìm kiếm bởi vì khi đó các biểu thức luận lý mà bạn dùng để so sánh được phép chứa nhiều toán tử so sánh khác nhau, trong khi đó biểu thức CASE đơn giản ở phần trên chỉ cho phép bạn thực hiện phép so sánh bằng (=) trên một biểu thức đơn giản.

***Ví dụ**:*

Để giảm giá bán hàng trong tháng 02/2013 theo quy tắc:

• Nếu số lượng hàng <= 2 thì không giảm giá.

• Nếu số lượng hàng từ 3 đến 10 thì giảm 10%.

• Nếu số lượng hàng > 10 thì giảm 20%.

Bạn sử dụng lệnh UPDATE SET có kết hợp biểu thức CASE tìm kiếm như sau:

```
UPDATE CTPXUAT
SET DGXUAT =
   CASE
        WHEN SLXUAT <=2 THEN DGXUAT
        WHEN SLXUAT BETWEEN 3 AND 10 THEN DGXUAT* 0.9
        ELSE DGXUAT *0.8
   END
FROM CTPXUAT CTPX
   INNER JOIN PXUAT PX ON PX.SOPX = CTPX.SOPX
WHERE CONVERT(CHAR(7), NGAYXUAT, 21) = '2013-02'
```

Tóm lại: biểu thức CASE có thể được phép kết hợp sử dụng trong các câu lệnh SELECT, UPDATE SET, DELETE dùng để biện luận các trường hợp khác nhau của các giá trị dữ liệu bên trong một câu lệnh truy vấn.

6.4.2. Cấu trúc rẽ nhánh IF... ELSE

Với cấu trúc rẻ nhánh, người lập trình có thể chỉ một một hoặc nhiều câu lệnh sẽ được thực hiện khi giá trị của một biểu thức luận lý là đúng hoặc là sai. Cấu trúc rẽ nhánh được phép sử dụng bên trong một lô (batch) các lệnh hoặc bên trong một **Procdure Store**. Cấu trúc rẽ nhánh được phép lồng nhiều cấp bên trong và cấp độ lồng nhau của các cấu trúc rẽ nhánh là không có giới hạn.

- **Cú pháp**:

```
IF Biểu_thức_luận_lý
      Câu_lệnh1 | Khối_lệnh1
[ELSE
      Câu_lệnh2 | Khối_lệnh2]
```

Trong đó:

• **Biểu thức luận lý**: thông thường là một biểu thức so sánh dùng để chỉ ra một điều kiện so sánh nào đó.

• **Câu lệnh 1/Khối lệnh 1**: các lệnh sẽ được thực hiện khi biểu thức luận lý so sánh có giá trị là đúng (True).

• **Câu lệnh 2/Khối lệnh 2**: các lệnh sẽ được thực hiện khi biểu thức luận lý so sánh có giá trị là sai (False).

- **Lưu ý:**

Khối lệnh được đề cập là một tập hợp từ hai câu lệnh trở lên, trong các trường hợp này bắt buộc bạn phải sử dụng các từ khóa BEGIN và END để hình thành một nhóm các câu lệnh trong một khối lệnh.

Cú pháp:

```
BEGIN
      Các_câu_lệnh_trong_khối_lệnh
END
```

Ví dụ:

Tính xem có vật tư nào đã bán ra với số lượng nhiều hơn 4 không ? Nếu có thì in ra danh sách các vật tư đó, ngược lại thì thông báo chưa bán được vật tư nào với số lượng nhiều hơn 4. Bạn sử dụng cú pháp IF... ELSE như sau:

```
IF (SELECT COUNT(*)
 FROM CTPXUAT
 WHERE SLXUAT >4) >0
BEGIN
      PRINT 'Danh sách cacs hàng hóa bán với số lượng
>4'
      SELECT CTPX.MAVTU, TENVTU, SLXUAT
      FROM CTPXUAT CTPX
      INNER JOIN VATTU VT ON VT.MAVTU = CTPX.MAVTU
      WHERE SLXUAT >4
END
ELSE
 PRINT 'Chưa bán hàng nào với số lượng > 4'
```

Kết quả trả về:

```
Danh sách các hàng hóa bán với số lượng > 4
MAVTU  TENVTU                SLXUAT
-------------------------------------------------
DD02   Đầu DVD Hitachi 3 đĩa 5
VD02   Đầu VCD Sony 3 đĩa    10

(2 row(s) affected)
```

- **Nhận xét:** trong ví dụ trên, sử dụng biểu thức luận lý là một biểu thức so sánh có sử dụng truy vấn SELECT COUNT WHERE dùng để đếm các hàng hóa bán ra có số lượng lớn hơn 4. Nếu kết quả đếm > 0 thì in danh sách các vật tư đó ra. Ngược lại thông báo chưa bán hàng hóa với số lượng nhiều hơn 4.

Khi lập trình trong Transaction-SQL, thông thường bạn cần phải kiểm tra dữ liệu có tồn tại bên trong các bảng trước khi thực hiện tiếp các hành động liên quan đến các dòng dữ liệu đó. Cú pháp **IF** có kết hợp từ khóa **EXISTS** dùng để kiểm tra sự tồn tại của các dòng dữ liệu bên trong bảng rất hữu hiệu.

Cú pháp:

```
IF EXISTS (Câu_lệnh_SELECT)
      Câu_lệnh1 | Khối_lệnh1
[ELSE
      Câu_lệnh2 | Khối_lệnh2]
```

Trong đó:

• **Từ khóa EXISTS:** dùng để kiểm tra sự tồn tại các dòng dữ liệu trong câu lệnh truy vấn SELECT sau đó. Kết quả IF trả về đúng (True) khi câu lệnh SELECT trả về ít nhất một dòng dữ liệu, ngược lại thì trả về sai (False).

***Ví dụ**:*

Thực hiện lại ví dụ trên nhưng sử dụng cú pháp **IF EXISTS** dùng để kiểm tra xem đã có hàng hóa nào bán ra với số lượng nhiều hơn 4 chưa.

```
IF EXISTS (SELECT *
 FROM CTPXUAT
 WHERE SLXUAT >4)
BEGIN
 PRINT 'Danh sách các hàng hóa bán với số lượng >4'
 SELECT CTPX.MAVTU, TENVTU, SLXUAT
 FROM CTPXUAT CTPX
      INNER JOIN VATTU VT ON VT.MAVTU = CTPX.MAVTU
 WHERE SLXUAT >4
END
ELSE
 PRINT 'Chưa bán hàng nào với số lượng > 4'
```

***Ví dụ**:*

Để kiểm tra xem có phiếu nhập hàng nào đã lập vào ngày chủ nhật không. Nếu có thì in ra danh sách các phiếu nhập hàng đó. Bạn sử dụng cú pháp **IF** như sau:

```
IF EXISTS (SELECT * FROM PNHAP
 WHERE DATENAME(DW, NGAYNHAP) = 'Sunday')
BEGIN
 PRINT 'Danh sách các phiếu nhập vào ngày chủ nhật'
 SELECT * FROM PNHAP
 WHERE DATENAME(dw, NGAYNHAP) = 'Sunday'
END
```

➢ **Nhận xét:** trong ví dụ trên sử dụng hàm DATENAME với tham số dw (viết tắt của từ day of week, ngày trong tuần) dùng để trả về ngày trong tuần của ngày nhập hàng.

6.4.3. Cấu trúc lặp WHILE

Với cấu trúc lặp, người lập trình có thể chỉ định một hoặc nhiều câu lệnh sẽ được thực hiện lặp lại nhiều lần trong khi giá trị của biểu thức luận lý so sánh vẫn còn đúng. Giống như cấu trúc rẽ nhánh, cấu trúc lặp được phép sử dụng bên trong một lô (batch) các lệnh hoặc bên trong một **Procdure Store**. Giữa cấu trúc rẽ nhánh và cấu trúc lặp không có thứ tự ưu tiên khi chúng lồng vào nhau và cấp độ lồng nhau là không có giới hạn.

Thực tế việc sử dụng cấu trúc lặp WHILE bị giới hạn trong nhiều trường hợp. Bởi vì bản thân các lệnh truy vấn cập nhật dữ liệu như là: SELECT, UPDATE SET, DELETE trong Transaction-SQL đã tự động thực hiện việc lặp từ dòng dữ liệu đầu tiên đến dòng dữ liệu cuối cùng bên trong bảng.

Cấu trúc lặp WHILE thông thường được dùng với các biến có kiểu dữ liệu cursor, cách thức sử dụng biến kiểu dữ liệu cursor sẽ được hướng dẫn trong các phần sau.

- **Cú pháp**:

```
WHILE Biểu_thức_luận_lý
BEGIN
      Các_lệnh_lặp
END
```

Trong đó:

• Biểu thức luận lý: thông thường là một biểu thức so sánh để chỉ các lệnh sẽ được lặp lại trong khi mà giá trị của biểu thức vẫn còn đúng.

• Các lệnh lặp: các câu lệnh được thực hiện bên trong vòng lặp.

Ví dụ:

Để in ra 10 số nguyên dương bắt đầu từ 100. Bạn sử dụng cấu trúc lặp WHILE như sau:

```
DECLARE @SoNguyen int
SET @SoNguyen = 100
While (@SoNguyen < 110)
BEGIN
 PRINT' Số nguyên:' + Convert(char(3), @Songuyen)
 SET @SoNguyen = @SoNguyen +1
END
```

Kết quả trả về:

```
Số nguyên : 100
Số nguyên : 101
Số nguyên : 102
Số nguyên : 103
Số nguyên : 104
Số nguyên : 105
Số nguyên : 106
Số nguyên : 107
Số nguyên : 108
Số nguyên : 109
```

Bạn có thể sử dụng từ khóa BREAK lồng vào cấu trúc lặp WHILE để có thể kết thúc việc lặp của các lệnh bên trong vòng lặp mà không cần xét đến giá trị trả về của biểu thức luận lý dùng để so sánh phía sau từ khóa WHILE phải là sai. Tuy nhiên từ khóa BREAK thường được sử dụng kèm theo với một biểu thức luận lý khác.

***Ví dụ**:*

Thực hiện việc lặp giống ví dụ trên, tuy nhiên muốn rằng vòng lặp sẽ bị kết thúc khi mới in tới số nguyên 105. Bạn sử dụng cấu trúc lặp WHILE như sau:

```
DECLARE @SoNguyen int
SET @SoNguyen = 100
While (@SoNguyen < 110)
BEGIN
 PRINT N' Số nguyên:' + Convert(char(3), @Songuyen)
 IF @SoNguyen = 105
      BREAK
 SET @SoNguyen = @SoNguyen +1
END
PRINT  N'Kết thúc vòng lặp'
```

Kết quả trả về:

```
Số nguyên : 100
Số nguyên : 101
Số nguyên : 102
Số nguyên : 103
Số nguyên : 104
Số nguyên : 105
Kết thúc vòng lặp
```

Bạn cũng có thể sử dụng từ khóa **CONTINUE** lồng vào cấu trúc lặp WHILE để chỉ định các lệnh bên trong vòng lặp ở phía dưới từ khóa CONTINUE tạm thời không thực hiện tiếp, khi đó con trỏ vòng lặp sẽ nhảy về đầu dòng lặp để kiểm tra giá trị của biểu thức

luận lý so sánh là vẫn còn đúng hay không. Tuy nhiên từ khóa CONTINUE thông thường được dùng kèm theo với một biểu thức luận lý khác.

***Ví dụ**:*

Thực hiện việc lặp giống các ví dụ trên, tuy nhiên muốn rằng vòng lặp sẽ in xót số nguyên 105. Bạn sử dụng cấu trúc lặp WHILE như sau:

```
DECLARE @SoNguyen int
SET @SoNguyen = 99
While (@SoNguyen < 110)
BEGIN
 SET @SoNguyen = @SoNguyen +1
 IF @SoNguyen = 105
      CONTINUE
 PRINT  N' Số nguyên:' + Convert(char(3), @Songuyen)
 END
PRINT  N'Kết thúc vòng lặp'
```

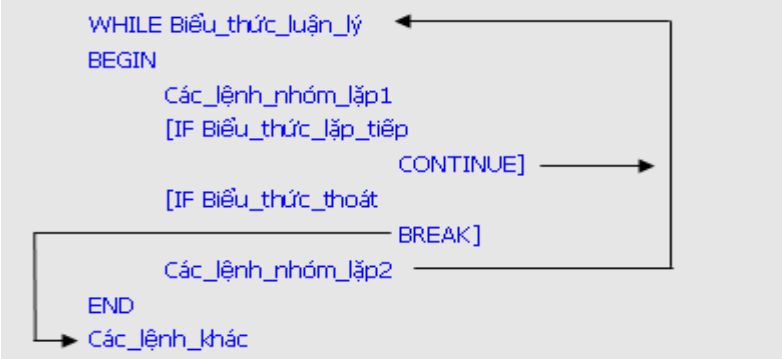
Kết quả trả về:

```
Số nguyên : 100
Số nguyên : 101
Số nguyên : 102
Số nguyên : 103
Số nguyên : 104
Số nguyên : 106
Số nguyên : 107
Số nguyên : 108
Số nguyên : 109
Số nguyên : 110
Kết thúc vòng lặp
```

Kết quả trả về bạn suy luận khi vòng lặp thực hiện đến giá trị của biến @Songuyen = 105 thì khi đó lệnh PRINT "Số nguyên: "... không được thực hiện và con trỏ chương trình được quay lên đầu vòng lặp để kiểm tra tiếp biểu thức luận lý so sánh.

Sơ đồ tóm tắt ý nghĩa của cấu trúc lặp WHILE kèm với các từ khóa CONTINUE hoặc BREAK:

- **Cú pháp**:



***Ví dụ**:*

Để tăng tự động tỷ lệ phần trăm cho các vật tư trong bảng VATTU theo quy tắc sau:

• Mỗi lần chỉ tăng lên 5% cho các vật tư có giá trị tại cột tỷ lệ nhỏ hơn 30%.

• Lặp lại hành động tăng trong khi mà giá trị trung bình tỷ lệ phần trăm của các vật tư vẫn còn thấp hơn 40%.

Bạn sử dụng các lệnh như sau:

```
--In giá trị trung bình trước khi tăng
PRINT 'Trung bình tỷ lệ phần trăm trước khi tăng'
SELECT AVG(PHANTRAM) FROM VATTU
--Bắt đầu thực hiện việc lặp tự động
DECLARE  @Lantang int
SET @LanTang = 0
While (Select AVG(PhanTram) From VATTU) < 40
BEGIN
 UPDATE VATTU
 SET PHANTRAM = PHANTRAM - 5
 WHERE PHANTRAM <30
 SET @Lantang = @Lantang +1
 IF NOT EXISTS (SELECT * FROM VATTU WHERE PHANTRAM
<30)
 BREAK
END
-- In giá trị trung bình tỷ lệ sau khi tăng
PRINT 'Đã tăng:'+ CONVERT(VARCHAR(7), @LanTang) +
'Lần'
PRINT 'Trung bình tỷ lệ phần trăm sau khi tăng '
SELECT AVG(PHANTRAM) FROM VATTU
```

- **Nhận xét:** ví dụ trên khá phức tạp, do thế bạn phải chèn thêm vào các ghi chú (comment) để giúp bạn thấy được từng xử lý rời rạc nhằm dễ xem, dễ hiểu. Trong Transaction-SQL, bạn có thể chèn các ghi chú trong các câu lệnh bằng hai dấu trừ liên tiếp nhau hoặc muốn che lại một khối các lệnh liên tiếp nhau thì bạn sẽ sử dụng cặp ký tự như bên dưới:

```
/*
Các lệnh được che đậy
*/
```

Ngoài ra trong vòng lặp bạn có thể sử dụng lệnh IF EXISTS dùng để kiểm tra trường hợp sau khi đã tăng hết tất cả tỷ lệ phần trăm các vật tư với đều lớn hơn 30 mà trung bình tỷ lệ phần trăm của các vật tư vẫn chưa lớn hơn 40 thì bắt buộc vòng lặp phải được thoát ra ngoài, bởi vì nếu không thì vòng lặp sẽ bị lặp vô tận không bao giờ thoát ra được.

6.5. SỬA, XÓA, ĐỔI TÊN THỦ TỤC (STORE PROCEDURE)

6.5.1. Hủy bỏ Procdure Store:

Khi một **Procdure Store** không còn cần sử dụng nữa thì bạn có thể hủy bỏ nó ra khỏi cơ sở dữ liệu. Cẩn thận khi sử dụng hành động này bởi vì bạn sẽ không có cơ hội phục hồi lại nội dung của thủ tục sau khi đã xóa. Cú pháp lệnh **DROP PROCEDURE** bên dưới cho phép bạn có thể hủy bỏ một **Procdure Store**.

- **Cú pháp**:

```
DROP PROC[EDURE] Tên_Thủ_Tục
```

Trong đó:

• **Tên thủ tục:** tên **Procdure Store** đã được tạo trước đó mà bạn muốn hủy bỏ khi không còn sử dụng nữa.

Ví dụ:

Để xóa thủ tục spud_MaxSLVattu_200201 đã được tạo ra trong ví dụ trước đó, bạn thực hiện lệnh **DROP PROCEDURE** như sau:

```
DROP PROC spud_MaSLVatTu_200201
```

6.5.2. Thay đổi nội dung của Procdure Store:

Đôi khi nội dung của các **Procdure Store** cần phải thay đổi lại để cho các hành động bên trong **Procdure Store** thực hiện được đúng đắn theo các yêu cầu mới. Bạn có hai cách thực hiện việc thay đổi nội dung bên trong của **Procdure Store**:

• Hoặc là dùng lệnh xóa bỏ **Procdure Store** cũ và tạo lại **Procdure Store** mới với nội dung được thay đổi mới phù hợp theo yêu cầu mới.

• Hoặc là dùng lệnh **ALTER PROCEDURE** để thay đổi nội dung cú pháp của lệnh **ALTER PROCEDURE** bên dưới hoàn toàn giống cú pháp của lệnh **CREATE PROCEDURE**.

- **Cú pháp**:

```
ALTER PROC[EDURE] Tên_thủ_tục
AS
      [DECLARE Biến_cục_bộ]
      Các_lệnh
```

***Ví dụ**:*

Cần bổ sung thêm phần kiểm tra dữ liệu đã có bán hàng trong tháng 01/2002 trong **Procdure Store** spud_MaxSLVattu_200201 đã được tạo ra ở ví dụ trước. Bạn thực hiện lệnh **ALTER PROCEDURE** như sau:

```
ALTER PROC spud_MaxSLVatTu_200201 as
DECLARE @sTenVTu varChar(50), @nMaxSL int
 if not Exists (Select MaVTu
          From CTPXuat CTPX
       INNER JOIN PXUAT PX ON PX.SOPX = CTPX.SOPX
       WHERE CONVERT(CHAR(7),NGAYXUAT,21) = '2002-01')
       BEGIN
            PRINT 'Tháng2002/01 chưa có bán vật tư nào cả!'
            Return
       END
       SELECT @sTenVTu = RTrim(TenVTu), @nMaxSL = SLXuat
       FROM CTPXUAT CTPX
            INNER JOIN VATTU VT ON VT.MAVTU = CTPX.MAVTU
            JOIN PXUAT PX ON PX.SOPX = CTPX.SOPX
       WHERE (CONVERT(CHAR(7), NGAYXUAT, 21) = '2002-01')
            AND SLXUAT = (SELECT MAX(SLXUAT)
       FROM CTPXUAT INNER JOIN PXUAT ON PXUAT.SOPX =
CTPXUAT.SOPX
       WHERE CONVERT(CHAR(7), NGAYXUAT, 21) = '2002-01')
       PRINT @sTenVTu + 'có doanh số bán cao nhất'
       PRINT 'Với số lượng là: ' + cast (@nMaxSL as
Char(10))
```

- **Nhận xét:**

Trong ví dụ trên, sử dụng lệnh **RETURN** dùng để thoát ra khỏi **Procdure Store** trong trường hợp khi không có vật tư nào bán ra trong tháng 01/2002. Ý nghĩa của lệnh **RETURN** dùng để thoát khỏi một **Procdure Store**, các dòng lệnh phía sau lệnh **RETURN** sẽ không được thực hiện sau khi thủ tục thực hiện lệnh **RETURN**.

6.6. XỬ LÝ LỖI TRONG STORE PROCEDURE (SP)

Vì các **Procdure Store** ngày càng phức tạp nên việc kết hợp các đoạn lệnh kiểm tra lỗi ở trong các **Procdure Store** là rất cần thiết. Chúng ta có thể sử dụng mã trả về hoặc câu lệnh RAISERROR để

6.6.1. Mã trả về

Trong các **Procdure Store**, các mã trả về chỉ có giá trị số nguyên. Mặc định, giá trị của mã là 0, chỉ ra rằng việc thực hiện **Procdure Store** là thành công. Để xuất ra mã trả về, chúng ta cần khai báo một biến và thực thi một **Procdure Store**. Sau đó chúng ta có thể xem giá trị biến đó. Mã trả về phải trả lại giá trị của chúng vào trong một biến số. Câu lệnh để khai báo biến số và sử dụng nó trong suốt quá trình thực thi thủ tục là:

```
DECLARE @return_variable_name data_type
EXECUTE @return_variable_name = procedure_name
```

Trong đó,

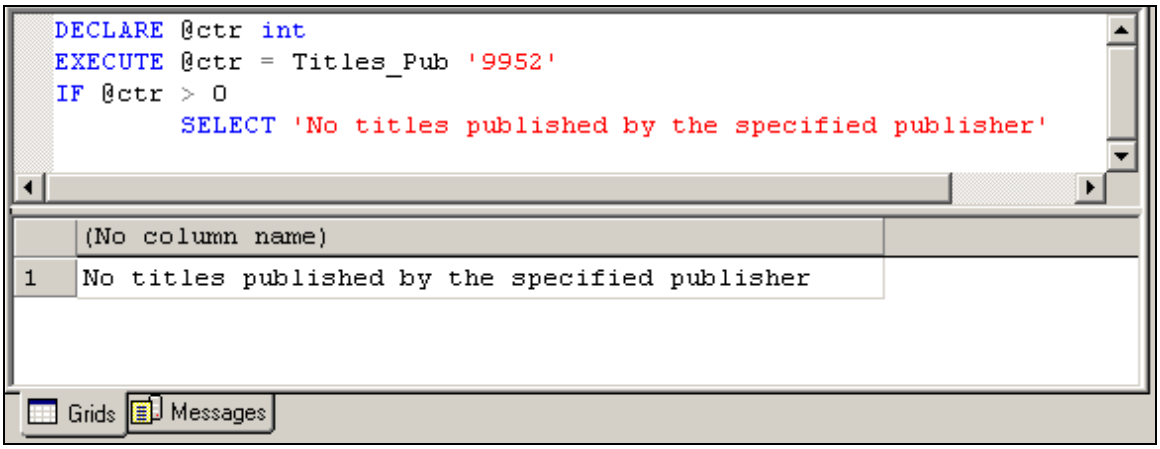
@return_variable_name chứa mã trả về bởi **Procdure Store**.

Ví dụ:

Hãy sửa đổi thủ tục **Titles_Pub** để trả về giá trị 0, giá trị mặc định của SQL Server, nếu nó thực hiện thành công thì hiển thị các hàng. Nếu không có hàng nào được tìm thấy, nó sẽ trả về giá trị 1.
Câu lệnh ALTER PROCEDURE sẽ như sau:

```
ALTER PROCEDURE Titles_Pub
@v_pubid char(4)
AS
     DECLARE @v_return int
     SELECT @v_return=COUNT(*)
          FROM titles WHERE pub_id = @v_pubid
     IF @v_return>0
          SELECT * FROM titles WHERE pub_id = @v_pubid
     ELSE
          RETURN @v_return+1
```

Kết quả thực thi thủ tục được trình bày trong bảng



Hình 6.3. Sử dụng mã trả về

Bởi vì không có tiêu đề nào được tìm thấy của publisher ID '9952', nên thủ tục trả lại giá trị 1 và thông báo tương ứng được hiển thị.

6.6.2. Câu lệnh RAISERROR

Trong một **Procdure Store**, chúng ta có thể sử dụng lệnh PRINT để hiển thị thông báo lỗi do người dùng định nghĩa. Tuy nhiên, các thông báo này chỉ là tạm thời và chỉ hiển thị cho người sử dụng. Câu lệnh RAISERROR được sử dụng để ghi lại những lỗi này và gán cho chúng các cấp độ .Cú pháp là:

```
RAISERROR ({msg_id | msg_str}{, severity, state}
[WITH option[,...n]]
```

Trong đó,

msg_id: Thông báo lỗi cụ thể do người sử dụng định nghĩa trong hệ thống bảng **sysmessages**.

msg_str: Thông báo đặc biệt mà chúng ta đã định nghĩa, nhiều nhất là đến 255 kí tự.

severity: Các cấp độ do người sử dụng định nghĩa liên quan với thông báo đặc biệt. Người sử dụng có thể sử dụng các cấp độ từ 0 đến 18. Các cấp độ từ 19 đến 25 được dành riêng cho các thành viên trong **sysadmin**, và yêu cầu đi kèm tùy chọn WITH LOG . Các cấp độ từ 20-25 được coi là mức cao.

state: Các giá trị từ 1 đến 127 trình bày tình trạng lỗi.

option: Chỉ ra khi nào lỗi sẽ được ghi vào trong nhật ký lỗi của server.

Ví dụ, đoạn mã dưới đây hiển thị thứ tự số bắt đầu từ số 5, theo thứ tự giảm dần. Khi giá trị này là 2, một lỗi được đưa ra và vòng lặp bị đóng.

```
CREATE PROCEDURE Myproc
AS
   DECLARE @v_ctr INT
```

```
SELECT @v_ctr = 5
WHILE @v_ctr > 0
BEGIN
   SELECT @v_ctr * @v_ctr
   SELECT @v_ctr = @v_ctr - 1
   IF @v_ctr = 2
   BEGIN
      RAISERROR('Counter has fallen below 3', 1, 2)
      BREAK
   END
END
```

Kết quả đầu ra của đoạn mã trên là:

```
25
16
9
```

Bảng thông báo sẽ hiển thị ra như sau:

```
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
Msg 50000, Level 1, State 50000
Counter has fallen below 3
```

6.7. BÀI TẬP THỰC HÀNH

1. Viết báo cáo bảng kê chứng từ bao gồm các thông tin: Mã Ct, Số Ct, Ngày Ct, Diễn giải, Mã Đt, Mã Vt, Tên Vt, Đvt, Số lượng, Đơn giá, Thành Tiền.
2. Viết báo cáo tổng hợp nhập kho bao gồm các thông tin: Mã vật tư, Tên Vt, Đvt, Số lượng, Đơn giá, Thành tiền (nhóm theo từng vật tư).
3. Viết báo cáo sổ chi tiết vật tư bao gồm các thông tin: Mã Ct, Số Ct, Ngày Ct, Diễn giải, Mã Vt, Tên Vt, Đvt, Sl Nhập, Tiền nhập, Sl xuất, Tiền xuất, Sl tồn, giá trị tồn (liệt kê chi tiết các lần nhập xuất).
4. Viết báo cáo tổng hợp nhập – xuất – tồn bao gồm các thông tin: Mã vật tư, Tên vật tư, Đvt, Tồn đầu, dư đầu, Sl Nhập, Tiền nhập, Sl Xuất, Tiền Xuất, Tồn cuối, Dư cuối (Nhóm theo từng vật tư).

Chương 7. CHUYỂN TÁC, BẪY LỖI VÀ KIỂU CON TRỎ TRONG SQL SERVER

7.1. CHUYỂN TÁC (TRANSACTION)

7.1.1. Khái niệm

Giao tác trong các loại cơ sở dữ liệu quan hệ lớn được sử dụng trong những trường hợp mà các hành động cập nhật dữ liệu trên nhiều bảng khác nhau được thực hiện trong cùng một đơn vị (unit). Nói một cách khác thì các hành động cập nhật dữ liệu trong một đơn vị sẽ được ghi nhận lại khi tất cả các hành động con bên trong đó thực hiện thành công, ngược lại nếu có ít nhất một hành động nào đó thực hiện thất bại thì tất cả các hành động bên trong đơn vị sẽ bị hủy bỏ để đảm bảo tính toàn vẹn của dữ liệu trong các bảng.

Ví dụ:

Bạn hình dung một khách hàng có cùng lúc 2 loại tài khoản trong ngân hàng. Một là tài khoản thanh toán dùng để thực hiện các giao dịch thu chi qua lại của khách hàng với các công ty khác. Hai là tài khoản tiết kiệm cá nhân của khách hàng cho phép khách hàng gởi tiền tiết kiệm để lấy tiền lãi cuối kỳ theo kỳ hạn 3 tháng.

Giả sử sau thời gian 3 tháng, khách hàng đến ngân hàng để nhận số tiền lãi từ tài khoản tiết kiệm cá nhân. Tuy nhiên khách hàng này muốn bộ phận giao dịch tài khoản thực hiện tự động chuyển số tiền lãi từ tài khoản tiết kiệm sang tài khoản thanh toán của mình.

Nhận xét thấy rằng trong hệ thống chương trình tại ngân hàng phải thực hiện hai hành động cập nhật dữ liệu: một là lấy ra số tiền lãi trong tài khoản tiết kiệm, hai là nạp số tiền lãi vào tài khoản thanh toán. Chuyện gì xảy ra nếu một trong hai hành động thực hiện không thành công mà hành động còn lại vẫn được ghi lại nhận vào cơ sở dữ liệu? Bạn cùng xem xét như sau:

• **Trường hợp 1**: nếu hành động rút số tiền lãi trong tài khoản tiết kiệm thực hiện thành công và hành động nạp số tiền lãi đó vào tài khỏan thanh toán thực hiện bị thất bại thì xem như khách hàng đã mất đi số tiền lãi của tài khoản tiết kiệm (khách hàng mất tiền).

• **Trường hợp 2**: nếu hành động rút số tiền lãi trong tài khỏan tiết kiệm thực hiện thất bại và hành động nạp số tiền lãi đó vào tài khoản thanh toán thực hiện thành công thì xem như khách hàng có thêm số tiền lãi ở cả hai tài khoản (ngân hàng mất tiền).

Nhận xét thấy rằng cả hai trường hợp nêu trên đều làm cho hệ thống vi phạm tính toàn vẹn dữ liệu và có ảnh hưởng đến uy tín chất lượng của ngân hàng. Nhưng nếu nhờ vào khái niệm của giao tác, bạn có thể quy định cả hai hành động trên sẽ được bao bên trong một đơn vị giao tác nhằm nói rằng chúng sẽ được ghi nhận lại khi cả hai hành động con bên trong đó thực hiện thành công, ngược lại nếu trường hợp 1 hoặc trường hợp 2 mô tả ở phần trên có xảy ra thì tất cả các hành động bên trong giao tác sẽ bị hủy bỏ (không ghi lại các thay đổi dữ liệu). Điều này sẽ làm cho hệ thống không vi phạm tính toàn vẹn dữ liệu.

7.1.2. Cách tạo và ứng dụng

7.1.2.1. Giao tác tường minh:

Để bắt đầu một giao tác tường minh, bạn phải sử dụng câu **BEGIN TRAN** trong dòng lệnh đầu tiên của một đơn vị xử lý. Để chỉ định cho Microsoft SQL Server kết thúc

giao tác và ghi nhận lại các hành động cập nhật dữ liệu thì bạn phải sử dụng lệnh **COMMIT TRAN** và ngược lại khi sử dụng lệnh **ROLLBACK TRAN** dùng để chỉ định cho Microsoft SQL Server kết thúc giao tác mà không ghi nhận lại các hành động cập nhật dữ liệu trong giao tác.

7.1.2.2. Lệnh chỉ định bắt đầu một giao tác:

Như phần trên đã trình bày lệnh **BEGIN TRAN** dùng để sử dụng trong các giao tác tường minh. Mỗi giao tác có thể được ghép lồng các giao tác con bên trong đó, bạn có thể chỉ định tên cho từng giao tác lồng nhau nhằm thực hiện dễ dàng việc kết thúc của mỗi giao tác. Biến hệ thống **@@TRANCOUNT** trả về cấp độ lồng hiện hành bên trong các giao tác. Cú pháp lệnh chỉ định bắt đầu một giao tác được mô tả như bên dưới.

- ***Cú pháp:***

```
BEGIN TRAN[SACTION] [Tên_giao_tác]
```

Trong đó:

• **Tên giao tác:** tên của giao tác được chỉ định rõ ràng, chỉ nên sử dụng tên giao tác khi cấp độ lồng nhau của các giao tác nhiều hơn hai cấp.

Ví dụ:

Sử dụng lệnh **BEGIN TRAN** để chỉ định bắt đầu thực hiện giao tác: thêm vật tư mới vào bảng VATTU, tuy nhiên khi kết thúc giao tác bạn không lưu lại vật tư này.

```
SET ANSI_WARNINGS OFF
GO
SELECT COUNT(*) AS 'Tổng vật tư trước khi thêm'
FROM VATTU
BEGIN TRAN
 INSERT INTO VATTU (MAVTU, TENVTU, DVTINH, PHANTRAM)
 VALUES('BU01', 'BÀN ỦI PHILIP', 'CÁI', 17)
 SELECT COUNT(*) AS 'Tổng vật tư sau khi thêm trong  gt'
 FROM VATTU
 ROLLBACK TRAN
 SELECT COUNT(*) AS 'Tổng vật tư hiện tại'
 FROM VATTU
SET ANSI_WARNINGS ON
```

Kết quả trả về:

```
Tổng vật tư trước khi thêm
-----------
11

Tổng vật tư sau khi thêm trong gt
-----------
12

Tổng vật tư hiện tại
-----------
11
```

Nhận xét thấy rằng trong ví dụ này, trước khi thực hiện giao tác, chúng ta có 11 vật tư, sau đó trong giao tác thêm vào một vật tư mới. Tuy nhiên cuối cùng khi kết thúc giao tác chúng ta không ghi lại hành động thêm vật tư bằng lệnh **ROLLBACK TRAN**, do đó tổng số vật tư vẫn là 11 vật tư khi kết thúc giao tác.

7.1.2.3. Các lệnh chỉ định kết thúc một giao tác:

Theo ví dụ trên chúng ta có thể hiểu ý nghĩa của lệnh **ROLLBACK TRAN** dùng để chỉ định kết thúc giao tác nhưng không ghi nhận lại các hành động cập nhật dữ liệu bên trong giao tác.

Ngoài ra chúng ta cỏ thể sử dụng lệnh **COMMIT TRAN** dùng để chỉ định kết thúc giao tác nhưng đồng ý ghi nhận lại các hành động cập nhật dữ liệu bên trong giao tác. Cú pháp của cả hai lệnh này được mô tả như bên dưới.

- ***Cú pháp***:

```
ROLLBACK   TRAN[SACTION] [Tên_giao_tác]
```

Hoặc

```
COMMIT     TRAN[SACTION] [Tên_giao_tác]
```

Trong đó:

• **Tên giao tác:** tên của giao tác được định nghĩa trước đó trong câu lệnh **BEGIN TRAN**.

Ví dụ:

Tạo một bảng tạm dùng để minh họa việc sử dụng các giao tác lồng nhau. Kết thúc giao tác ngoài cùng bằng lệnh **ROLLBACK TRAN** và không ghi nhận lại các hành động cập nhật dữ liệu của các giao tác con trước đó. Điều này có nghĩa là dữ liệu của bảng tạm *#TestTran* là hoàn toàn trống.

```
CREATE TABLE #TestTran(
 CotA INT PRIMARY KEY,
 cotB CHAR(3)
)
```

```
BEGIN TRAN Cap1 --@@TRANCOUNT = 1
 INSERT INTO #TestTran VALUES (1, 'aaa')
 BEGIN TRAN Cap2 --@@TRANCOUNT = 2
      INSERT INTO #TestTran VALUES(2, 'bbb')
           BEGIN TRAN Cap3 --@@TRANCOUNT = 3
                INSERT INTO #TestTran VALUES(3,'CCC')
           COMMIT TRAN Cap3 --@@TRANCOUNT = 2
           GO
      COMMIT TRAN Cap2 --@@TRANCOUNT = 1
 GO
 ROLLBACK TRAN Cap1 --@@TRANCOUNT = 0
 GO
 SELECT *FROM #TestTran
 GO
 DROP TABLE #TestTran
 GO
```

Nhận xét thấy rằng trong ví dụ này tên của các giao tác được sử dụng trong các lệnh **ROLLBACK TRAN** hoặc **COMMIT TRAN** chỉ để giúp cho chúng ta dễ đọc và dễ thấy được cấp độ hiện hành của các giao tác lồng nhau, nó hoàn toàn không có một mối liên hệ gì giữa tên giao tác trong các lệnh **BEGIN TRAN** trước đó.

Việc sử dụng đối tượng **Procdure Store** để cung cấp các dữ liệu, các tính toán trên các màn hình nhập liệu, báo cáo bên trong ứng dụng sẽ làm cho tốc độ các xử lý tại nhánh máy chủ được nhanh hơn trong các ứng dụng mô hình khách chủ.

7.1.2.4. Phân vùng trong giao tác:

Chúng ta có thể chỉ định việc đồng ý ghi nhận hoặc không ghi nhận lại các hành động cập nhật dữ liệu riêng lẻ bên trong một giao tác bằng cách phân chia thành nhiều vùng nhỏ cho các câu lệnh bên trong một giao tác.

Bằng cách này chúng ta chia nhỏ các hành động bên trong giao tác ra thành nhiều phần, tương ứng từng phần nhỏ chúng ta có thể dễ dàng chủ động đồng ý ghi nhận hoặc không ghi nhận lại việc cập nhật dữ liệu. Cú pháp của lệnh **SAVE TRANSACTION** cho phép chúng ta có thể làm được những điều như đã mô tả ở trên.

- ***Cú pháp***:

```
SAVE TRAN[SACTION] [Tên_vùng]
Các_lệnh
```

Trong đó:

• **Tên vùng:** dùng để chỉ định vùng chứa các lệnh cập nhật dữ liệu và tên vùng nên duy nhất trong một giao tác.

• **Các lệnh:** các lệnh được phân chia theo vùng bên trong giao tác.

***Ví dụ**:*

Như ví dụ trên, tuy nhiên chúng ta muốn phân chia lệnh thêm mới mẫu tin thứ nhất và thứ hai trong vùng thứ nhất, lệnh thêm mới mẫu tin thứ ba trong một vùng thứ hai trong cùng một giao tác. Kết thúc giao tác thực hiện ghi nhận lại các lệnh trong vùng thứ nhất nhưng không ghi nhận lại các lệnh trong vùng thứ hai (chỉ có mẫu tin thứ nhất và thứ hai được ghi lại).

```
CREATE TABLE #TestTran(
      CotA INT PRIMARY KEY,
      CotB CHAR(3)
)
BEGIN TRAN
      --Vùng thứ 1
      SAVE TRAN Dong_1_2
      INSERT INTO #TestTran VALUES(1,'AAA')
      INSERT INTO #TestTran VALUES(2, 'BBB')
      --Vùng thứ 2
      SAVE TRAN Dong_3
      INSERT INTO #TestTran VALUES (3, 'CCC')
      ROLLBACK TRAN Dong_1_2
      GO
      SELECT *FROM #TestTran
      GO
      DROP TABLE #TestTran
      GO
```

7.1.2.5. Kiểm lỗi bên trong giao tác:

Thông thường khi làm việc bên trong giao tác, chúng ta sẽ không bao giờ chỉ định rõ ràng việc kết thúc một giao tác bằng các lệnh cụ thể **COMMIT TRAN** hoặc **ROLLBACK TRAN** mà thay vào đó chúng ta sẽ kiểm tra theo một điều kiện quy định trước. Nếu điều kiện này bị sai thì bắt buộc chúng ta sẽ không ghi nhận các hành động cập nhật dữ liệu trong giao tác, ngược lại sẽ đồng ý ghi nhận các hành động đó.

Để làm được điều này, thông thường chúng ta sử dụng giá trị của biến hệ thống **@@ERROR** trong việc kiểm tra để biết kết quả của câu lệnh thực hiện gần nhất là thành công hay thất bại.

Giá trị của biến hệ thống **@@ERROR** trả về bằng không khi câu lệnh gần nhất thực hiện thành công, ngược lại thì trả về giá trị khác không khi câu lệnh gần nhất thực hiện có lỗi.

***Ví dụ**:*

Thực hiện công việc cấp phát số chứng từ tự động cho các bảng DONDH, PNHAP, PXUAT đảm bảo rằng các số này không bị trùng lắp khi cùng lúc có nhiều người sử dụng cùng lập các chứng từ liên quan. Thực hiện từng bước như sau:

Đầu tiên chúng ta xây dựng bảng CAP_SOCTU dùng lưu trữ số chứng từ được cấp kế tiếp cho các bảng, gồm có các cột: tên bảng (tenbang), số chứng từ (soctu), ký tự đầu (kytu). Trong đó cột tên bảng tham gia làm khóa chính.

```
CREATE TABLE CAP_SOCTU
(
 TENBANG CHAR(20),
 SOCTU INT,
 KYTU CHAR(1),
 PRIMARY KEY (TENBANG)
)
```

Kế tiếp lần lượt thêm các dòng dữ liệu vào bảng CAP_SOCTU:

```
INSERT INTO CAP_SOCTU
 VALUES ('DONDH',100,'D')
INSERT INTO CAP_SOCTU
 VALUES ('PNHAP', 100, 'N')
INSERT INTO CAP_SOCTU
 VALUES ('PXUAT', 100, 'X')
```

Sau cùng, chúng ta xây dựng thủ tục cấp số chứng từ tự động đảm bảo không trùng lắp. Có sử dụng việc kiểm tra lỗi khi thực hiện các lệnh trong giao tác.

```
CREATE PROCEDURE SPUD_CAP_SOCTU_KE
      @sTenbang char(20),
      @sSoctuke char(20) output
as
Declare @nError Int, @nRowCount int,
 @nSoctuke int, @sChuoitam char(4),
 @sKytu char(1)
Begin Tran
 -- Tăng số chứng từ kế tiếp
 Update Cap_soctu
 Set Soctu = Soctu +1
 Where TenBang = @sTenbang

 -- Kiểm tra việc tăng có thành công hay không?
 Select @nError = @@Error,
      @nRowcount = @@Rowcount
```

```
    --Nếu có lỗi hoặc không cập được mẫu tin nào
    If (@nError <> 0 or @nRowCount <> 1)
    Begin
        RollBack Tran
        Return -999
    End
    -- Lấy số chứng từ mà chúng ta đã tăng thành công
    Select @nSoctuke = Soctu,
        @sKytu = Kytu
    From Cap_soctu
    Where Tenbang= @sTenBang

    --Kiểm tra việc lấy dữ liệu có thành công không?
    Select @nError = @@Error,
        @nRowCount = @@RowCount
    --Nếu có lỗi hoặc không lấy được mẫu tin nào
    if @nError <> 0 Or @nRowCount <>1
    Begin
        RollBack Tran
        Return -998
    End
    --Tính số chứng từ khi không có lỗi nào hết
    Set @sChuoiTam = LTrim(Str(@nSoctuke))
    set @sSoctuke = @sKytu +
        Replicate('0', 3 - len(@sChuoiTam))+@sChuoiTam
  Commit Tran
  Return 0
go
```

Gọi thực hiện thủ tục trên để có được số chứng từ kế tiếp cho bảng PXUAT.

```
  DECLARE @sSoctu Char(4), @nGttv Int
  EXEC @nGttv = SPUD_CAP_SOCTU_KE 'PXUAT',
   @sSoctu output
  if @nGttv <> 0
```

```
    Print N'Có lỗi cấp chứng từ, xem lại...'
else
    Print N'Số chứng từ mới là: ' + @sSoctu
```

Kết quả trả về:

```
Số chứng từ mới là : X101
```

Tóm lại: việc sử dụng đối tượng **Procdure Store** để cung cấp các dữ liệu, các tính toán trên các màn hình nhập liệu, báo cáo bên trong ứng dụng sẽ làm cho tốc độ các xử lý tại nhánh máy chủ được nhanh hơn trong các ứng dụng mô hình khách chủ.

7.2. BẪY LỖI (TRIGGER)

7.2.1. Khái niệm

Trigger là một trường hợp đặc biệt của store procedure, nó sẽ có hiệu lực khi chúng ta thay đổi dữ liệu trên một bảng dữ liệu cụ thể, hoặc các xử lý làm thay đổi dữ liệu của các lệnh: insert, update, delete. Trigger có thể chứa các lệnh truy vấn từ các bảng khác hoặc bao gồm những lệnh SQL phức tạp.

Một số thuận lợi khi sử dụng trigger:

Trigger chạy một cách tự động: chúng được kích hoạt ngay tức thì khi có sự thay đổi dữ liệu trên bảng dữ liệu. Trigger có thể thực hiện cascade khi việc thi hành có ảnh hưởng đến những bảng liên quan.

Trigger có những hiệu lực ít bị hạn chế hơn so với ràng buộc giá trị nghĩa là có thể ràng buộc tham chiếu đến những cột của những bảng dữ liệu khác.

Khi trigger được kích hoạt bởi 1 lệnh Transact-SQL insert để thêm một bộ mới vào bảng AAA thì bộ mới này được lưu tạm thời vào một bảng tạm có tên là inserted có cùng cấu trúc với bảng AAA. Khi kết thúc trigger này thì bộ dữ liệu mới thật sự lưu xuống CSDL.

Tương tự đối với lệnh delete, các bộ dữ liệu bị xóa sẽ chuyển tạm vào bảng tạm deleted.

7.2.2. Cách tạo và ứng dụng

7.2.2.1. Lệnh tạo Trigger

```
Create Trigger trigger_name on table_name
For [insert, update, delete]
As
Begin
        {Khai báo các biến xử lý}
        {Các lệnh Transact-SQL}
End
```

7.2.2.2. Lệnh xóa Trigger

```
Drop Trigger trigger_Name
```

Ví dụ: Tạo trigger cho thao tác xóa một đầu sách trong bảng Muon.

```
CREATE TRIGGER tg_delMuon ON muon
```

```
FOR delete
AS
Begin
     DECLARE @isbn int, @ma_cuonsach smallint
     SELECT @isbn = isbn, @ma_cuonsach = ma_cuonsach
     FROM deleted
     UPDATE cuonsach
     SET tinhtrang = yes
     WHERE isbn = @isbn AND ma_cuonsach = @ma_cuonsach
End
```

7.3. KIỂU CON TRỎ (SQL CURSOR)

7.3.1. Khái niệm

Nếu giải thích một cách ngắn gọn thì cursor tương tự như recordset hay dataset trong các ngôn ngữ lập trình. Nghĩa là chúng ta trích lọc một số dữ liệu vào bộ nhớ, sau đó có thể lần lượt làm việc với từng bản ghi bằng cách di chuyển lần lượt qua từng bản ghi (Move Next...).

Có 3 loại cursors là Transact-SQL Cursors, API Cursors và Client Cursors. Trong đó Transact-SQL và API thuộc loại Server Cursors nghĩa là cursors được load lên và làm việc bên phía server. Trong khuôn khổ bài học này ta chỉ nghiên cứu Transact-SQL cursors.

7.3.2. Cách tạo và ứng dụng

Transact-SQL cursors được tạo ra trên server bằng các câu lệnh Transact-SQL và chủ yếu được dùng trong stored procedure và trigger. Trước hết hãy xem qua một ví dụ về cursor:

Viết thủ tục sử dụng Cursor để in ra lần lượt danh sách khách hàng, nhà cung cấp trong danh mục đối tượng (thủ tục đặt tên là usp_DmDtPrint).

```
CREATE PROCEDURE [dbo].[usp_DmDtPrint]
AS
BEGIN
  SET NOCOUNT ON;
  DECLARE @_Ma_Dt NVARCHAR(40),
       @_Ten_Dt NVARCHAR(20)s
  DECLARE DmDtCursor CURSOR FOR
  SELECT Ma_Dt, Ten_Dt FROM DmDt
  OPEN DmDtCursor
  FETCH NEXT FROM DmDtCursor INTO @_Ma_Dt, @_Ten_Dt
  WHILE @@FETCH_STATUS = 0
  BEGIN
```

```
     PRINT N'Đối tượng: ' + RTRIM(@_Ma_Dt) + ' - ' +
LTRIM(@_Ten_Dt)
     FETCH NEXT FROM DmDtCursor INTO @_Ma_Dt, @_Ten_Dt
   END
   CLOSE DmDtCursor
   DEALLOCATE DmDtCursor
END
```

Trong ví dụ ở trên ta sẽ trích chọn Ma_Dt và Ten_Dt từ bảng DmDt của Database QLHTK và nạp vào DmDtCursor, sau đó lần lượt in tên của các đối tượng ra màn hình. Để làm việc với một cursor ta cần theo các bước sau:

- Dùng câu lệnh DECLARE CURSOR để khai báo một cursor. Khi khai báo ta cũng phải cho biết câu lệnh SELECT sẽ được thực hiện để lấy dữ liệu.

```
DECLARE DmDtCursor CURSOR FOR
   SELECT Ma_Dt, Ten_Dt FROM DmDt
```

- Dùng câu lệnh OPEN để đưa dữ liệu lên bộ nhớ ảo (memory). Đây chính là lúc thực hiện câu lệnh SELECT vốn được khai báo ở trên.

```
OPEN DmDtCursor
```

- Dùng câu lệnh FETCH để lấy từng hàng data từ record set. Cụ thể là ta phải gọi câu lệnh FETCH nhiều lần. FETCH tương tự như lệnh Move trong ADO recordset ở chỗ nó có thể di chuyển tới lui bằng câu lệnh FETCH FIRST, FETCH NEXT, FETCH PRIOR, FETCH LAST, FETCH ABSOLUTE n, FETCH RELATIVE n nhưng khác ở chỗ là nó lấy data bỏ vào trong variable (FETCH...FROM...INTO variable_name). Thông thường ta FETCH data trước sau đó loop cho tới record cuối của Cursor bằng vòng lặp WHILE bằng cách kiểm tra global variable @@FETCH_STATUS (=0 nghĩa là thành công).

```
FETCH NEXT FROM DmDtCursor INTO @_Ma_Dt, @_Ten_Dt
```

- Khi ta di chuyển tới từng bản ghi ta có thể UPDATE hay DELETE tùy theo nhu cầu (trong thí dụ này chỉ dùng lệnh PRINT)

```
PRINT N'Đối tượng: ' + RTRIM(@_Ma_Dt) + ' - ' +
LTRIM(@_Ten_Dt)
```

- Dùng câu lệnh CLOSE để đóng cursor. Một số tài nguyên (memory resource) sẽ được giải phóng nhưng cursor vẫn còn được khai báo và có thể OPEN trở lại.

```
CLOSE DmDtCursor
```

- Dùng câu lệnh DEALLOCATE để phóng thích hoàn toàn các tài nguyên dành cho cursor (kể cả tên của cursor).

```
DEALLOCATE DmDtCursor
```

Lưu ý: Trong ví dụ ở trên trước khi dùng Cursor chúng ta phải khai báo trước hai biến @_Ma_Dt và @_Ten_Dt để chứa các giá trị lấy được từ cursor.

7.4. BÀI TẬP THỰC HÀNH

1. Hãy thêm trường ModifiedAt vào tất cả các bảng trong Database QLHTK sau đó viết Trigger để lưu lại thời gian cập nhật vào trường này.
2. Hãy viết Trigger đổi mã cho tất cả các danh mục.

Chương 8. QUẢN LÝ BẢO MẬT VÀ NGƯỜI DÙNG TRONG SQL SERVER

8.1. KHÁI NIỆM

Mỗi CSDL có 1 hay nhiều users được chỉ định quyền truy xuất dữ liệu. Người quản trị có thể cấp quyền truy xuất CSDL bằng cách tạo một tài khoản đăng nhập (login) SQL Server cho User, thêm User vào CSDL và gán quyền cho User trên CSDL đó. Bao gồm các loại quyền:

- Quyền truy cập vào SQL Server.
- Quyền truy xuất CSDL.
- Quyền thực hiện trên các đối tượng của CSDL.
- Quyền xử lý dữ liệu.

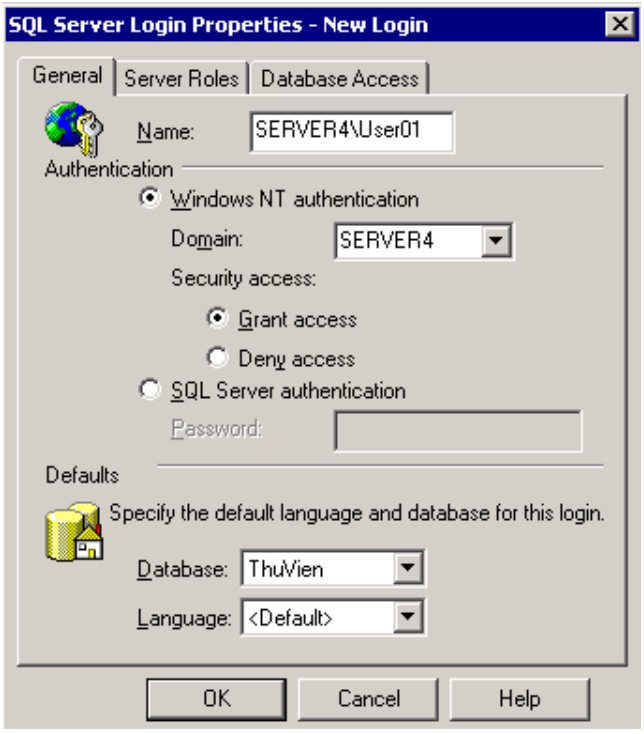
8.2. TẠO VÀ QUẢN LÝ NGƯỜI DÙNG ĐĂNG NHẬP

8.2.1. Sử dụng Enterprise Manager

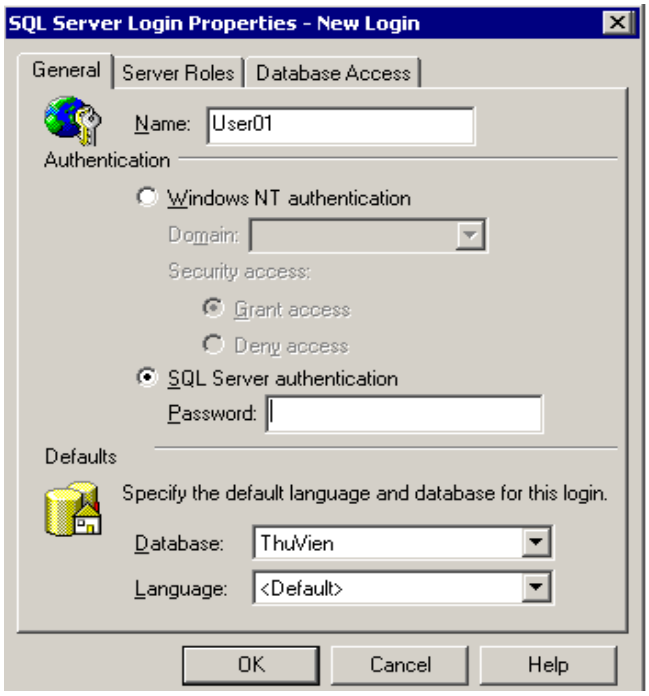
8.2.1.1. Tạo tài khoản đăng nhập (Login Acount):

(1) Mở mục Security, click phải mục Login và chọn New Login…

(2) Trên trang General, nhập tên đăng nhập, chế độ xác nhận, CSDL mặc định.



(Hình 8.1)



(Hình 8.2)

Chú ý: Đăng nhập với SQL Server thường dùng cho việc kết nối quay số và mạng peer-to-peer.

(3) **Trang Server Roles:** Chọn vai trò quản trị mức Server cho tài khoản đăng nhập

System Administrators	Đặc quyền cao nhất; cho phép thực hiện mọi tác vụ trên SQL
Security Administrators	Quản lý các server logins.
Server Administrators	Cho phép bạn định cấu hình những cài đặt server-wide.
Setup Administrators	Cho phép thêm và xóa các linked servers, và truy xuất vài SP
Database Creator	Tạo và hiệu chỉnh databases.
Disk Administrators	Quản lý các files trên đĩa.
Process Administrators	Quản lý tiến trình đang chạy trong một thể hiện của SQL Server.
Bulk Administrators	Thực hiện phát biểu BULK INSERT.

Chú ý: Bất kỳ Users của Windows NT thuộc nhóm BUILTIN\Administrators đều có vai trò **sysadmin**.

(4) **Trang Database Access:** Chọn CSDL được phép truy xuất và vai trò của nó trong từng CSDL được chọn.

Fixed Database	**Role Description**
Public	Vai trò chung cho tất cả người dùng.
db_owner	Quyền cao nhất trong database.
db_accessadmin	Điều khiển truy xuất, cài đặt hoặc xóa user accounts.
db_datareader	Đọc tất cả dữ liệu trên database.
db_datawriter	Thêm, sửa, xóa dữ liệu trên các tables người dùng trong database.
db_addladmin	Thêm, sửa, xóa các đối tượng objects (runs all DDLs).
db_securityadmin	Quản lý các roles, các thành viên của role, giấy phép trong database.
db_backupoperator	Cho phép back up database.
db_denydatareader	Từ chối quyền truy vấy dữ liệu trong database.
db_denydatawriter	Từ chối quyền thay đổi dữ liệu trong database.

Sau khi tạo login, nó tự động nhập vào tập Users của mỗi database được chọn, với tên User trùng với tên Login. Bạn có thể thay đổi thu hồi vai trò của nó trên từng CSDL bằng cách sửa đổi thuộc tính của Login, hoặc chuyển đến tập Users của database và thay đổi thuộc tính hoặc xoá user nào mà bạn không muốn cho truy xuất data của bạn.

Các Login được lưu trong table SysLogins của CSDL Master:

```
If Exists(Select 1 From Master..SysLogins Where Name = 'Login')
```

Các User trong một CSDL được lưu trong table SysUsers của CSDL đó

```
If Exists(Select 1 From SysUsers Where Name = 'User01')
```

8.2.1.2. Thay đổi thuộc tính cho Login:

Bấm đúp vào tên Login hoặc click phải và chọn mục Properties

8.2.2. Sử dụng Query Analyzer:

8.2.2.1. Thêm Tài khoản với chế độ xác nhận SQL Server:

SP_ADDLOGIN [@login =] <'Tênđăngnhập'>
[, @password =] <'password'>
[, @defdb =] <'TênCSDL'>
[, @deflanguage =] <'Ngônngữ'>
[, @sid =] <'Mã nhận dạng Login'>
[, @encriptopt = 'skip_encription']

- @defdb: tên CSDL mặc định được mở khi đăng nhập. Nếu không chỉ định mặc định là CSDL **Master.**
- @deflanguage: Ngôn ngữ mặc định
- @sid: varbinary(16): Mã nhận dạng của hệ thống. Nếu không chỉ định, hệ thống tự tạo một mã số mới.
- @encriptopt varchar(20): Mặc định Password sẽ được mã hóa khi lưu trong các table hệ thống. Ngoại trừ bạn gán giá trị **skip_encryption** cho tham số này.
- Thủ tục trả về giá trị 0 nếu thành công, ngược lại trả về giá trị 1.

Ví dụ: Tạo tài khoản tên 'myname', mật khẩu 'mypwd' vào CSDL 'QLDeTai'

Sp_AddLogin 'user01', '01', 'QLDeTai'

Kết quả trả về: New loigin created

8.2.2.2. Cấp quyền kết nối cho User hay nhóm User của Windows kết nối đến SQL Server:

SP_GRANTLOGIN [@loginame =] 'login'

'Login': <Domain>|<Computer name>\<tênnhóm>

Ví dụ: Thêm tài khoản cho user Windows NT [Server4\User01] kết nối đến SQL Server.

EXEC sp_grantlogin ' Server4\User01'

Hay EXEC sp_grantlogin [Server4\User01]

Chú ý: *Chỉ được thức hiện bởi những thành viên có vai trò* ***sysadmin*** *và* ***securityadmin****.*

8.3. CẤP PHÁT QUYỀN CHO NGƯỜI DÙNG

8.3.1. Sử dụng Enterprise Manager

8.3.1.1. Cấp quyền thực thi trên mỗi CSDL:

Chọn database, trong mục Users bấm đúp vào tên User cần hiệu chỉnh (Login-ID). Click nút Permission để chỉ định quyền truy cập dữ liệu trên từng Table, View. Quyền

kiểm tra RB tham chiếu (DRI - Declarative Referential Integrity). Quyền thực hiện các **Procdure Store** trử.

***Chú thích**: Quyền kiểm tra RB tham chiếu được sử dụng khi Table A được cấp quyền Update hoặc Insert. Table A có RB FOREIGN KEY với table B, mà table B không được cấp quyền SELECT.*

8.3.1.2. Cấp quyền tạo đối tượng trên CSDL:

- Click phải vào tên CSDL, chọn Properties
- Trong HT Properties, chọn trang Permissions
- Đánh dấu chọn các phát biểu được cần cấp quyền thực hiện cho các User.

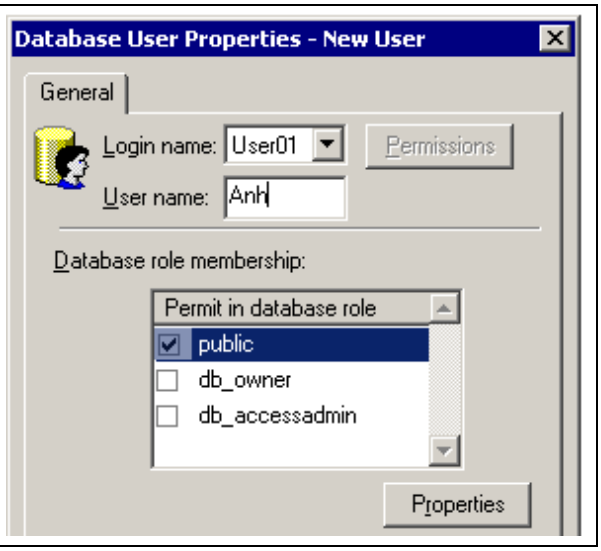
Xóa User trên một CSDL:

Click phải vào tên user và chọn Delete.

Tạo User với Login đã có:

Click phải vào mục **Users** và chọn **New Database User...**

Chú ý: Bạn có thể thay đổi tên User bằng cách xóa và tạo lại với login cũ.



(Hình 10.3)

8.3.1.3. Xóa Tài Khoản Đăng Nhập:

Click phải vào tên Login và chọn Delete

Với cách này sẽ xóa bỏ các user trong các CSDL đã chọn cho tài khoản này.

8.3.2. Sử dụng Query Analyzer

8.3.2.1. Cấp quyền truy xuất CSDL hiện hành cho Login:

Sp_GrantDBAccess [@loginname =] 'Tênđăngnhập'
[[, @name_in_db =] 'TênUser']

Ví dụ:

```
Use QLDeTai
Go
sp_GrantDBAccess 'user01', 'Anh'
```

8.3.2.2. Cấp quyền thực thi trên CSDL:

Bao gồm các quyền: Select, Insert, Update, Delete, Reference, Excecute.

GRANT ALL | <quyền> [,…]

ON <TênTable|View>[(<têncột>,..)] | <tênSP>
TO <tên Login hoặc Role> [,…]
[WITH GRANT OPTION]
[**AS** <tên Role>]

- **All:** Cấp tất cả các quyền thực thi Select, Insert, Update, Delete hay Reference trên table hay view; quyền Excecute cho SP.
- **TO** <tên Login hoặc Role>: Khi cấp quyền cho nhóm hay user của Windows NT, phải chỉ định: <Domain>|<Computer name>\<tênnhóm>.

  Để cấp quyền cho nhóm cục bộ Windows NT built-in, dùng từ khóa BUILTIN thay thế tên domain hoặc computer name.

  Quyền thực thi đã cấp cho role **Public** được áp dụng cho tất cả users trong CSDL. Quyền thực thi đã cấp cho user **Guest** được sử dụng cho tất cả Users không được phân quyền truy xuất trên CSDL.

Ví dụ:
```
USE QLDeTai
Grant All On DeTai To User01
GO
GRANT SELECT ON DeTai TO public
GO
GRANT INSERT, UPDATE, DELETE ON DeTai TO Mary, [
Corporate\BobJ]
GO
```

- **WITH GRANT OPTION:** Cho phép Login cấp quyền đã chỉ định trên đối tượng cho Login khác.
- **AS {group | role}**: Được dùng khi quyền thực thi trên một đối tượng đã cấp cho nhóm hoặc role, và một User của nhóm hoặc Role muốn cấp quyền thực thi cho User khác không là thành viên của nhóm hoặc Role.

Ví dụ:

table NhatKy được tạo bởi user Lac. Lac cấp quyền SELECT table NhatKy cho Role BanBe với mệnh đề WITH GRANT OPTION để các user thành viên của Role BanBe có thể nhường quyền này cho các user khác không thuộc Role BanBe.

User Hong, là thành viên của BanBe, muốn nhường quyền SELECT table NhatKy cho user Khoa, không là thành viên của role BanBe.

```
/* User Lac */
GRANT SELECT ON NhatKy TO BanBe WITH GRANT OPTION
/* User Hong */
GRANT SELECT ON NhatKy TO Khoa AS BanBe
```

8.3.2.3. Cấp Quyền tạo đối tượng trong CSDL:

GRANT ALL | <lệnh> [,…]
TO <tên Login hoặc Role> [,…]

Bao gồm các lệnh: CREATE DATABASE; CREATE DEFAULT; CREATE PROCEDURE; CREATE RULE; CREATE TABLE; CREATE VIEW; BACKUP DATABASE; BACKUP LOG

Ví dụ:
```
GRANT CREATE DATABASE, CREATE TABLE
    TO Mary, John, [Corporate\BobJ]
```

8.3.2.4. Từ Chối quyền thực thi trên CSDL:

DENY ALL | <quyền> [,…]
[(*column* [,...*n*])] ON { *table* | *view* }
| ON *table* | *view* [(*column* [,...*n*])]
| ON *stored_procedure*
TO <tên Login hoặc Role> [,…] [Cascade]

Ví dụ:
```
USE QLDeTai
GO
GRANT SELECT ON DeTai TO public
GO
DENY SELECT, INSERT, UPDATE, DELETE
    ON DeTai TO Mary, John, Tom
```

8.4. THU HỒI QUYỀN ĐÃ CẤP PHÁT CHO NGƯỜI DÙNG

8.4.1. Xóa quyền truy xuất CSDL hiện hành:

Sp_RevokeDBAccess [@name_in_db =] 'TênUser'

Ví dụ:
```
Sp_RevokeDBAccess 'Anh'
```

8.4.2. Xóa bỏ quyền thực thi đã cấp hoặc từ chối trên CSDL:

REVOKE [GRANT OPTION FOR]
{ ALL | *permission* [,...*n*] }
[(*column* [,...*n*])] ON { *table* | *view* }
| ON { *table* | *view* } [(*column* [,...*n*])]
| ON { *stored_procedure* | *extended_procedure* }
| ON { *user_defined_function* }
TO | FROM *security_account* [,...*n*]
[CASCADE]
[AS { *group* | *role* }]

Ví dụ: Xóa bỏ quyền Select đã từ chối cho User Mary trên table DeTai.
```
REVOKE SELECT ON DeTai TO Mary
```

8.4.3. Xóa bỏ quyền tạo đối tượng đã cấp hoặc từ chối trên CSDL:

REVOKE { ALL | *statement* [,...*n*] } **FROM** *security_account* [,...*n*]

Ví dụ: REVOKE CREATE TABLE FROM Joe, [Corporate\BobJ]

TÀI LIỆU THAM KHẢO

[1]. Marcilina S. Garcia, Jamie Reding, Edward Whalen, Steve Adrien DeLuca, *SQL Server 2000 Administrator's Companion*, Microsoft Press, 2000.
[2]. *Microsoft SQL Server 2000 System Administration Training kit*
[3]. Trần Nguyên Phong, *Bài giảng SQL Server* , Đại học khoa học, Đại học Huế.

TRƯỜNG ĐẠI HỌC PHẠM VĂN ĐỒNG
KHOA CÔNG NGHỆ THÔNG TIN

BÀI GIẢNG:

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU 2
SQL SERVER

Giảng viên: Th.S *Nguyễn Trí Nhân*

Quảng Ngãi, 12/2013